

KI-unterstützte Erstellung von ETL-Prozessen - Eine experimentelle Evaluierung

Bachelorarbeit

zur Erlangung des Grades

Bachelor of Science

der Fachrichtung Informatik

an der Dualen Hochschule Baden-Württemberg (DHBW) Stuttgart

von

Luis Geiger

07.09.2026

Matrikelnummer:

9144849

Kurs:

TINF23CE

Betreuer im Unternehmen:

Jens Vierling

Gutachter an der DHBW Stuttgart:

Pawel Wójcik

Erklärung

Ich erkläre hiermit ehrenwörtlich, dass ich die vorliegende Bachelorarbeit mit dem Thema „*KI-unterstützte Erstellung von ETL-Prozessen - Eine experimentelle Evaluierung*“ selbständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Aus den benutzten Quellen, direkt oder indirekt, übernommene Gedanken habe ich als solche kenntlich gemacht. Diese Arbeit wurde bisher in gleicher oder ähnlicher Form oder auszugsweise noch keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht. Ich habe bei der Erstellung der Arbeit KI-Werkzeuge eingesetzt. Dies habe ich in der Anmerkung zur Nutzung von Künstlicher Intelligenz kenntlich gemacht.

Stuttgart, 07.09.2026

Ort, Datum

L.Geiger

Luis Geiger

Abstract

Klassische Extract, Transform, Load (ETL)-Prozesse sind regelbasiert und setzen voraus, dass jede Verarbeitungsvorschrift zuvor vollständig formuliert wird. Die vorliegende Arbeit untersucht experimentell, inwieweit Large Language Models (LLMs) diese Aufgabe übernehmen können. Auf einem synthetischen Datensatz mit kontrolliert eingebauten Qualitätsproblemen werden vier Modelle, drei Prompting-Strategien und neun ETL-Aufgaben gegen eine regelbasiert implementierte Vergleichspipeline gemessen, ergänzt um eine verkettete Ausführung, einen isolierten Durchlauf zur Trennung von eigener Schrittleistung und geerbtem Fehler, einen Vergleich zweier Ausführungsformen sowie zwei Ebenen zum Zuschnitt der Aufgabenstellung. Die geplanten Versuchseinheiten werden von den 711 tatsächlichen Modellaufrufen und den technischen Wiederholungsversuchen getrennt ausgewiesen.

Die Ergebnisqualität hängt weit stärker vom Aufgabentyp ab als vom Modell: Zwischen dem besten und dem schwächsten kommerziellen Modell liegen rund acht Prozentpunkte, zwischen der am besten und der am schlechtesten gelösten Aufgabe nahezu hundert. Als zweite Einflussgröße erweist sich die Ausführungsform. Bei der Länder-Vereinheitlichung erreicht die direkte Verarbeitung höhere Bestwerte als die regelbasierte Referenz, während die Code-Generierung zuverlässiger rechnet und je Zeile rund ein Fünftel kostet. Am stärksten wirkt der Zuschnitt der Aufgabe: Dieselbe Verarbeitung bleibt als eine zusammengesetzte Aufgabe ungelöst und wird in Teilschritte zerlegt fehlerfrei bewältigt, wobei der messbare Unterschied zwischen den Prompting-Strategien dabei auf eine geringe beobachtete Spanne zurückgeht. Die Arbeit leitet daraus eine hybride Architektur ab, die regelbasierte und modellgestützte Schritte je nach Aufgabentyp kombiniert und nach jedem Schritt inhaltlich prüft. Da sie nur für zwei der neun Schritte ein Modell benötigt, senkt sie zugleich Rechenaufwand, Kosten und die an externe Anbieter übertragene Datenmenge: Ergebnisqualität und Ressourcenschonung führen hier zu derselben Gestaltungsentscheidung.

Classical ETL processes are rule-based and require every transformation rule to be specified in advance. This thesis experimentally evaluates to what extent LLMs can take over that task. Using a synthetic dataset with deliberately injected quality defects, four models, three prompting strategies and nine ETL tasks are measured against a rule-based reference pipeline, complemented by a chained execution, an isolated run separating a step's own performance from inherited error, a comparison of two execution modes, and two levels addressing task framing. The evaluation rests on 711 model calls.

Result quality depends far more on task type than on the model: roughly eight percentage points separate the best from the weakest commercial model, whereas nearly a hundred separate the best-solved from the worst-solved task. Execution mode is a second decisive factor: direct processing achieves higher best-case scores for country normalisation than the rule-based reference, while code generation computes more reliably at roughly one-fifteenth of the cost per row under the original pricing scenario. The strongest lever is task framing: the same transformation remains unsolved as one composite task, but is completed without error once decomposed into steps, with

prompting strategy differences shrinking to a narrow range. The thesis derives a hybrid architecture combining rule-based and model-based steps with per-step validation. Requiring a model for only two of the nine steps, it simultaneously reduces computational effort, cost and the volume of data sent to external providers, so that result quality and resource efficiency point to the same design decision.

Inhaltsverzeichnis

Abstract	II
Abbildungsverzeichnis	VII
Tabellenverzeichnis	VII
Skriptverzeichnis	VII
Abkürzungsverzeichnis	VIII
1. Einleitung	1
1.1. Motivation und Problemstellung	1
1.2. Zielsetzung und Forschungsfragen	2
1.3. Methodisches Vorgehen	2
1.4. Aufbau der Arbeit	3
2. Grundlagen	3
2.1. ETL-Prozesse	3
2.1.1. Definition und Phasen	3
2.1.2. Typische Herausforderungen	4
2.1.3. Klassische Werkzeuge und Verfahren	5
2.2. Datenqualität und Datenbereinigung	5
2.2.1. Dimensionen der Datenqualität	5
2.2.2. Häufige Datenprobleme	6
2.3. Künstliche Intelligenz und maschinelles Lernen	6
2.4. Large Language Models	7
2.4.1. Funktionsweise und Architektur	7
2.4.2. Überblick aktueller Modelle	8
2.4.3. Stärken und Limitationen bei strukturierten Daten	8
2.5. Prompt Engineering	9
2.5.1. Grundlegende Prompting-Strategien	9
2.5.2. Einflussfaktoren auf die Ergebnisqualität	9
3. Stand der Forschung	10
3.1. Vorgehen der Literaturrecherche	10
3.2. KI-Einsatz in der Datenverarbeitung	11
3.3. LLMs für strukturierte Daten und synthetische Datensätze	12
3.4. Verwandte Arbeiten zur Automatisierung von ETL	12
3.5. Forschungslücke und Abgrenzung	13

4. Konzeption der experimentellen Evaluierung	14
4.1. Untersuchungsdesign und Hypothesen	14
4.2. Auswahl der Large Language Models	17
4.3. Definition der ETL-Aufgaben	18
4.3.1. Datenbereinigung	18
4.3.2. Datentransformation	19
4.3.3. Duplikaterkennung	19
4.4. Prompting-Strategien	20
4.5. Evaluationskriterien	20
4.5.1. Genauigkeit	21
4.5.2. Vollständigkeit und Konsistenz	22
4.5.3. Effizienz	22
4.6. Vergleich gegen eine regelbasierte ETL-Pipeline	22
5. Erstellung des synthetischen Datensatzes	23
5.1. Anforderungen an den Datensatz	24
5.2. Datenmodell und -struktur	24
5.3. Eingebaute Datenqualitätsprobleme	25
5.4. Generierungsverfahren und verwendete Werkzeuge	27
5.5. Validierung des Datensatzes	28
6. Implementierung	29
6.1. Systemarchitektur und Versuchsumgebung	29
6.2. Eingesetzte Technologien und Bibliotheken	30
6.3. Umsetzung der klassischen ETL-Pipeline	31
6.4. Integration der LLMs in die Pipeline	31
6.5. Umsetzung der Prompting-Strategien	33
6.6. Mess- und Auswertungsverfahren	33
7. Durchführung der Experimente	35
7.1. Versuchsablauf	35
7.2. Reproduzierbarkeit und Versuchsbedingungen	38
8. Ergebnisse und Auswertung	39
8.1. Ergebnisse je ETL-Aufgabe	39
8.1.1. Datenbereinigung	41
8.1.2. Datentransformation	42
8.1.3. Duplikaterkennung	43
8.2. Einfluss der Modellwahl	44
8.3. Vergleich der drei Prompt-Vorlagen	45

8.4. Ergebnisse der verketteten Pipeline	47
8.4.1. Qualitätsverlauf über die Kette	47
8.4.2. Trennung von eigener Leistung und Fehlerfortpflanzung	48
8.5. Reproduzierbarkeit der Ergebnisse	49
8.6. Vergleich der Ausführungsmodi	51
8.7. Zuschnitt der Aufgabenstellung	52
8.7.1. Gruppierte Strecke	52
8.7.2. Explizite statt impliziter Anforderungen	54
8.8. Aufwand und Kosten	55
8.9. Ressourcenverbrauch und Nachhaltigkeitsaspekte	57
8.9.1. Ökologische Dimension	57
8.9.2. Ökonomische und soziale Dimension	58
8.10. Vergleich mit der regelbasierten Pipeline	61
8.11. Stärken, Schwächen und Risiken	62
8.12. Einordnung in den Stand der Forschung	63
8.13. Beantwortung der Forschungsfragen und Hypothesen	64
9. Handlungsempfehlungen für die Praxis	66
9.1. Eignung von LLMs für unterschiedliche ETL-Szenarien	67
9.2. Empfehlungen zur Modell- und Prompt-Auswahl	68
9.3. Hybride Architekturen	70
10. Fazit und Ausblick	72
10.1. Zusammenfassung der Ergebnisse	72
10.2. Limitationen der Arbeit	73
10.3. Ausblick auf weitere Forschung	75
Literatur	77
A. Anhang	83
A.1. Prompt-Vorlagen	83
A.2. Aufgabentexte der neun ETL-Aufgaben	85
A.3. Aufgabentext der explizit gegliederten Variante	87
A.4. Laufzeitumgebung und Modellparameter	88
A.5. Repositorium	88

Abbildungsverzeichnis

6.1. Systemarchitektur und Datenfluss eines Laufs	30
7.1. Laufzahlen je Untersuchungsebene	37
8.1. Bestes Ergebnis je Aufgabe und Ansatz	40
8.2. Modellvergleich über alle Läufe	44
8.3. Prompting-Strategien je Aufgabe	46
8.4. Qualitätsverlauf der Pipeline	47
8.5. Streuung des Endschritts über fünf Wiederholungen	50
8.6. Wirkung des Aufgabenzuschnitts	53
8.7. Wirkung einer expliziten Aufgabenstellung	54
8.8. Aufwand der beiden Ausführungsmodi	56

Tabellenverzeichnis

4.1. Untersuchungsebenen des Versuchsdesigns	16
4.2. Untersuchte Modelle	18
5.1. Beispiele verfälschter Werte im Datensatz	26
5.2. Beispiele eingefügter Duplikate	27
8.1. Ergebnisse je ETL-Aufgabe	41
8.2. Unvollständige Ergebnistabellen im Direkt-Modus	51
8.3. Aufwand je Ausführungsmodus	56
8.4. Nachhaltigkeitsbezogene Befunde	60
9.1. Eignungsmatrix nach Aufgabentyp	68
A.1. Eingesetzte Software- und Bibliotheksversionen	88
A.2. Sampling- und Reasoning-Parameter der Modelle	88

Skriptverzeichnis

8.1. Zwei lauffähige Lösungen derselben Bereinigungsaufgabe	41
---	----

Abkürzungsverzeichnis

ELT Extract, Load, Transform

ETL Extract, Transform, Load

KI Künstliche Intelligenz

LLM Large Language Model

ML Maschinelles Lernen

VRAM Video Random Access Memory

1. Einleitung

1.1. Motivation und Problemstellung

Die Verfügbarkeit, Qualität und zeitnahe Bereitstellung von Daten sind zu einem zentralen Erfolgsfaktor für datengetriebene Organisationen geworden. Im Zentrum der dafür notwendigen Dateninfrastruktur stehen seit Jahrzehnten Extract, Transform, Load (ETL)-Prozesse, die Daten aus heterogenen Quellsystemen extrahieren, in eine einheitliche Struktur überführen und in Zielsysteme wie Data Warehouses oder Analyseplattformen laden [1, 2]. Trotz ihrer langen Etablierung gelten klassische ETL-Pipelines zunehmend als Engpass: Sie sind in der Regel regelbasiert, statisch modelliert und erfordern erheblichen manuellen Aufwand bei Entwurf, Anpassung und Wartung [2, 3]. Mit dem Zuwachs von Datenvolumen, der Heterogenität der Quellen und steigenden Anforderungen an Aktualität geraten herkömmliche ETL-Architekturen an ihre Grenzen [2, 4].

Parallel dazu haben sich in den letzten Jahren Large Language Models (LLMs) als leistungsfähige Werkzeuge etabliert, die neben natürlicher Sprache auch strukturierte und semi-strukturierte Daten verarbeiten können [5]. Arbeiten zeigen, dass LLMs im Kontext der Datenverarbeitung Aufgaben wie Datenbereinigung, Schema-Mapping, Transformationslogik oder Anomalieerkennung übernehmen oder zumindest unterstützen können [6, 7]. Damit eröffnet sich die Perspektive, klassische ETL-Prozesse durch Künstliche Intelligenz (KI)-gestützte Komponenten zu ergänzen oder partiell zu ersetzen, um manuellen Aufwand zu reduzieren und die Anpassungsfähigkeit der Pipelines zu erhöhen. Der praktische Anreiz liegt dabei weniger in der technischen Machbarkeit an sich als vielmehr in der Aussicht auf Effizienz- und Kostenvorteile: Übernehmen Modelle Routineschritte der Transformationslogik ganz oder unterstützend, lässt sich der zeitintensive und damit kostenträchtige manuelle Entwicklungs- und Wartungsaufwand spürbar verringern, was den Einsatz gerade im wirtschaftlichen Kontext motiviert.

Empirische Arbeiten zeigen bereits, dass Sprachmodelle Aufgaben der Datenbereinigung und -integration lösen können. Narayan et al. untersuchen unterschiedliche Datenaufgaben und die Wirkung der Prompt-Gestaltung, Zhang et al. evaluieren mit Jellyfish speziell angepasste lokale Modelle [8][9]. Diese Ergebnisse begründen eine aufgabenbezogene Bewertung, lassen sich jedoch nicht unmittelbar auf jede konkrete ETL-Strecke übertragen. Die vorliegende Arbeit ergänzt diesen Forschungsstand durch den Vergleich ausgewählter Modelle und Prompt-Vorlagen auf einem gemeinsamen synthetischen Datensatz sowie durch zusätzliche Betrachtungen der Ausführungsform und verketteter Verarbeitung.

Die vorliegende Arbeit betrachtet den autonomen Einsatz der Modelle, also die Verarbeitung ohne menschliche Nachbearbeitung des erzeugten Ergebnisses. Der assistierte Einsatz, bei dem eine Fachkraft die Ausgabe prüft, korrigiert und verantwortet, wird nicht experimentell untersucht. Die erhobenen Werte beschreiben daher ausschließlich die Leistung ohne Nachbearbeitung. Wie eine menschliche Prüfung die Qualität und den Gesamtaufwand verändert, bleibt offen, eine garantierte Unter- oder Obergrenze für den assistierten Einsatz lässt sich daraus nicht ableiten.

1.2. Zielsetzung und Forschungsfragen

Ziel dieser Arbeit ist eine experimentelle Evaluierung, inwieweit sich aktuelle LLMs für ausgewählte ETL-Aufgaben eignen, im autonomen Einsatz ohne menschliche Nachbearbeitung. Im Zentrum steht der Vergleich zwischen KI-gestützter und klassischer, regelbasierter Verarbeitung, bewertet anhand von Genauigkeit, Vollständigkeit, Konsistenz und Effizienz bei der Lösung der definierten ETL-Teilaufgaben. Modellwahl und Prompting-Strategie werden als Einflussgrößen mitgeführt, nicht als eigener Leistungsvergleich der Anbieter. Grundlage ist ein kontrolliert erstellter synthetischer Datensatz, der typische Datenqualitätsprobleme abbildet, sodass sich Stärken, Schwächen und praktische Einsatzgrenzen KI-gestützter Ansätze fundiert einordnen lassen. Auf dieser Grundlage sollen Handlungsempfehlungen für den Einsatz von LLMs in realen ETL-Szenarien abgeleitet werden. Aus dieser Zielsetzung ergeben sich zwei leitende Forschungsfragen:

FF1: Inwieweit eignen sich aktuelle LLMs für die Bearbeitung typischer ETL-Teilaufgaben (Datenbereinigung, Datentransformation und Duplikaterkennung) im Vergleich zu einer klassischen, regelbasierten Pipeline? Wie hängt diese Eignung neben dem Aufgabentyp davon ab, ob das Modell Verarbeitungscode erzeugt oder die Daten unmittelbar verarbeitet, und wie die Aufgabe zugeschnitten ist? Welches Vorgehen legen die Ergebnisse je Aufgabentyp nahe?

FF2: Wie stark beeinflusst die Wahl der Prompting-Strategie die Ergebnisqualität der untersuchten LLMs, und welche Strategien sind für die betrachteten ETL-Aufgaben zu empfehlen?

Da LLMs probabilistische Ausgaben erzeugen, wird die Reproduzierbarkeit ihrer Ergebnisse nicht als eigenständige Forschungsfrage, sondern als querschnittliche Untersuchungsdimension über beide Fragen hinweg behandelt: Jede Konfiguration wird mehrfach ausgeführt, sodass die Stabilität der Ergebnisse durchgängig mitbewertet werden kann.

1.3. Methodisches Vorgehen

Die Arbeit folgt einem experimentellen Vorgehen: Nach einer literaturbasierten Fundierung wird ein kontrolliertes Versuchsdesign entwickelt, dessen Ergebnisse gegen eine regelbasiert implementierte ETL-Pipeline als Maßstab gestellt werden. Grundlage aller Messungen ist ein synthetischer Datensatz mit gezielt eingebauten Qualitätsproblemen, dessen Soll-Lösung damit exakt bekannt ist. Gemessen wird auf mehreren Untersuchungsebenen, die nacheinander je einen Einflussfaktor freilegen. Den Anfang macht eine vollständig gekreuzte Faktormatrix aus Modell, Prompting-Strategie und Aufgabe. Es folgen eine verkettete Pipeline aus neun aufeinander aufbauenden Schritten und ein sonst identischer isolierter Durchlauf, der eigene Schrittleistung von geerbtem Fehler trennt. Hinzu kommen ein Vergleich zweier Ausführungsformen, in denen das Modell entweder Verarbeitungscode erzeugt oder die Daten unmittelbar verarbeitet, sowie zwei Ebenen, die den Zuschnitt der Aufgabenstellung selbst zur Untersuchungsgröße machen. Da LLMs probabilistische Ausgaben erzeugen, wird je-

de Konfiguration mehrfach ausgeführt und die Streuung durchgängig mitberichtet. Die Auswertung verbindet die quantitativen Kennzahlen mit einer qualitativen Einordnung beobachteter Fehlermuster, aus der die abschließenden Handlungsempfehlungen abgeleitet werden.

1.4. Aufbau der Arbeit

Die Arbeit gliedert sich in zehn Kapitel. Kapitel 2 führt die Grundlagen zu ETL-Prozessen, Datenqualität, LLMs und Prompt Engineering ein, Kapitel 3 arbeitet den Stand der Forschung auf und präzisiert die Forschungslücke. Kapitel 4 entwirft das Untersuchungsdesign mit Modellauswahl, Aufgabendefinition, Prompting-Strategien, Evaluationskriterien und regelbasierter Vergleichspipeline. Kapitel 5 dokumentiert den synthetischen Datensatz mit seinen eingebauten Qualitätsproblemen und deren Validierung. Kapitel 6 beschreibt die technische Umsetzung von Versuchssystem und Messverfahren, Kapitel 7 legt Versuchsablauf und Versuchsbedingungen offen. Kapitel 8 wertet die Ergebnisse aus, spiegelt sie an den Hypothesen und beantwortet die Forschungsfragen. Daraus leitet Kapitel 9 Handlungsempfehlungen für die Praxis ab, bevor Kapitel 10 zusammenfasst, die Limitationen benennt und einen Ausblick gibt.

2. Grundlagen

Dieses Kapitel legt die theoretischen Grundlagen dar, auf denen die experimentelle Evaluierung der folgenden Kapitel aufbaut. Zunächst werden ETL-Prozesse als etabliertes Verfahren der Datenintegration beschrieben (Abschnitt 2.1). Anschließend wird das Konzept der Datenqualität als zentraler Bezugspunkt für die untersuchten ETL-Teilaufgaben eingeführt (Abschnitt 2.2). Darauf folgt eine Einordnung von Künstlicher Intelligenz und maschinellem Lernen als übergeordnetem Rahmen (Abschnitt 2.3), bevor Large Language Models als die in dieser Arbeit eingesetzte verarbeitende Komponente betrachtet werden (Abschnitt 2.4). Den Abschluss bildet eine Darstellung des Prompt Engineering als zentrale Steuerungsmethode für diese Modelle (Abschnitt 2.5).

2.1. ETL-Prozesse

2.1.1. Definition und Phasen

Der Begriff ETL steht für die drei aufeinanderfolgenden Phasen *Extract*, *Transform* und *Load* und bezeichnet einen der zentralen Prozesse der Datenintegration, mit dem Daten aus unterschiedlichen Quellsystemen in ein einheitliches Zielsystem überführt werden. Vassiliadis hat diese Prozessklasse in einer frühen Übersicht systematisiert und bereits dort darauf hingewiesen, dass der Entwurf und die Wartung solcher Strecken den überwiegenden Teil des Aufwands eines Data-Warehouse-Projekts ausmachen [10]. Diese Einschätzung wird von neueren Arbeiten fortgeschrieben [1, 2]. Historisch ist der ETL-Prozess eng mit dem Aufbau von Data Warehouses verbunden, in de-

nen konsolidierte und bereinigte Daten als Grundlage für Auswertungen und Entscheidungsunterstützung bereitgestellt werden [1, 11]. Der Prozess bildet damit das Bindeglied zwischen unverarbeiteten Rohdaten und den daraus abgeleiteten, nutzbaren Informationen.

In der ersten Phase, der Extraktion, werden die relevanten Daten aus einer oder mehreren Quellen ausgelesen. Diese Quellen sind in der Praxis heterogen und reichen von relationalen Datenbanken über Dateien und Tabellen bis hin zu Schnittstellen externer Dienste, sodass bereits hier unterschiedliche Formate und Strukturen zusammengeführt werden müssen [2].

Die Transformationsphase bildet den inhaltlichen Kern des Prozesses: Die extrahierten Daten werden bereinigt, vereinheitlicht und in die vom Zielsystem geforderte Struktur überführt. Typische Operationen sind die Korrektur fehlerhafter oder fehlender Werte, die Vereinheitlichung von Formaten, die Zusammenführung und Aggregation von Datensätzen, das Auflösen von Duplikaten sowie das Abbilden der Quellstrukturen auf das Zielschema [1, 2, 4]. In der abschließenden Ladephase werden die transformierten Daten schließlich in das Zielsystem geschrieben, etwa in ein Data Warehouse oder eine Analyseplattform, wobei zwischen vollständigem Laden und inkrementellem Aktualisieren unterschieden wird [1, 11]. Ob die Transformation vor dem Laden oder, wie beim Muster Extract, Load, Transform (ELT), erst im Zielsystem erfolgt, ist für die vorliegende Arbeit ohne Belang, da der Schwerpunkt auf den Transformationsschritten selbst liegt [2, 4].

2.1.2. Typische Herausforderungen

Obwohl ETL-Prozesse seit Jahrzehnten etabliert sind, gelten sie in modernen Datenlandschaften zunehmend als anspruchsvoll und wartungsintensiv. Diese Einschätzung gründet auf mehreren miteinander verbundenen Herausforderungen. Eine zentrale davon ist die Sicherung der Datenqualität: Da die Eingangsdaten aus unterschiedlichen Quellen stammen, weisen sie häufig Fehler, Lücken und Inkonsistenzen auf, die im Transformationsschritt erkannt und behoben werden müssen, um verlässliche Analyseergebnisse zu gewährleisten [1, 12]. Eng damit verbunden ist die Heterogenität der Quellen, die sich in unterschiedlichen Formaten, Schemata und Bedeutungen gleichartiger Felder äußert und einen erheblichen Aufwand bei der Vereinheitlichung verursacht [1, 2].

Hinzu kommt die Frage der Skalierbarkeit. Mit dem stetigen Wachstum von Datenvolumen und -vielfalt geraten klassische, batch-orientierte ETL-Architekturen an ihre Grenzen und können den Anforderungen an Verarbeitungsgeschwindigkeit und Aktualität nur noch eingeschränkt gerecht werden [2, 4]. Ein weiterer kritischer Aspekt ist der hohe manuelle Aufwand bei Entwurf, Anpassung und Wartung: Klassische Pipelines sind in der Regel regelbasiert und statisch modelliert, sodass jede Änderung an den Quellstrukturen oder den fachlichen Anforderungen einen manuellen Eingriff erfordert [2, 3]. Diese geringe Anpassungsfähigkeit führt dazu, dass ein Großteil der Arbeitszeit von Datenfachleuten in die Erstellung und Pflege solcher Pipelines fließt, was sie zu einem häufig benannten Engpass datengetriebener Organisationen macht [4, 13]. Genau diese Schwachstellen motivieren die in dieser Arbeit untersuchte Frage, inwieweit

lernende und sprachbasierte Verfahren den manuellen Aufwand reduzieren und die Flexibilität klassischer ETL-Prozesse erhöhen können.

2.1.3. Klassische Werkzeuge und Verfahren

Zur Umsetzung von ETL-Prozessen hat sich über die Jahre ein breites Spektrum an Werkzeugen und Verfahren herausgebildet. Auf der einen Seite stehen dedizierte ETL-Plattformen mit grafischer Modellierung wie Talend, Pentaho oder Informatica, die den Prozess über vorkonfigurierte Komponenten abbilden und sich vor allem in betrieblichen Data-Warehouse-Umgebungen etabliert haben [1, 11]. Auf der anderen Seite werden ETL-Strecken zunehmend programmatisch umgesetzt, etwa mit der Python-Bibliothek Pandas für die Verarbeitung tabellarischer Daten oder mit verteilten Frameworks wie Apache Spark für große Datenmengen [1, 2].

Allen klassischen Ansätzen ist gemeinsam, dass die Verarbeitungslogik regelbasiert und explizit vorgegeben wird: Entwicklerinnen und Entwickler definieren die Transformations- und Bereinigungsregeln manuell, etwa in Form von Bedingungen, Zuordnungstabellen oder Skripten [11, 13]. Dieser Ansatz bietet den Vorteil hoher Nachvollziehbarkeit und deterministischer, reproduzierbarer Ergebnisse, stößt jedoch dort an Grenzen, wo Datenprobleme vielfältig, kontextabhängig oder schwer im Voraus zu formalisieren sind [2, 3]. In der vorliegenden Arbeit dient eine solche regelbasiert implementierte Pipeline als Vergleichsmaßstab, gegen den die untersuchten LLM-gestützten Verfahren gemessen werden.

2.2. Datenqualität und Datenbereinigung

2.2.1. Dimensionen der Datenqualität

Datenqualität ist ein mehrdimensionales Konstrukt, das beschreibt, inwieweit ein Datenbestand für seinen vorgesehenen Verwendungszweck geeignet ist. Diese Bestimmung als *fitness for use* geht auf Wang und Strong zurück, die Datenqualität erstmals empirisch aus der Sicht der Datennutzenden erhoben und daraus eine hierarchische Ordnung von Qualitätsdimensionen abgeleitet haben, statt sie allein an der Übereinstimmung mit der Realität zu messen [14]. Aufbauend darauf hat Batini et al. in einer vergleichenden Übersicht der etablierten Bewertungs- und Verbesserungsmethoden herausgearbeitet, welche Dimensionen methodenübergreifend wiederkehren [15]. Den so entstandenen Kern bilden insbesondere Genauigkeit, Vollständigkeit, Konsistenz, Aktualität und Eindeutigkeit.

Die Genauigkeit beschreibt, inwieweit die gespeicherten Werte die reale Sachlage korrekt abbilden, während die Vollständigkeit den Grad angibt, in dem alle erforderlichen Werte tatsächlich vorhanden sind. Die Konsistenz bezieht sich auf die Widerspruchsfreiheit der Daten innerhalb eines Bestands und über Systemgrenzen hinweg, etwa bei einheitlichen Formaten und Bezeichnungen. Die Aktualität erfasst, ob die Daten den jeweils gültigen Zeitstand widerspiegeln, und die Eindeutigkeit verlangt, dass jeder reale Sachverhalt nur einmal und nicht in mehreren widersprüchlichen oder duplizierten Datensätzen abgelegt ist [15, 16].

Mit dem Aufkommen von Big Data ist dieser klassische Kanon erweitert worden, da Volumen, Geschwindigkeit und Vielfalt der Daten zusätzliche Anforderungen stellen und feinere Differenzierungen der Qualitätsdimensionen erforderlich machen [12, 16]. Für die vorliegende Arbeit sind insbesondere die Dimensionen Genauigkeit, Vollständigkeit, Konsistenz und Eindeutigkeit von Bedeutung, da sie unmittelbar mit den untersuchten ETL-Teilaufgaben der Datenbereinigung, Datentransformation und Duplikaterkennung korrespondieren.

2.2.2. Häufige Datenprobleme

Aus den Qualitätsdimensionen lassen sich die in der Praxis am häufigsten auftretenden Datenprobleme ableiten, die im Rahmen der Datenbereinigung erkannt und behoben werden müssen. Fehlende Werte gehören zu den verbreitetsten Mängeln und beeinträchtigen unmittelbar die Vollständigkeit. Sie erfordern Entscheidungen über Imputation, Entfernung oder Kennzeichnung der betroffenen Datensätze [12, 17]. Inkonsistente Formate, etwa unterschiedliche Datums-, Zahlen- oder Einheitendarstellungen sowie abweichende Schreibweisen gleicher Inhalte, beeinträchtigen die Konsistenz und müssen im Transformationsschritt vereinheitlicht werden [12, 16].

Ein weiteres zentrales Problem sind Duplikate, also mehrfach vorhandene Datensätze, die denselben realen Sachverhalt beschreiben, sich aber in Schreibweise oder Vollständigkeit unterscheiden können. Ihre Erkennung ist anspruchsvoll, da sie über den exakten Abgleich hinaus eine unscharfe Ähnlichkeitsbetrachtung erfordert. Elmagarmid et al. ordnen die dafür entwickelten Verfahren in ihrer Übersicht zur Duplikaterkennung nach der zugrunde liegenden Ähnlichkeitsmetrik und zeigen, dass die Wahl dieser Metrik und ihres Schwellenwerts das Ergebnis maßgeblich bestimmt [18]. Wie schwierig die Aufgabe bleibt, verdeutlicht Binette und Steorts: Selbst bei sorgfältiger Modellierung ist die Abgrenzung übereinstimmender von lediglich ähnlichen Datensätzen nicht vollständig automatisierbar [17, 19]. Hinzu kommen Ausreißer und fehlerhafte Werte, die durch Eingabefehler, Messfehler oder Systembrüche entstehen und die Genauigkeit des Datenbestands gefährden [12, 17]. Die Bedeutung einer gründlichen Bereinigung ergibt sich daraus, dass mangelhafte Eingangsdaten unmittelbar auf nachgelagerte Analysen und Modelle durchschlagen, was häufig mit dem Grundsatz „garbage in, garbage out“ umschrieben wird [12, 16]. Die gezielte Konstruktion solcher Datenprobleme bildet später die Grundlage des in Kapitel 5 beschriebenen synthetischen Datensatzes.

2.3. Künstliche Intelligenz und maschinelles Lernen

Die in dieser Arbeit untersuchten Sprachmodelle sind Teil eines übergeordneten Forschungs- und Anwendungsfeldes, das dieser Abschnitt einordnet, bevor die Modelle selbst betrachtet werden. Unter dem Begriff der KI wird das Bestreben verstanden, Maschinen mit Fähigkeiten auszustatten, die üblicherweise menschliche Intelligenz voraussetzen, etwa das Erkennen von Mustern, das Ziehen von Schlüssen oder das Treffen von Entscheidungen [20, 21]. KI ist damit ein weit gefasster Oberbegriff, der von regelbasierten Expertensystemen bis zu lernenden Verfahren reicht.

Den heute dominierenden Zweig der KI bildet Maschinelles Lernen (ML). Anders als regelbasierte Systeme, deren Verhalten explizit vorgegeben wird, leiten ML-Verfahren die zugrunde liegenden Zusammenhänge selbstständig aus Beispieldaten ab und übertragen sie auf neue Fälle [7, 21]. Genau dieser Unterschied, gelernte statt fest programmierter Verarbeitungslogik, begründet den Kontrast zwischen den untersuchten LLMs und der regelbasierten Vergleichspipeline. In der Datenverarbeitung sind solche Verfahren bereits etabliert, etwa zur Bereinigung, zur Imputation fehlender Werte, zum Abgleich von Entitäten und zur Anomalieerkennung [7, 17]. Ihre leistungsfähigste Ausprägung ist das auf tiefen neuronalen Netzen beruhende *Deep Learning*, dessen Fortschritte, insbesondere die Transformer-Architektur, die Grundlage der im folgenden Abschnitt behandelten LLMs bilden [5, 22].

2.4. Large Language Models

2.4.1. Funktionsweise und Architektur

LLMs sind Sprachmodelle mit einer sehr großen Zahl an Parametern, die auf großen Mengen an Textdaten vortrainiert werden und in der Lage sind, natürliche Sprache zu verstehen und zu erzeugen [5, 22]. Technische Grundlage nahezu aller aktuellen Modelle ist die von Vaswani et al. eingeführte Transformer-Architektur, die auf Rekurrenz und Faltung vollständig verzichtet und deren Selbstaufmerksamkeitsmechanismus es erlaubt, Abhängigkeiten zwischen weit auseinanderliegenden Textbestandteilen effizient und zugleich parallelisierbar zu erfassen [23]. Genau diese Parallelisierbarkeit ist die Voraussetzung dafür, Modelle in der heute üblichen Größenordnung überhaupt zu trainieren [5, 22]. Eingaben werden dabei zunächst in kleinere Einheiten, sogenannte Token, zerlegt, die das Modell intern als numerische Vektoren verarbeitet. Die Ausgabe entsteht, indem das Modell schrittweise das jeweils wahrscheinlichste nachfolgende Token vorhersagt [5].

Charakteristisch für LLMs ist der zweistufige Entstehungsprozess aus einem aufwendigen Vortraining auf großen, allgemeinen Datenmengen und einer anschließenden Anpassung. Ouyang et al. haben für diese zweite Stufe gezeigt, dass ein Training mit menschlichem Feedback die Modelle deutlich besser an die Absicht der Anweisung ausrichtet, und dass ein so angepasstes Modell mit erheblich weniger Parametern gegenüber einem größeren, nicht angepassten bevorzugt wird [24]. Die Fähigkeit, einer Aufgabenbeschreibung zu folgen, ist damit nicht allein eine Frage der Modellgröße [5]. Ein zentraler Befund der Forschung ist, dass mit zunehmender Modell- und Datengröße neue Fähigkeiten auftreten, die kleinere Modelle nicht aufweisen. Dazu zählt insbesondere das In-Context-Lernen, also die Fähigkeit, eine Aufgabe allein anhand von Anweisungen und Beispielen im Eingabetext zu lösen, ohne dass die Modellparameter verändert werden [5, 25]. Genau diese Eigenschaft bildet die Grundlage dafür, LLMs ohne aufgabenspezifisches Nachtraining für ETL-Teilaufgaben einzusetzen. Zugleich ist die Ausgabe der Modelle probabilistischer Natur: Da das nächste Token aus einer Wahrscheinlichkeitsverteilung gezogen wird, können identische Eingaben zu unterschiedlichen Ausgaben führen, was unmittelbare Folgen für die Reproduzierbarkeit der Ergebnisse hat [5].

2.4.2. Überblick aktueller Modelle

Seit der breiten öffentlichen Aufmerksamkeit für dialogfähige Modelle hat sich das Feld der LLMs rasant entwickelt und ausdifferenziert [5, 26]. Auf dem Markt konkurriert heute eine Reihe leistungsfähiger Modellfamilien, darunter die GPT-Modelle von OpenAI, die Claude-Modelle von Anthropic, die Gemini-Modelle von Google sowie offen verfügbare Modelle wie die Llama-Reihe [5, 26]. Diese Modelle unterscheiden sich nicht nur in ihrer Architektur und Parameterzahl, sondern auch in praxisrelevanten Eigenschaften wie der Größe des Kontextfensters, der Verarbeitungsgeschwindigkeit, den Nutzungskosten und der Verfügbarkeit als offenes oder geschlossenes Modell [5, 26].

Eine wesentliche Entwicklung der jüngeren Vergangenheit ist die Erweiterung hin zu multimodalen Modellen, die neben Text auch weitere Modalitäten wie Bilder verarbeiten können [5, 22]. Angesichts der hohen Innovationsgeschwindigkeit verschieben sich die Leistungsgrenzen zwischen den Modellen kontinuierlich, weshalb vergleichende Bewertungen anhand standardisierter Kennzahlen wie Qualität, Geschwindigkeit und Preis an Bedeutung gewonnen haben [26, 27]. Für die vorliegende Arbeit ist diese Heterogenität von unmittelbarer Relevanz, da sie die Auswahl mehrerer aktueller Modelle für den experimentellen Vergleich begründet und nahelegt, dass sich die Eignung für ETL-Aufgaben zwischen den Modellen unterscheiden kann.

2.4.3. Stärken und Limitationen bei strukturierten Daten

Im Kontext der Datenverarbeitung liegt die Stärke von LLMs in ihrer Flexibilität: Sie können semi- und unstrukturierte Inhalte interpretieren, kontextabhängige Zusammenhänge erfassen und über reine Mustererkennung hinaus auch implizites Wissen einbringen, ohne dass die zugrunde liegenden Regeln explizit vorgegeben werden müssen [5, 6]. Damit eignen sie sich potenziell für Aufgaben wie die Bereinigung uneinheitlicher Werte, die Vereinheitlichung von Formaten oder das Erkennen ähnlicher Datensätze, die mit starren Regeln nur schwer abzudecken sind [6].

Diesen Stärken stehen jedoch erhebliche Limitationen gegenüber. Eine zentrale Einschränkung ist die Neigung zu Halluzinationen, also zur Erzeugung plausibel wirkender, aber sachlich falscher oder frei erfundener Ausgaben, was bei der Verarbeitung strukturierter Daten zu schwer erkennbaren Fehlern führen kann [5, 6]. Hinzu kommt die Begrenzung durch das Kontextfenster, die der Menge an Daten, die in einem einzelnen Verarbeitungsschritt übergeben werden kann, eine technische Obergrenze setzt [5]. Ferner fallen die Nutzungskosten und die Verarbeitungszeit ins Gewicht, die insbesondere bei großen Datenmengen zu einem begrenzenden Faktor werden [5, 26]. Schließlich erschwert die bereits beschriebene probabilistische Natur der Modelle die Reproduzierbarkeit, da wiederholte Ausführungen identischer Aufgaben voneinander abweichende Ergebnisse liefern können [5, 6]. Diese Spannung zwischen Flexibilität und Verlässlichkeit motiviert den Vergleich mit regelbasierten Verfahren, der dieser Arbeit zugrunde liegt.

2.5. Prompt Engineering

2.5.1. Grundlegende Prompting-Strategien

Da LLMs ihre Aufgaben primär über das In-Context-Lernen lösen, kommt der Gestaltung der Eingabe, dem sogenannten Prompt, eine entscheidende Bedeutung zu. Unter Prompt Engineering versteht man das gezielte Entwerfen und Optimieren solcher Eingaben, um das Modellverhalten zu steuern, ohne die Modellparameter selbst zu verändern [28, 29]. Die einfachste Strategie ist das Zero-Shot-Prompting, bei dem das Modell eine Aufgabe allein anhand einer Anweisung und ohne weitere Beispiele bearbeitet [25, 28]. Beim Few-Shot-Prompting werden der Anweisung hingegen einige beispielhafte Ein- und Ausgabepaare beigefügt, anhand derer das Modell das gewünschte Verhalten ableitet. Brown et al. haben diese Fähigkeit an GPT-3 systematisch nachgewiesen und gezeigt, dass hinreichend große Modelle allein aus solchen Demonstrationen im Prompt lernen, ohne dass ihre Parameter angepasst werden [30]. Der Ansatz verbessert die Ergebnisqualität bei vielen Aufgaben spürbar [25, 28].

Darüber hinaus haben sich anspruchsvollere Strategien etabliert. Beim Chain-of-Thought-Prompting wird das Modell angeleitet, seine Lösung in nachvollziehbaren Zwischenschritten zu entwickeln. Wei et al. haben diese Technik eingeführt und gezeigt, dass sie die Leistung bei arithmetischen, schlussfolgernden und symbolischen Aufgaben deutlich verbessert, dieser Gewinn jedoch erst ab einer bestimmten Modellgröße auftritt [31]. Für mehrstufige Aufgaben gilt sie seither als wirksames Standardverfahren [28]. Eine weitere verbreitete Technik ist das Role-Prompting, bei dem dem Modell eine bestimmte Rolle oder Perspektive zugewiesen wird, um Stil und Inhalt der Ausgabe zu lenken [28, 29]. Es wird in dieser Arbeit nicht als eigener Untersuchungsfaktor geführt, da eine Rollenzuweisung, die eines erfahrenen Data Engineers, in allen eingesetzten Systemanweisungen einheitlich enthalten ist und damit für alle verglichenen Strategien gleichermaßen gilt. Sie wäre als Faktor nicht von den übrigen Strategien trennbar. Angesichts der Vielzahl mittlerweile beschriebener Techniken weisen systematische Übersichten allerdings auf eine uneinheitliche Begriffsbildung und eine teils fragmentierte Forschungslage hin, was eine sorgfältige Auswahl und Definition der eingesetzten Strategien erforderlich macht [28, 29]. Für die vorliegende Arbeit werden daher drei Strategien verglichen, die sich allein im Umfang des mitgegebenen Kontexts unterscheiden.

2.5.2. Einflussfaktoren auf die Ergebnisqualität

Die Qualität der von einem LLM erzeugten Ausgabe hängt von einer Reihe von Einflussfaktoren ab, die über die reine Wahl der Prompting-Strategie hinausgehen. Beim Few-Shot-Prompting wirken sich Auswahl, Anzahl und Reihenfolge der mitgegebenen Beispiele unmittelbar auf das Ergebnis aus, sodass schon kleine Änderungen an den Beispielen die Modellantwort verschieben können [25, 29]. Auch die genaue Formulierung und Strukturierung der Anweisung beeinflusst die Ergebnisqualität erheblich, weshalb das Auffinden geeigneter Prompts häufig einen iterativen Prozess darstellt [28, 32].

Ein technisch bedeutsamer Parameter ist die Temperatur, die steuert, wie stark die Ausgabe von der wahrscheinlichsten Vorhersage abweicht: Niedrige Werte führen zu deterministischeren und stabileren, hohe Werte zu vielfältigeren, aber weniger vorher-sagbaren Antworten, was die Reproduzierbarkeit der Ergebnisse unmittelbar berührt [5, 29]. Zudem skaliert die Leistungsfähigkeit beim Prompting mit der Größe und Leis-tungsfähigkeit des zugrunde liegenden Modells, sodass dieselbe Strategie je nach Mo-dell unterschiedlich gut wirkt [25, 29]. Schließlich ist mit längeren und beispielreicheren Prompts ein höherer Verbrauch an Token und damit ein höherer Aufwand verbunden, sodass zwischen Ergebnisqualität und Effizienz abzuwägen ist [29, 32]. Diese Fak-toren begründen, warum in der vorliegenden Arbeit sowohl unterschiedliche Modelle als auch unterschiedliche Prompting-Strategien unter kontrollierten Bedingungen ver-glichen und ihre Ergebnisse mehrfach erhoben werden.

3. Stand der Forschung

Aufbauend auf den theoretischen Grundlagen ordnet dieses Kapitel die vorliegende Arbeit in den aktuellen Forschungsstand ein. Vorangestellt ist das Vorgehen der Litera-turrecherche, das die Auswahl der herangezogenen Arbeiten offenlegt (Abschnitt 3.1). Anschließend wird der breitere Einsatz von Künstlicher Intelligenz in der Datenverar-beitung betrachtet (Abschnitt 3.2). Anschließend wird die Forschung zur Verarbeitung strukturierter und tabellarischer Daten durch LLMs sowie zur Erzeugung synthetischer Datensätze aufgearbeitet, die die Wahl der Datengrundlage dieser Arbeit begründet (Abschnitt 3.3), bevor verwandte Arbeiten zur Automatisierung von ETL-Prozessen diskutiert werden (Abschnitt 3.4). Auf dieser Grundlage wird abschließend die For-schungslücke präzisiert, an der die vorliegende Arbeit ansetzt (Abschnitt 3.5).

3.1. Vorgehen der Literaturrecherche

Um die Auswahl der herangezogenen Arbeiten nachvollziehbar zu machen, wird das Vorgehen der Recherche vorangestellt. Gesucht wurde entlang der drei Themensträn-ge, die sich aus der Zielsetzung ergeben: ETL-Prozesse und Datenqualität, der Ein-satz von LLMs auf strukturierten Daten sowie Prompt Engineering. Verwendet wur-den dafür englischsprachige Suchbegriffe und deren Kombinationen, unter anderem „ETL automation“, „LLM data cleaning“, „data wrangling foundation models“, „entity matching“, „data quality dimensions“ und „prompt engineering survey“. Als Einstiegs-punkte dienten Google Scholar und arXiv, ergänzt um die Vorwärts- und Rückwärtssu-che ausgehend von den zentralen Übersichtsarbeiten.

Aufgenommen wurden Arbeiten, die einen der drei Themenstränge unmittelbar behan-deln und entweder eine empirische Untersuchung, eine systematische Übersicht oder eine begründete Architekturbeschreibung liefern. Der Schwerpunkt liegt bewusst auf aktuellen Veröffentlichungen, da sich das Feld der LLMs in Monatsabständen verän-dert; ältere Arbeiten wurden nur dort einbezogen, wo sie einen etablierten Begriff oder ein etabliertes Verfahren begründen, etwa zur Duplikaterkennung. Der so entstandene Bestand umfasst 41 Quellen aus den Jahren 2007 bis 2026 mit deutlichem Schwer-punkt auf den Jahren 2023 bis 2026, davon 24 Zeitschriftenaufsätze, fünf Konferenz-

beiträge, elf Online-Quellen und eine Hochschulschrift.

Für die Einordnung der folgenden Abschnitte ist eine Beobachtung zur Beschaffenheit dieses Bestands wesentlich. Die empirischen Arbeiten zur Verarbeitung strukturierter Daten durch LLMs erscheinen überwiegend in begutachteten Fachzeitschriften und auf etablierten Konferenzen. Die konzeptionelle Literatur zur KI-gestützten Automatisierung von ETL-Prozessen, die den in Abschnitt 3.4 referierten Erwartungshorizont prägt, erscheint dagegen zu einem erheblichen Teil in Publikationsorganen ohne erkennbar strenge Begutachtung. Ihre Aussagen sind entsprechend als Beschreibung eines Erwartungshorizonts und nicht als gesicherter Befund zu lesen. Gerade dieser Umstand stützt die Anlage der vorliegenden Arbeit, denn er verweist auf einen Bedarf an kontrollierter empirischer Prüfung dessen, was dort konzeptionell in Aussicht gestellt wird.

3.2. KI-Einsatz in der Datenverarbeitung

Der Einsatz von Künstlicher Intelligenz und maschinellem Lernen hat die Datenverarbeitung in den vergangenen Jahren grundlegend verändert. Schon vor dem Aufkommen großer Sprachmodelle wurden Verfahren des maschinellen Lernens eingesetzt, um Schritte der Datenaufbereitung, der Mustererkennung und der Entscheidungsunterstützung zu automatisieren und so aus großen Datenbeständen Erkenntnisse zu gewinnen [20, 21]. Insbesondere im Bereich der Datenaufbereitung haben sich lernende Verfahren als nützlich erwiesen, etwa bei der Datenbereinigung, der Imputation fehlender Werte, dem Abgleich von Entitäten und der Erkennung von Ausreißern [7, 17]. Ein einschlägiges Beispiel ist HoloClean von Rekatsinas et al., das Integritätsbedingungen, externe Referenzdaten und statistische Eigenschaften des Bestands in einem probabilistischen Modell zusammenführt und daraus Reparaturvorschläge ableitet, statt für jeden Fehlertyp eine eigene Regel zu verlangen [33]. Bereits vor dem Aufkommen großer Sprachmodelle bestand der Beitrag lernender Verfahren damit vor allem darin, die vollständige Vorabformulierung der Bereinigungsvorschrift zu ersetzen, also genau jene Eigenschaft, die in dieser Arbeit für LLMs untersucht wird. Eine systematische Übersicht zeigt dabei ein wechselseitiges Verhältnis: Maschinelles Lernen wird einerseits zur Datenbereinigung genutzt, andererseits ist eine hohe Datenqualität Voraussetzung für die Leistungsfähigkeit der Modelle selbst [7, 17].

Mit der Verfügbarkeit dialogfähiger Sprachmodelle hat sich dieser Trend noch einmal verstärkt. Frühe Arbeiten beschreiben, wie Modelle wie ChatGPT Datenfachleute bei zahlreichen Teilschritten ihres Arbeitsablaufs unterstützen können, von der Datenbereinigung und Vorverarbeitung über die Modellbildung bis hin zur Interpretation der Ergebnisse [6, 21]. Gleichzeitig betonen diese Beiträge, dass der Einsatz solcher Modelle mit Herausforderungen wie Verzerrungen, mangelnder Nachvollziehbarkeit und Fragen der Verlässlichkeit verbunden ist und daher einer kritischen Begleitung bedarf [6, 20]. Insgesamt zeichnet die Literatur das Bild einer zunehmend KI-gestützten Datenverarbeitung, in der die Automatisierung von Routineaufgaben im Vordergrund steht, eine systematische empirische Bewertung der erzielten Qualität jedoch häufig noch aussteht [7, 17].

3.3. LLMs für strukturierte Daten und synthetische Datensätze

Dieser Abschnitt klärt zwei Voraussetzungen der Untersuchung: dass LLMs tabellarische Daten überhaupt verarbeiten können, und dass ein synthetisch erzeugter Datensatz als Prüfgrundlage taugt.

Für den Einsatz von LLMs auf strukturierten Daten liegen direkte empirische Untersuchungen vor. Ein früher und vielzitiert Beitrag ist Ditto von Li et al.: Die Autoren führen den Abgleich zweier Datensätze auf eine Klassifikationsaufgabe zurück und lösen sie mit einem vortrainierten Sprachmodell, das für diesen Zweck nachtrainiert wird [34]. Damit ist die in dieser Arbeit als anspruchsvollste Bereinigungsstufe geführte Erkennung unscharfer Duplikate bereits als originäres Anwendungsfeld vortrainierter Sprachmodelle belegt, allerdings mit aufgabenspezifisch angepassten Modellen und nicht, wie hier, mit unverändert eingesetzten. Narayan et al. gehen einen Schritt weiter und formulieren fünf Aufgaben der Datenbereinigung und -integration als reine Prompting-Aufgaben ohne Nachtraining, wobei sie neben der Ergebnisqualität auch die Empfindlichkeit gegenüber Prompt-Format und Beispielauswahl untersuchen [8]. Damit ist die grundsätzliche Eignung bereits experimentell untersucht, die Ergebnisse sprechen zugleich dafür, konkrete Aufgaben und Prompt-Varianten getrennt zu bewerten.

Davon zu unterscheiden ist die Erzeugung synthetischer Daten durch generative Modelle. Übersichtsarbeiten behandeln deren Verfahren und Grenzen [35–37]. Li et al. untersuchen beispielsweise LLM-generierte Trainingsdaten für Textklassifikation [38]. Die vorliegende Arbeit erzeugt ihre Testdaten dagegen regelbasiert mit Faker. Ergebnisse über generierte Trainingsdaten sind daher kein unmittelbarer Qualitätsnachweis für diesen Testdatensatz. Dessen Eignung wird anhand des dokumentierten Datenmodells, der eingebrachten Fehler und der Validierung in Kapitel 5 begründet.

3.4. Verwandte Arbeiten zur Automatisierung von ETL

Die Automatisierung von ETL-Prozessen mit Methoden der Künstlichen Intelligenz ist Gegenstand einer wachsenden Zahl von Arbeiten. Ein erster, schon länger verfolgter Ansatz nutzt klassische Verfahren des maschinellen Lernens, um einzelne Schritte der Pipeline zu automatisieren, etwa das Abbilden von Schemata, die Erkennung von Anomalien und die Ableitung von Transformationsregeln aus den Daten [13, 39]. Darauf aufbauend beschreibt eine Reihe neuerer Beiträge umfassendere Architekturen, in denen KI-Komponenten auf allen Stufen des ETL-Prozesses eingebettet werden, um Aufgaben wie automatisierte Schemaerkennung, intelligente Qualitätssicherung und Leistungsoptimierung zu übernehmen [2, 40].

Für die folgenden Beiträge gilt dabei der in Abschnitt 3.1 benannte Vorbehalt in besonderem Maße: Sie beschreiben überwiegend Architekturentwürfe und Erwartungen, ohne diese empirisch zu prüfen, und erscheinen teilweise in Organen ohne strenge Begutachtung. Sie werden hier referiert, um den Erwartungshorizont abzustecken, gegen den die Ergebnisse dieser Arbeit in Abschnitt 8.12 gestellt werden.

Ein wiederkehrendes Leitbild dieser Arbeiten ist die Vorstellung selbstheilender und adaptiver Datenpipelines, die Probleme proaktiv erkennen, eigenständig beheben und sich an veränderte Datenstrukturen anpassen, statt rein reaktiv auf Fehler zu reagieren [39, 41]. In diesem Zusammenhang werden zunehmend autonome KI-Agenten diskutiert, die Datenaufnahme, Bereinigung und Integration weitgehend selbstständig ausführen sollen [41, 42]. Mehrere Beiträge widmen sich dabei spezifischen Teilaufgaben, etwa der intelligenten Anomalieerkennung in ETL-Strecken, der automatisierten Qualitätssicherung oder der automatisierten Erzeugung von Merkmalen [43, 44]. Insbesondere wird betont, dass nicht ein vollständiger Ersatz, sondern eine hybride Verknüpfung von regelbasierten Systemen und KI-Modellen erstrebenswert sei, um Nachvollziehbarkeit und Kontrolle bei gleichzeitig höherer Flexibilität zu wahren [2, 40].

Ein weiterer empirischer Ansatz ist Jellyfish von Zhang et al.: Die Autoren passen lokale Sprachmodelle mit 7 bis 13 Milliarden Parametern durch Instruction-Tuning an Fehlererkennung, Imputation, Schemaabgleich und Entitätsabgleich an [9]. Das verdeutlicht, dass neben Modellgröße und Betriebsform auch die aufgabenspezifische Anpassung zu berücksichtigen ist. Die hier untersuchte lokale Llama-Konfiguration bildet diese spezialisierte Modellklasse nicht ab.

Die vorliegende Untersuchung knüpft an diese Arbeiten an, evaluiert jedoch unverändert eingesetzte Modelle auf den selbst definierten Aufgaben. Ihr Schwerpunkt liegt auf der Gegenüberstellung mit einer regelbasierten Referenz und der gemeinsamen Betrachtung von Prompt-Vorlagen, Code-Generierung, Direktverarbeitung und verketteter Ausführung. Ein numerischer Leistungsvergleich mit den genannten Veröffentlichungen wäre wegen unterschiedlicher Datensätze, Aufgaben und Metriken nicht sachgerecht.

3.5. Forschungslücke und Abgrenzung

Die Forschungslücke dieser Arbeit liegt nicht im grundsätzlichen Fehlen empirischer LLM-Evaluierungen. Die genannten Arbeiten untersuchen bereits mehrere Datenaufgaben und unterschiedliche Formen der Modellanpassung und Prompt-Gestaltung. Ergänzend wird hier für ein einheitlich konstruiertes ETL-Szenario betrachtet, wie sich die Leistung zwischen unabhängigen Aufgaben, verketteter Verarbeitung und Durchlaufen mit Soll-Zwischentabellen unterscheidet. Hinzu kommen der Vergleich von Code-Generierung und direkter Datenverarbeitung sowie die explorativen Ebenen zum Aufgabenzuschnitt.

Die wiederholte Ausführung dient dabei als querschnittliche Untersuchungsdimension. Sie beschreibt die beobachtete Schwankung der Kennzahlen unter den jeweiligen Versuchsbedingungen. Wegen der begrenzten Wiederholungszahl handelt es sich um eine deskriptive Untersuchung, eine allgemeine Aussage über die Reproduzierbarkeit einer Modellfamilie oder eine vollständige statistische Absicherung der Unterschiede wird damit nicht beansprucht.

FF1 adressiert die aufgabenbezogene Eignung gegenüber der regelbasierten Vergleichspipeline und die ergänzenden Ausführungsebenen. FF2 betrifft Unterschiede zwischen den drei konkret eingesetzten Prompt-Vorlagen. Die wiederholten Ausführun-

gen helfen bei beiden Fragen, beobachtete Unterschiede im Verhältnis zur Streuung einzuordnen.

4. Konzeption der experimentellen Evaluierung

Aufbauend auf den theoretischen Grundlagen aus Kapitel 2 und der in Kapitel 3 präzisierten Forschungslücke beschreibt dieses Kapitel die Konzeption der experimentellen Evaluierung. Es überführt die in Abschnitt 1.2 formulierten Forschungsfragen in ein konkretes Untersuchungsdesign und legt damit die methodische Grundlage für die folgende Umsetzung. Zunächst wird das übergeordnete Untersuchungsdesign und die daraus abgeleiteten Hypothesen dargelegt (Abschnitt 4.1). Anschließend werden die Auswahl der untersuchten LLMs (Abschnitt 4.2), die Definition der betrachteten ETL-Aufgaben (Abschnitt 4.3) sowie die verglichenen Prompting-Strategien (Abschnitt 4.4) beschrieben. Darauf folgt die Operationalisierung der Evaluationskriterien (Abschnitt 4.5), bevor abschließend die klassische ETL-Pipeline als Vergleichsbasis definiert wird (Abschnitt 4.6). Die konkrete technische Umsetzung des hier entworfenen Designs ist Gegenstand der Kapitel 5 bis 7.

4.1. Untersuchungsdesign und Hypothesen

Zur Beantwortung der Forschungsfragen wird ein experimentelles Untersuchungsdesign gewählt, das den Einsatz von LLMs für ausgewählte ETL-Teilaufgaben unter kontrollierten Bedingungen mit einer klassischen, regelbasierten Pipeline vergleicht. Ein experimenteller Zugang ist hier angemessen, da die Forschungslücke gerade im Fehlen belastbarer, vergleichender Evaluierungen unter einheitlichen Bedingungen besteht, während das grundsätzliche Potenzial KI-gestützter ETL-Prozesse in der Literatur bereits vielfach konzeptionell beschrieben wurde [2, 45]. Den Kern des Designs bildet ein vollständig gekreuztes Versuchsschema über die drei Faktoren Modell, Prompting-Strategie und ETL-Aufgabe: Für jede Kombination dieser Faktoren wird die Verarbeitung durchgeführt und anhand einheitlicher, in Abschnitt 4.5 definierter Kriterien bewertet, sodass sich die Einflüsse der einzelnen Faktoren systematisch isolieren lassen.

Eine zentrale Gestaltungsentscheidung betrifft die Einbindung der Modelle in den Verarbeitungsprozess. Als primärer Ausführungsmodus werden sie angewiesen, einen ETL-Verarbeitungsschritt als Python-Code zu erzeugen, der anschließend deterministisch auf den Daten ausgeführt und dessen Ergebnis gegen eine bekannte Soll-Lösung (Ground Truth) geprüft wird. Diese Code-Generierung entkoppelt die Verarbeitung von der Datenmenge und damit von der Begrenzung durch das Kontextfenster, erzeugt ein nachvollziehbares und wiederausführbares Artefakt und stellt die unmittelbare Vergleichbarkeit mit der regelbasierten Pipeline her, die ebenfalls programmatisch umgesetzt wird [1, 3].

Ergänzend wird ein zweiter Ausführungsmodus untersucht, die direkte In-Context-

Verarbeitung, bei der dem Modell die Eingabedaten im Prompt bereitgestellt werden und es die Ergebnistabelle ohne den Umweg über generierten Code zurückgibt. Dieser Modus zielt auf die semantisch geprägten Aufgaben, bei denen sich das erforderliche Wissen nicht vollständig in feste Regeln überführen lässt, etwa bei uneinheitlich geschriebenen Länderangaben oder unscharfen Duplikaten: Das Modell wendet sein Weltwissen unmittelbar auf jeden einzelnen Wert an, statt eine verallgemeinernde Vorschrift zu formulieren [6, 46]. Da die Daten dabei vollständig im Prompt mitgeführt werden, begrenzt das Kontext- und Ausgabefenster diesen Modus auf überschaubare Datenmengen. Er wird deshalb nicht als vollständig gekreuzter Faktor, sondern als ergänzender Vergleich auf denselben Aufgaben und Modellen geführt [5, 32].

Die bis hierhin beschriebene Faktormatrix setzt für jede Aufgabe unmittelbar auf den unveränderten Rohdaten auf. Eine reale ETL-Strecke besteht jedoch aus einer Kette, in der jeder Schritt auf dem Ergebnis seines Vorgängers aufsetzt [1, 2]. Das Design wird daher um eine verkettete Ausführung ergänzt, in der die Ausgabe jedes Schritts die Eingabe des nächsten bildet und ein Abbruch die abhängigen Schritte blockiert, sodass sichtbar wird, wie sich lokal geringe Abweichungen über die Kette hinweg auswirken. Damit ein so gemessener Verlust nicht fälschlich dem betreffenden Schritt zugeschrieben wird, obwohl er aus fehlerhaften Eingabedaten resultiert, tritt ein zweiter, sonst identischer Durchlauf hinzu, in dem jeder Schritt statt der Ausgabe seines Vorgängers die entsprechende Soll-Zwischentabelle erhält. Die Differenz beider Durchläufe beziffert unmittelbar den Anteil des Ergebnisverlusts, der auf vorgelagerte Schritte zurückgeht, eine für FF1 wesentliche Trennung, da eine Empfehlung je Aufgabentyp voraussetzt, dass ein Fehler dem verursachenden Schritt und nicht seinem Nachfolger zugerechnet wird [3].

Da LLMs auch bei identischer Eingabe voneinander abweichende Ausgaben erzeugen können, wird die Reproduzierbarkeit als querschnittliche Dimension in das Design aufgenommen: Jede Konfiguration wird mehrfach ausgeführt, sodass sich die Streuung über die Wiederholungen quantifizieren lässt [5, 6]. Die Wiederholung bezieht sich dabei auf den Generierungsschritt. Das Modell wird jedes Mal erneut zur Code-Erzeugung aufgefordert, während die Ausführung eines einzelnen erzeugten Skripts deterministisch bleibt. Jeder Aufruf erfolgt dabei zustandslos. Aus vorangegangenen Läufen wird nichts übernommen. Steuerbare Einflussgrößen wie die in Abschnitt 2.5.2 eingeführte Temperatur bleiben über alle Durchläufe konstant und niedrig, um die nicht modellbedingte Variabilität zu minimieren [5, 29].

Aus den Forschungsfragen und dem Forschungsstand lassen sich die folgenden Hypothesen ableiten, die im Rahmen der Auswertung in Kapitel 8 geprüft werden. Zu FF1 werden dabei zwei Hypothesen formuliert, die ihre beiden Teilfragen, den Aufgabentyp und die Ausführungsform, getrennt adressieren, zu FF2 eine. Jede bleibt damit unmittelbar auf eine der beiden Fragen bezogen und bereitet deren Beantwortung vor:

H1 (zu FF1): Die Eignung von LLMs hängt stark vom Aufgabentyp ab. Bei klar formalisierbaren Aufgaben erreichen sie bestenfalls eine mit der regelbasierten Pipeline vergleichbare Ergebnisqualität, während sie bei Aufgaben mit hoher semantischer Anforderung Vorteile zeigen und mit zunehmender struktureller Komplexität an Zuverlässigkeit verlieren. Ein über alle Aufgaben hinweg überlegener Ansatz

existiert daher nicht [6, 17].

H1a (zu FF1): Die Eignung wird nicht allein vom Aufgabentyp bestimmt, sondern zusätzlich von der Ausführungsform. Erwartet wird, dass der Ausführungsmodus darüber entscheidet, welche Fähigkeit eines Modells überhaupt zur Wirkung kommt, nämlich Einzelfallverständnis in der Direktverarbeitung und verlässliche Wiederholbarkeit in der Code-Generierung, und dass der Zuschnitt der Aufgabe die Ergebnisqualität stärker beeinflusst als die Wahl des Modells, da mit der Zahl der in einem Arbeitsschritt gleichzeitig zu treffenden Entscheidungen die Fehleranfälligkeit überproportional wächst [1, 46].

H2 (zu FF2): Aufwändigere Prompting-Strategien, die zusätzlichen Kontext, Beispiele oder Zwischenschritte bereitstellen, verbessern die Ergebnisqualität gegenüber einem einfachen Zero-Shot-Prompt, wobei der Nutzenzuwachs mit steigendem Aufwand abnimmt und der zusätzliche Kontext bei zu enger Führung auch nachteilig wirken kann [28, 29].

Tabelle 4.1 fasst die daraus entstehenden Untersuchungsebenen zusammen. Sie unterscheiden sich darin, wie eine Aufgabe gestellt wird, ob unabhängig auf den Rohdaten, als Glied einer Kette oder als Gruppe zusammengefasster Schritte, und welche Eingabe sie dabei erhält. Erst ihr Zusammenspiel erlaubt die Zuordnung eines beobachteten Fehlers zu seiner Ursache: Die Faktormatrix misst die Leistung bei einer isoliert gestellten Aufgabe, die verkettete Ausführung die Leistung im Realbetrieb, und die Differenz zum isolierten Durchlauf beziffert den Anteil, der aus vorgelagerten Schritten stammt.

Tabelle 4.1.: Die Ebenen des Untersuchungsdesigns mit ihrer jeweiligen Fragestellung und Eingabeform. Die konkreten Laufzahlen je Ebene nennt Kapitel 7.

Ebene	Eingabe je Aufgabe	Beantwortet
Faktormatrix	Rohdaten, kein Vorgängeresgebnis	Einfluss von Modell, Strategie und Aufgabentyp
Direkt-Modus	Rohdaten, im Prompt eingebettet	Einfluss des Ausführungsmodus
Verkettete Pipeline	Ausgabe des Vorgängerschritts	Leistung im Realbetrieb einer Strecke
Isolierter Durchlauf	Soll-Ausgabe des Vorgängers	Trennung eigener Leistung von geerbtem Fehler
Gruppierte Strecke	Soll-Ausgabe der Vorgängerguppe	Einfluss des Aufgabenzuschnitts
Explizit gegliederte Aufgabe	Rohdaten, benannte Teilschritte	Einfluss der Ausdrücklichkeit der Anforderung
Regelbasierte Pipeline	wie die jeweilige Ebene	Vergleichsmaßstab für alle Ebenen

Die gruppierte Strecke und die explizit gegliederte Aufgabe wurden erst nach einer ersten Sichtung der Ergebnisse ergänzt, um eine im Zuschnitt der Aufgabenstellung erkennbar gewordene Blindstelle zu schließen. Die auf ihnen beruhenden Befunde sind daher als explorativ zu lesen, was bei der Bewertung von Hypothese H1a zu berücksichtigen ist.

Die Reproduzierbarkeit wird, wie in Abschnitt 1.2 festgelegt, nicht als eigenständige Hypothese geführt, sondern als querschnittliche Dimension zu beiden: Für H1 ist die Stabilität Bestandteil der Eignungsbeurteilung, da ein im Mittel gutes, aber stark schwankendes Ergebnis produktiv einen anderen Wert besitzt als ein gleich gutes, verlässlich wiederholbares. Für H2 ist sie zusätzliches Unterscheidungsmerkmal zwischen den Strategien. Gemessen wird, wie stark identische Konfigurationen trotz niedriger Temperatur voneinander abweichen und ob die Stabilität mit zunehmender Aufgabenkomplexität abnimmt [5, 6].

Der Gegenstand dieser Arbeit ist auf die Transformationsphase des ETL-Prozesses beschränkt. Extraktion und Ladevorgang bleiben ebenso außen vor wie Aspekte des produktiven Betriebs, etwa Fehlerbehandlung, Überwachung, inkrementelle Verarbeitung oder Datenschutzanforderungen beim Übertragen von Daten an externe Anbieter. Gerade der letzte Punkt ist für die praktische Anwendbarkeit erheblich und wird hier nur insoweit berührt, als mit dem lokal betriebenen Modell eine Option ohne Datenabfluss mituntersucht wird.

4.2. Auswahl der Large Language Models

Die Auswahl der untersuchten Modelle folgt dem Ziel, ein für den praktischen Einsatz repräsentatives und zugleich aussagekräftiges Spektrum aktueller LLMs abzudecken. Wie in Abschnitt 2.4.2 dargestellt, unterscheiden sich die verfügbaren Modellfamilien nicht nur in Architektur und Parameterzahl, sondern auch in praxisrelevanten Eigenschaften wie Kontextfenster, Verarbeitungsgeschwindigkeit, Nutzungskosten und Verfügbarkeit als offenes oder geschlossenes Modell [5, 26]. Gerade diese Heterogenität begründet den Vergleich mehrerer Modelle, da sich die Eignung für ETL-Aufgaben zwischen ihnen unterscheiden kann und ein Einzelmodell keine verallgemeinerbaren Aussagen zuließe [26, 27].

Als Auswahlkriterien werden die Aktualität und Verbreitung der Modelle, die Abdeckung unterschiedlicher Anbieter sowie die Berücksichtigung sowohl geschlossener als auch offen verfügbarer Modelle herangezogen. Auf dieser Grundlage werden je ein Vertreter der etablierten geschlossenen Modellfamilien von OpenAI, Anthropic und Google sowie ein offen verfügbares, lokal ausführbares Modell aus der Llama-Reihe untersucht. Damit deckt die Auswahl unterschiedliche Kosten-, Leistungs- und Betriebsmodelle ab und erlaubt insbesondere den praktisch bedeutsamen Vergleich zwischen kommerziell betriebenen und lokal betreibbaren Modellen [5, 26]. Tabelle 4.2 weist die konkret eingesetzten Modelle mit ihrer Versionskennung und den zugrunde gelegten Preisen aus.

Tabelle 4.2.: Die vier untersuchten Modelle mit Versionskennung und den für die Kostenschätzung zugrunde gelegten Listenpreisen in US-Dollar je einer Million Token (Stand: 27. Juli 2026).

Anbieter	Versionskennung	Betrieb	Eingabe	Ausgabe
Anthropic	claude-sonnet-5	Cloud	2,00	10,00
OpenAI	gpt-5.6-terra	Cloud	2,50	15,00
Google	gemini-3.6-flash	Cloud	1,50	7,50
Meta	llama3.1:8b	lokal	keine nutzungsabhängigen Kosten	

Bei den drei kommerziellen Modellen handelt es sich jeweils um das mittlere Leistungsniveau der aktuellen Generation des Anbieters, nicht um dessen Spitzenmodell. Alle drei führen intern einen Denkprozess aus, dessen Token zum Ausgabetarif abgerechnet werden und in den ausgewiesenen Kosten enthalten sind. Die angegebenen Preise sind die zum genannten Stichtag herangezogenen Listenpreise.

Um die Modelle technisch austauschbar ansprechen zu können, werden sie hinter einer einheitlichen Schnittstelle gekapselt, sodass der Versuchsablauf unabhängig vom konkreten Anbieter erfolgt. Die Details dieser Abstraktion werden in Kapitel 6 beschrieben.

4.3. Definition der ETL-Aufgaben

Die untersuchten ETL-Aufgaben orientieren sich an den in Abschnitt 2.2.2 beschriebenen, in der Praxis am häufigsten auftretenden Datenproblemen und decken die drei in dieser Arbeit fokussierten Teilbereiche der Transformationsphase ab: die Datenbereinigung, die Datentransformation und die Duplikaterkennung. Um nicht nur die grundsätzliche Eignung, sondern auch deren Grenzen in Abhängigkeit von der Aufgabenkomplexität zu erfassen, wird jeder Teilbereich in drei aufeinander aufbauenden Schwierigkeitsstufen operationalisiert. Diese reichen von einer einfachen, klar formalisierbaren Aufgabe über eine mittlere Stufe bis hin zu einer anspruchsvollen Variante, die semantisches Verständnis oder mehrstufige Verarbeitung erfordert. Daraus ergibt sich eine Matrix aus neun Aufgaben, die zugleich nach Schwierigkeit gestaffelt ist und so eine differenzierte Beurteilung der Modelle erlaubt [16, 17]. Alle Aufgaben beziehen sich auf den synthetischen Datensatz aus Kapitel 5 und sind so gestaltet, dass für jede von ihnen eine eindeutige Soll-Lösung existiert.

4.3.1. Datenbereinigung

Die Aufgaben zur Datenbereinigung adressieren die Beseitigung von Mängeln, die unmittelbar die Genauigkeit, Vollständigkeit und Konsistenz des Datenbestands beeinträchtigen [12, 17]. Die einfache Stufe umfasst grundlegende Bereinigungsoperationen wie das Entfernen überflüssiger Leerzeichen in Textspalten und das einheitliche Kennzeichnen fehlender Werte. Diese Aufgabe ist mit wenigen Regeln vollständig beschreibbar und dient als Referenzpunkt für die grundsätzliche Verarbeitungsfähigkeit der Modelle. Die mittlere Stufe verlangt die Vereinheitlichung uneinheitlich formatierter Datumsangaben, die in mehreren unterschiedlichen Darstellungen vorliegen, auf

ein einheitliches Zielformat und prüft damit die Fähigkeit zur formatübergreifenden Normalisierung. Die anspruchsvolle Stufe schließlich erfordert die semantische Vereinheitlichung von Länderangaben, die in verschiedenen Sprachen, Abkürzungen und Schreibweisen einschließlich Tippfehlern vorliegen, auf einen standardisierten Code. Diese Aufgabe geht über den rein syntaktischen Abgleich hinaus und setzt ein inhaltliches Verständnis der Werte voraus, das mit starren Regeln nur schwer abzubilden ist [6, 46].

4.3.2. Datentransformation

Die Aufgaben zur Datentransformation betreffen die Überführung der Daten in die geforderte Struktur und Typisierung sowie die Ableitung neuer Informationen aus vorhandenen Werten. Die einfache Stufe umfasst Typkonvertierungen, etwa die Umwandlung uneinheitlich formatierter Preisangaben in einen numerischen Wert und textueller Wahrheitswerte in einen Boolean-Datentyp, und prüft damit die korrekte Behandlung von Datentypen. Die mittlere Stufe verlangt die Berechnung abgeleiteter Spalten aus bestehenden Feldern, beispielsweise die Ermittlung eines Gesamtbetrags sowie die Extraktion einzelner Datumsbestandteile, und adressiert damit zeilenweise Berechnungslogik. Die anspruchsvolle Stufe bildet die komplexeste Aufgabe der gesamten Untersuchung und verbindet mehrere Verarbeitungsschritte: Sie erfordert die Verknüpfung der drei Tabellen über ihre Schlüsselbeziehungen, eine begleitende Bereinigung der beteiligten Felder sowie eine anschließende Gruppierung und Aggregation der Ergebnisse. Damit prüft sie, ob die Modelle eine mehrstufige, aus Verknüpfung, Bereinigung und Aggregation zusammengesetzte Verarbeitungslogik korrekt umsetzen können, wie sie für reale ETL-Strecken charakteristisch ist [1, 2].

4.3.3. Duplikaterkennung

Die Aufgaben zur Duplikaterkennung zielen auf die Eindeutigkeit des Datenbestands und damit auf die Identifikation und Auflösung mehrfach vorhandener Datensätze [17, 18]. Die einfache Stufe beschränkt sich auf exakte Duplikate, bei denen vollständig identische Datensätze zu entfernen sind, und ist mit einem deterministischen Abgleich vollständig lösbar. Die mittlere Stufe verlangt die schlüsselbasierte Entdoppelung anhand eines eindeutigen Merkmals, wobei bei mehreren übereinstimmenden Datensätzen anhand einer definierten Regel der zu behaltende Datensatz ausgewählt werden muss. Die anspruchsvolle Stufe adressiert schließlich unscharfe Duplikate (Fuzzy-Duplikate), bei denen derselbe reale Sachverhalt in abweichenden Schreibweisen vorliegt, etwa durch Tippfehler, unterschiedliche Groß- und Kleinschreibung oder geringfügig abweichende Formatierungen. Da hier ein exakter Abgleich nicht ausreicht und stattdessen eine Ähnlichkeitsbetrachtung im Sinne der von Elmagarmid et al. beschriebenen Verfahren erforderlich ist, stellt diese Aufgabe die zweite semantisch geprägte Anforderung dar, an der sich die semantischen Fähigkeiten der Modelle gegenüber regelbasierten Verfahren zeigen [6, 18].

4.4. Prompting-Strategien

Da LLMs ihre Aufgaben primär über das In-Context-Lernen lösen, kommt der Gestaltung der Eingabe eine entscheidende Bedeutung für die Ergebnisqualität zu, weshalb die Prompting-Strategie als eigenständiger Untersuchungsfaktor in das Design aufgenommen wird [28, 29]. Angesichts der in Abschnitt 2.5.1 angesprochenen uneinheitlichen Begriffsbildung im Feld werden für diese Arbeit klar abgegrenzte Strategien definiert, die sich in ihrem Aufwand und dem Umfang der bereitgestellten Hilfestellung systematisch unterscheiden und gegeneinander verglichen werden.

Den Ausgangspunkt bildet das Zero-Shot-Prompting, bei dem das Modell die Aufgabe allein anhand einer präzisen Anweisung und ohne weitere Beispiele bearbeitet. Diese Strategie dient als Referenzbasis für die vorliegende Aufgabenbeschreibung [25, 28]. Darauf aufbauend wird das Few-Shot-Prompting untersucht, dessen Wirksamkeit Brown et al. an GPT-3 nachgewiesen haben [30]. Es wird hier durch ein einzelnes Beispiel einer anderen Aufgabe operationalisiert und stellt deshalb im engeren Sinne eine One-Shot-Variante dar [25].

Als dritte Strategie wird das Chain-of-Thought-Prompting nach Wei et al. einbezogen, bei dem das Modell zur Entwicklung seiner Lösung in nachvollziehbaren Zwischenschritten angeleitet wird, was sich insbesondere bei den mehrstufigen Aufgaben dieser Untersuchung als wirksam erweisen könnte [28, 31]. Die Untersuchung beschränkt sich bewusst auf diese drei etablierten Grundformen, um eine reine Aufgabenanweisung, eine beispielgestützte Vorlage und eine explizite Schrittanleitung gegenüberzustellen. Die drei Vorlagen bilden keine vollständige oder streng geordnete Skala des Prompt-Aufwands. Um faire Vergleichsbedingungen zu gewährleisten, werden die Strategien für alle Aufgaben nach einem einheitlichen Aufbau formuliert und versioniert vorgehalten, sodass jede eingesetzte Prompt-Variante eindeutig dokumentiert und wiederverwendbar bleibt [29, 32].

4.5. Evaluationskriterien

Damit die Ergebnisse aller Verfahren über Modelle, Prompting-Strategien und Aufgaben hinweg vergleichbar gemessen werden können, werden die Bewertungskriterien aus den in Abschnitt 2.2.1 eingeführten Dimensionen der Datenqualität abgeleitet und operationalisiert [16]. Grundlage jeder Bewertung ist der Abgleich der erzeugten Ausgabedatei mit der für jede Aufgabe bekannten Soll-Lösung. Die folgenden Unterabschnitte beschreiben die vier herangezogenen Kriterien Genauigkeit, Vollständigkeit, Konsistenz und Effizienz. Die Stabilität der Ergebnisse wird, wie in Abschnitt 4.1 festgelegt, nicht als separates Einzelkriterium erhoben, sondern daraus abgeleitet, wie stark die genannten Qualitätskennzahlen über die wiederholten Ausführungen einer Konfiguration streuen: Eine geringe Streuung entspricht dabei einer hohen, eine große Streuung einer geringen Stabilität [5, 6].

4.5.1. Genauigkeit

Die Genauigkeit erfasst, inwieweit die erzeugten Werte mit der Soll-Lösung übereinstimmen, und bildet das zentrale Korrektheitskriterium [12, 16]. Sie wird als Anteil der korrekt verarbeiteten Werte am Gesamtbestand operationalisiert, sodass auch teilweise korrekte Ergebnisse differenziert bewertet werden. Da programmatisch erzeugte Ergebnistabellen korrekte Werte häufig in abweichender Zeilenreihenfolge liefern, werden beide Tabellen vor dem Abgleich einheitlich sortiert.

Weicht die Datensatzanzahl von der Soll-Lösung ab, lassen sich die Zeilen nicht mehr eindeutig zuordnen. Ein strikter Abgleich würde ein nahezu vollständiges Ergebnis dann genauso bewerten wie ein vollständig falsches und gerade bei der Duplikaterkennung, wo die Datensatzanzahl das eigentliche Ergebnis ist, die gesamte inhaltliche Information verwerfen. Deshalb werden zwei Ausprägungen unterschieden. Die *strenge* Genauigkeit setzt die Übereinstimmung von Spaltenmenge und Datensatzanzahl voraus und beträgt andernfalls null. Die *abgegliche* Genauigkeit verlangt lediglich die Übereinstimmung des Zielschemas, ordnet die Tabellen über die gemeinsamen Schlüssel beziehungsweise die sortierte Reihenfolge einander zu und verbucht überzählige oder fehlende Datensätze als Abweichung. Numerische Felder werden mit einer Toleranz verglichen.

Als Hauptkennzahl dient die abgegliche Genauigkeit, da sie den Grad der Aufgabenerfüllung abbildet und nicht nur deren Gelingen oder Scheitern. Die strenge Ausprägung wird herangezogen, wo die exakte Datensatzanzahl selbst Teil der Anforderung ist. Ergänzend weist die *Erfolgsquote* den Anteil der Läufe aus, die eine strukturell verwertbare Ausgabe erzeugt haben, also Zielschema und erwartete Datensatzanzahl einhalten. Ihre Differenz zur Genauigkeit wird nur deskriptiv berichtet, sie lässt keine direkte Bestimmung der Zahl strukturell abweichender Läufe zu. Fehlgeschlagene Ausführungen gehen mit dem Wert null in alle Mittelwerte ein.

Bewertet werden sämtliche Spalten der Soll-Tabelle. Schlüsselspalten werden bei einem schlüsselbasierten Abgleich zur Zuordnung verwendet und anschließend nicht nochmals als Inhaltzellen gezählt, während alle übrigen Soll-Spalten in den Zähler und Nenner eingehen. Bei eindeutigen Schlüsseln werden gleiche Schlüssel über einen Vorkommensindex zugeordnet. Fehlt ein eindeutiger Schlüssel, werden Soll- und Ist-Tabelle nach allen Soll-Spalten als Text sortiert und positionsweise verglichen. Für fehlende oder zusätzliche Zeilen gilt $N = \max(N_{\text{Soll}}, N_{\text{Ist}})$, für die bewerteten Inhaltfelder C damit

$$\text{Genauigkeit} = \frac{\text{korrekte Zellen}}{N \cdot C}.$$

Numerische Werte gelten bei einer relativen Toleranz von 10^{-5} und einer absoluten Toleranz von 0,01 als gleich, Textwerte werden nach einer einheitlichen Ersetzung fehlender Werte durch einen Platzhalter exakt verglichen, sodass zwei fehlende Werte als übereinstimmend gelten. Beispiel: Bei einer Soll-Tabelle mit 5 Zeilen, 1 Schlüsselspalte und 4 Inhaltspfeln sowie einer Ist-Tabelle mit 4 korrekt zugeordneten Zeilen sind $4 \cdot 4 = 16$ von $5 \cdot 4 = 20$ Zellen korrekt, die abgegliche Genauigkeit beträgt somit 80 Prozent. Die fehlende Zeile zählt vollständig als Abweichung.

4.5.2. Vollständigkeit und Konsistenz

Die Vollständigkeit gibt an, inwieweit das Ergebnis alle geforderten Datensätze und Felder enthält, ohne dass erwartete Werte fehlen oder unzulässig entfernt werden. Die Konsistenz bezieht sich auf die Widerspruchsfreiheit der Ausgabe, insbesondere auf die Übereinstimmung von Datentypen und Formaten mit dem Zielschema [12, 16]. Beide werden über den Abgleich mit der Soll-Lösung erfasst, die eine über das Vorhandensein aller erwarteten Spalten und die korrekte Datensatzanzahl, die andere über Schemakonformität und Typisierung. Die Vollständigkeit ist dabei vor allem für die Duplikaterkennung von Bedeutung, bei der sowohl unzureichendes als auch übermäßiges Entfernen sie verletzt, die Konsistenz vor allem für die Transformationsaufgaben, bei denen die korrekte Typisierung ein wesentliches Ergebnismerkmal ist.

Anders als die Genauigkeit werden beide in der Auswertung nicht als eigenständige Kennzahlen berichtet, sondern erfüllen eine vorgelagerte Funktion. Die Vollständigkeit entscheidet darüber, ob ein Ergebnis überhaupt vergleichbar ist, und geht in die abgeglichene Genauigkeit ein, indem fehlende oder überzählige Datensätze dort als Abweichung gewertet werden. Wo der Umfang eigenständige Aussagekraft besitzt, etwa bei der Zahl der entfernten Duplikate, wird er unmittelbar als Datensatzanzahl berichtet. Die Konsistenz wirkt als Ausschlusskriterium: Eine Ausgabe, die das Zielschema verletzt, gilt als strukturell nicht verwertbar, geht mit dem Wert null in die Genauigkeit ein und wird als *Schema-Bruch* gesondert von einem Programmabbruch ausgewiesen. Berichtet werden in Kapitel 8 folglich zwei Qualitätsgrößen, die abgeglichene Genauigkeit und die Erfolgsquote, in denen Vollständigkeit und Konsistenz als strukturelle Voraussetzungen aufgehen, sowie die Effizienzgrößen des folgenden Abschnitts.

4.5.3. Effizienz

Die Effizienz erfasst den für die Verarbeitung erforderlichen Aufwand und ergänzt die qualitätsbezogenen Kriterien um die für den praktischen Einsatz entscheidende Wirtschaftlichkeitsdimension [26, 32]. Erhoben werden die Verarbeitungszeit eines Aufrufs, der Verbrauch an Token sowie die daraus resultierenden Nutzungskosten. Diese Kennzahlen sind insbesondere für die vergleichende Bewertung der Prompting-Strategien relevant, da aufwändigere Strategien mit längeren Eingaben einen höheren Token-Verbrauch verursachen und somit zwischen Ergebnisqualität und Effizienz abzuwägen ist [29, 32]. Zugleich ermöglichen sie den praxisnahen Vergleich zwischen kommerziell betriebenen und lokal ausgeführten Modellen sowie die Gegenüberstellung mit der klassischen Pipeline, deren Ausführung keine vergleichbaren Modellkosten verursacht.

4.6. Vergleich gegen eine regelbasierte ETL-Pipeline

Um die Ergebnisse der LLM-gestützten Verarbeitung einordnen zu können, wird als Vergleichsmaßstab eine klassische, regelbasiert implementierte ETL-Pipeline herangezogen, die dieselben neun Aufgaben auf demselben Datensatz löst. Diese Pipeline bildet eine einfache regelbasierte Referenz, bei dem die Verarbeitungslogik manuell und explizit in Form von Regeln und Skripten vorgegeben wird, und sie wird ebenfalls

programmatisch auf Basis der Bibliothek Pandas umgesetzt, um eine technisch vergleichbare Ausgangslage herzustellen [13]. Sie dient als Referenzpunkt, gegen den sich die Stärken und Schwächen der KI-gestützten Ansätze messen lassen, und liefert damit die Grundlage zur Beantwortung von FF1.

Charakteristisch für diese Vergleichsbasis ist, dass sie deterministische und vollständig nachvollziehbare Ergebnisse erzeugt [2, 3]. Damit bildet die regelbasierte Pipeline insbesondere bei den klar formalisierbaren Aufgaben die Referenz, während die semantisch geprägten Aufgaben der Länder-Vereinheitlichung und der Fuzzy-Duplikaterkennung gezielt an die Grenzen rein regelbasierter Verfahren führen. Diese arbeiten zwar auch bei solchen Aufgaben deterministisch, erfassen die zugrunde liegende semantische Ähnlichkeit jedoch nur eingeschränkt. Die Pipeline wird mit den identischen Evaluationskriterien aus Abschnitt 4.5 bewertet, sodass ein unmittelbarer Vergleich zwischen klassischem und KI-gestütztem Vorgehen unter gleichen Bedingungen möglich ist und sich auf dieser Grundlage in Kapitel 9 Handlungsempfehlungen ableiten lassen. Ein hybrider Ansatz, der beide Verfahren kombiniert, ist dabei nicht selbst Gegenstand der Evaluierung, obwohl sich die Nachvollziehbarkeit regelbasierter und die Flexibilität KI-gestützter Verfahren in der Praxis häufig ergänzen [2, 40].

5. Erstellung des synthetischen Datensatzes

Das in Kapitel 4 entworfene Untersuchungsdesign setzt einen Datensatz voraus, dessen Eigenschaften exakt bekannt sind: Nur wenn die enthaltenen Datenqualitätsprobleme gezielt eingebracht wurden und für jede Aufgabe eine eindeutige Soll-Lösung existiert, lassen sich die in Abschnitt 4.5 definierten Kriterien überhaupt objektiv messen. Dieses Kapitel dokumentiert daher die Erstellung des synthetischen Datensatzes. Er wird einmalig vorab erzeugt und über sämtliche Läufe unverändert verwendet. Seine Erzeugung erfolgt in zwei Schritten, indem zunächst eine vollständig saubere, schemakonforme Fassung entsteht und diese anschließend kontrolliert verfälscht wird. Es schließt unmittelbar an die in Abschnitt 4.3 definierten neun ETL-Aufgaben an und zeigt, wie genau die dort geforderten Probleme in kontrolliertem Umfang eingebracht werden.

Zunächst werden die aus dem Untersuchungsdesign abgeleiteten Anforderungen begründet (Abschnitt 5.1), anschließend das Datenmodell und die saubere Zielstruktur beschrieben (Abschnitt 5.2). Darauf folgt die detaillierte Zuordnung der gezielt eingebauten Qualitätsprobleme zu den einzelnen Aufgaben (Abschnitt 5.3), bevor das zweistufige Generierungsverfahren (Abschnitt 5.4) und die abschließende Validierung des Datensatzes (Abschnitt 5.5) dargelegt werden. Der so erzeugte Datensatz bildet die Eingabe für die technische Umsetzung in Kapitel 6 und die Durchführung der Experimente in Kapitel 7.

5.1. Anforderungen an den Datensatz

Aus den Forschungsfragen und dem in Kapitel 4 festgelegten Design ergeben sich vier Anforderungen:

Repräsentativität: Der Datensatz muss die in Abschnitt 2.2.2 beschriebenen, praxistypischen Datenqualitätsprobleme abbilden, und zwar gerade jene, die den neun in Abschnitt 4.3 definierten Aufgaben entsprechen, sodass jede Aufgabe an einem tatsächlich vorhandenen Mangel ansetzt [16, 17].

Kontrollierbarkeit: Sollzustand sowie Art und Umfang der eingebrachten Fehler müssen exakt bekannt sein, da die Verarbeitungsqualität gegen eine bekannte Soll-Lösung gemessen wird. Bei realen Beständen wäre diese Referenz nur mit erheblichem Annotationsaufwand und verbleibender Unsicherheit herstellbar. Die Wahl regelbasiert erzeugter Testdaten ist eine Setzung dieses Versuchsdesigns. Sie ermöglicht die Kontrolle der eingebrachten Muster, belegt aber keine Repräsentativität für reale Datenbestände.

Reproduzierbarkeit: Die Generierung muss deterministisch sein, damit die Datengrundlage keine Variabilität beisteuert, die der querschnittlich erhobenen Reproduzierbarkeit der Modellergebnisse zugerechnet würde.

Angemessener Umfang: Der Datensatz muss groß genug sein, um die drei Schwierigkeitsstufen jeder Aufgabe auszuprägen, und klein genug, um die in Abschnitt 4.5.3 erhobenen Aufwandskennzahlen nicht durch reine Datenmenge zu dominieren.

5.2. Datenmodell und -struktur

Als Anwendungsszenario dient ein bewusst einfach gehaltenes E-Commerce-Datenmodell, das aus den drei Tabellen Kunden, Produkte und Bestellungen besteht. Diese Wahl ist kein Selbstzweck, sondern folgt aus den Aufgaben: Das Szenario ist allgemein verständlich, deckt sämtliche in Abschnitt 4.3 geforderten Problemtypen ab und stellt über die Beziehung der Bestellungen zu Kunden und Produkten zugleich die Schlüsselstruktur bereit, die für die anspruchsvolle Transformationsaufgabe aus Verknüpfung, Bereinigung und Aggregation benötigt wird [1, 2]. Die Tabelle der Bestellungen verweist über Fremdschlüssel auf Kunden und Produkte und verbindet die drei Tabellen so zu einem kleinen relationalen Modell.

Grundlage des Modells ist die Definition eines sauberen Zielschemas für jede Tabelle, das den fehlerfreien Sollzustand festlegt. Für die Kundentabelle umfasst dieses Schema eine eindeutige Kundennummer, den vollständigen Namen, die E-Mail-Adresse, einen zweistelligen Länder-Code nach ISO-3166-1-alpha-2 sowie das Registrierungsdatum. Die Produkttabelle besteht aus einer Produktnummer, der Bezeichnung, einer Kategorie, dem Preis als nichtnegativem Dezimalwert und einem Wahrheitswert zur Verfügbarkeit. Die Bestelltabelle schließlich verknüpft Bestell-, Kunden- und Produktnummer mit der bestellten Menge, dem Einzelpreis zum Bestellzeitpunkt und dem Bestelldatum. Der Gesamtbetrag ist dort bewusst *nicht* enthalten, da seine Berechnung

Gegenstand der mittleren Transformationsaufgabe ist. Er tritt erst in deren Soll-Lösung als abgeleitete Spalte hinzu.

Dieses saubere Schema erfüllt eine doppelte Funktion: Es beschreibt einerseits das von den Modellen zu erreichende Zielformat und dient andererseits, in seiner konkret erzeugten Ausprägung, als bekannte Referenz für die Bewertung. Die typisierte, maschinenlesbare Festlegung des Schemas stellt dabei sicher, dass die als Referenz erzeugten Daten strikt schemakonform sind und damit als verlässliche Ground Truth taugen [12, 16].

Der Umfang des Datensatzes ist so gewählt, dass er die genannte Abwägung zwischen Vielfalt und Verarbeitungsaufwand einlöst. Die Kundentabelle umfasst in ihrer sauberen Fassung fünfhundert Datensätze, die Produkttabelle einhundert und die Bestelltabelle zweitausend Datensätze. Diese Größenordnung ist eine Setzung des Versuchsdesigns: groß genug, um Qualitätsprobleme merklich einzustreuen und die Aggregation über Länder und Kategorien belastbar zu füllen, klein genug für die wiederholte Ausführung über alle Kombinationen. Ob sie trägt, belegen erst die Aufwandskennzahlen in Abschnitt 8.8.

5.3. Eingebaute Datenqualitätsprobleme

Kern der Datensatzkonstruktion ist das gezielte Einbringen von Qualitätsproblemen, das dieser Abschnitt entlang der drei in Abschnitt 4.3 definierten Aufgabenkategorien beschreibt. Jedes Problem ist dabei nicht beliebig gewählt, sondern korrespondiert mit genau einer Aufgabe und einer der in Abschnitt 2.2.1 eingeführten Qualitätsdimensionen, sodass sich die spätere Bewertung lückenlos auf einen bewusst gesetzten Mangel zurückführen lässt [16, 17]. Auf diese Weise ist die in Abschnitt 4.3 eingeführte Matrix aus neun Aufgaben und drei Schwierigkeitsstufen unmittelbar in der Struktur des Datensatzes verankert.

Die Probleme der *Datenbereinigung* sind ausschließlich in der Kundentabelle verortet und in aufsteigender Schwierigkeit gestaffelt: zusätzliche Leerzeichen in den Namensfeldern und entfernte Länderangaben für die einfache Stufe, das in vier Darstellungen abgelegte Registrierungsdatum für die mittlere und die Verfälschung der Länderangaben durch wechselnde Schreibweisen, Sprachen, Abkürzungen und gezielte Tippfehler für die anspruchsvolle [12, 17]. Gerade die letztgenannte Verfälschung bildet die in Abschnitt 4.3.1 hervorgehobene semantische Achse ab, da ihre Auflösung ein inhaltliches Verständnis der Werte und nicht nur einen syntaktischen Abgleich erfordert [6, 46].

Die Probleme der *Datentransformation* betreffen die Produkt- und die Bestelltabelle und knüpfen an die Dimension der Konsistenz an. In der Produkttabelle werden die numerischen Preisangaben in uneinheitliche textuelle Schreibweisen überführt und die Verfügbarkeit von einem Wahrheitswert in wechselnde textuelle Entsprechungen umgeschrieben, womit die einfache Aufgabe der Typkonvertierung ihre Grundlage erhält. Die mittlere Stufe benötigt keine eigene Verfälschung, sondern nutzt die vorhandene Struktur der Bestelltabelle, aus der sich Gesamtbetrag und Datumsbestandteile ableiten lassen. Die anspruchsvolle Stufe schließlich verlangt die Verknüpfung aller drei Tabellen über ihre Schlüsselbeziehungen, eine begleitende Bereinigung der zuvor be-

schriebenen Mängel und eine abschließende Aggregation über Land und Produktkategorie. Sie bündelt damit bewusst mehrere eingebaute Probleme in einem mehrstufigen Verarbeitungsschritt, wie er für reale ETL-Strecken charakteristisch ist [1, 2].

Die Probleme der *Duplikaterkennung* zielen auf die Eindeutigkeit der Kundentabelle. In die saubere Ausgangsbasis von 500 Kunden werden für die einfache und mittlere Stufe 20 vollständig identische Datensätze eingefügt, die sowohl als exakte Duplikate als auch über die mehrfach vergebene E-Mail-Adresse erkennbar sind. Für die anspruchsvolle Stufe treten weitere 20 Datensätze als unscharfe Duplikate (Fuzzy-Duplikate) mit abweichender Schreibweise von Name und E-Mail hinzu. Die verfälschte Kundentabelle (`customers_raw.csv`) umfasst somit insgesamt 540 Zeilen. Die Soll-Lösungen der drei Duplikataufgaben unterscheiden sich entsprechend im Umfang. Die einfache und die mittlere Stufe verlangen jeweils nur die Entfernung der 20 exakten beziehungsweise schlüsselgleichen Datensätze, ihre Soll-Lösung umfasst folglich $540 - 20 = 520$ Datensätze. Die anspruchsvolle Stufe zielt dagegen auf die tatsächlich eindeutigen Personen und verlangt damit die Auflösung beider Duplikatklassen, sodass ihre Soll-Lösung $540 - 20 - 20 = 500$ Datensätze umfasst, die 500 eindeutigen Kunden des Sollzustands. Dieselben 500 Datensätze werden auch in der durchgängigen Bereinigung und in der verketteten Pipeline erreicht, dort jedoch über zwei nacheinander ausgeführte Schritte [6, 17]. Der Anteil der eingebrachten Probleme ist so bemessen, dass er die Ergebniskennzahlen merklich beeinflusst, ohne den Charakter des Datensatzes als überwiegend sauberen Bestand zu verfälschen.

Tabelle 5.1.: Beispiele tatsächlich erzeugter Rohwerte und ihrer Sollwerte, gegliedert nach der zugehörigen Aufgabe. Sichtbare Leerzeichen sind mit `_` gekennzeichnet.

Aufgabe	Rohwert	Sollwert
Leerzeichen fehlender Wert	<code>__Sigfried Römer-Butte__</code> (leer)	Sigfried Römer-Butte UNKNOWN
Datumsformat	2024-12-11 (ISO)	2024-12-11
Datumsformat	02.01.2023 (deutsch)	2023-01-02
Datumsformat	August 09 2025 (US)	2025-08-09
Datumsformat	1628121600 (Unix)	2021-08-05
Länderangabe	Deutschland / Germany	DE
Länderangabe	Deutshcland (Tippfehler)	DE
Länderangabe	SUI / Schweiz	CH
Länderangabe	Holland / Niederlande	NL
Länderangabe	England / UK	GB
Preisangabe	494.35 EUR	494.35
Preisangabe	€6.53	6.53
Preisangabe	155,17	155.17
Verfügbarkeit	yes / true / 1	True
Verfügbarkeit	no / false / 0	False

Wie sich das konkret in den Daten niederschlägt, zeigt Tabelle 5.1 anhand tatsächlicher Zeilen des erzeugten Datensatzes. Die Gegenüberstellung von Roh- und Sollwert macht zugleich sichtbar, weshalb die Aufgaben unterschiedlich schwer sind: Während sich Leerzeichen und Datumsformate durch eine endliche Menge syntaktischer

Regeln vollständig auflösen lassen, ist bei den Länderangaben für jeden einzelnen Wert zu entscheiden, welches Land gemeint ist. Fehlende Werte werden nicht gelöscht, sondern einheitlich als UNKNOWN gekennzeichnet, sodass die Zeilenzahl erhalten bleibt und die Vollständigkeit im Sinne von Abschnitt 4.5.2 messbar bleibt.

Da der Kern eines Duplikats im Verhältnis zweier Datensätze zueinander liegt, stellt Tabelle 5.2 sie zeilenweise gegenüber. Bei den exakten Duplikaten ist der eingefügte Datensatz in allen Feldern einschließlich der Kundennummer identisch. Bei den unscharfen weichen Name und E-Mail-Adresse durch genau jene Muster ab, die auch in realen Beständen entstehen, veränderte Groß- und Kleinschreibung, ein zusätzliches Leerzeichen, ein Punkt im lokalen Teil der Adresse. Die inhaltliche Identität ist bei manueller Sichtung erkennbar, ist über einen exakten Vergleich jedoch nicht mehr zugänglich, weshalb diese Aufgabe die in Abschnitt 4.3.3 beschriebene Ähnlichkeitsbetrachtung erzwingt.

Tabelle 5.2.: Eingefügte Duplikate im Vergleich zu ihrem Ausgangsdatsatz. Bei den unscharfen Duplikaten sind die abweichenden Stellen die einzige Änderung gegenüber dem Original.

Art	Nr.	Name	E-Mail
exakt (Original)	21	Janina Pieper-Junken	patriziakarge@example.com
exakt (Duplikat)	21	Janina Pieper-Junken	patriziakarge@example.com
unscharf (Original)	49	Herr Alban Kusch	janeroskoth@example.net
unscharf (Duplikat)	503	Herr░░Alban░░Kusch	░janeroskoth@example.net░
unscharf (Original)	494	Dr. Käthe Mülischen	doris08@example.net
unscharf (Duplikat)	501	DR. KÄTHE MÜLICHEN	DORIS08@example.net
unscharf (Original)	391	Nadine Butte	shertrampf@example.org
unscharf (Duplikat)	508	nadine butte	shert.rampf@example.org

Aus den 500 sauberen Kundendatensätzen entstehen durch das Anfügen von 20 exakten und 20 unscharfen Duplikaten insgesamt 540 Rohdatensätze, was die in Abschnitt 5.2 genannte Zielgröße von 500 eindeutigen Kunden als Sollzustand der Duplikataufgaben festlegt. Die Häufigkeitsverteilung der Länderangaben bestätigt zugleich die beabsichtigte Streuung: Für die zehn im Datensatz vertretenen Länder liegen je Land drei bis sechs verschiedene Schreibweisen vor, von Standardcodes wie DE über ausgeschriebene deutsche und englische Namen bis zu untypischen Kürzeln wie SUI für die Schweiz oder GER für Deutschland, und 38 Angaben fehlen vollständig.

5.4. Generierungsverfahren und verwendete Werkzeuge

Die Erzeugung des Datensatzes folgt einem zweistufigen Prinzip, das die in Abschnitt 5.1 geforderte Kontrollierbarkeit unmittelbar umsetzt. Im ersten Schritt werden für jede Tabelle vollständig saubere, strikt schemakonforme Daten erzeugt, die den fehlerfreien Sollzustand verkörpern. Im zweiten Schritt werden aus dieser sauberen Fassung durch eine kontrollierte Verfälschung die tatsächlich zu verarbeitenden Roh-

daten abgeleitet, während die saubere Fassung selbst als Referenz erhalten bleibt. Dieses Vorgehen kehrt den üblichen Ablauf einer ETL-Strecke bewusst um: Nicht die Bereinigung wird nachträglich auf unbekannte Rohdaten angewandt, sondern die Verunreinigung wird gezielt auf bekannte saubere Daten aufgebracht, sodass die exakte Umkehrung jeder Verfälschung und damit die eindeutige Soll-Lösung jeder Aufgabe von vornherein feststeht.

Für die Erzeugung realistisch wirkender Basiswerte wie Namen und E-Mail-Adressen wird die Bibliothek `Faker` eingesetzt, deren deutschsprachige Lokalisierung dem Szenario angemessene Werte liefert. Die kontrollierte Verfälschung, also das Einstreuen von Leerzeichen, fehlenden Werten, uneinheitlichen Formaten, verfälschten Länderangaben sowie exakten und unscharfen Duplikaten, erfolgt anschließend durch eigens implementierte Routinen, die jede der in Abschnitt 5.3 beschriebenen Problemklassen erzeugen. Sowohl die Basisdaten als auch die Verfälschung werden über einen zentralen Zufallszahlengenerator gesteuert, der mit einem festen Startwert initialisiert wird. Dadurch ist die Generierung vollständig deterministisch: Derselbe Startwert führt stets zu einem identischen Datensatz, während unterschiedliche Startwerte voneinander unabhängige, aber strukturell gleichartige Datensätze erzeugen. Die erzeugten Rohdaten werden in einem gängigen tabellarischen Austauschformat abgelegt und dienen zusammen mit der separat gespeicherten sauberen Referenz als Eingabe für die in Kapitel 6 beschriebene Verarbeitungsumgebung.

5.5. Validierung des Datensatzes

Der Datensatz wird vor seiner Verwendung programmatisch geprüft. Ein Validierungsskript erzeugt ihn dazu aus dem hinterlegten Startwert neu und führt vier Prüfungen aus, von denen jede fehlschlagen und den Datensatz damit verwerfen kann. Die erste validiert die saubere Referenz zeilenweise gegen die typisierte Schemadefinition aus Abschnitt 5.2: Datentyp, Wertebereich und Format jedes Feldes müssen der Festlegung entsprechen [12, 16]. Nur unter dieser Voraussetzung ist der spätere Abgleich der Modellergebnisse mit der Referenz aussagekräftig.

Die zweite Prüfung weist für jede der neun Aufgaben aus Abschnitt 4.3 den Mangel nach, an dem sie ansetzt, und zählt ihn aus: Randleerzeichen und fehlende Werte, die Zahl der auftretenden Datums- und Preisformate, die Zahl der vom Zielschema abweichenden Länderangaben sowie mehrfach vorkommende und unscharf abgewandelte Datensätze. Die dritte stellt sicher, dass zu jeder Aufgabe eine nicht leere Soll-Lösung vorliegt, was die in Abschnitt 4.5 definierten Kriterien überhaupt erst berechenbar macht [17]. Die vierte erzeugt den Datensatz ein zweites Mal und vergleicht beide Fassungen bytegenau, was die Reproduzierbarkeit der Generierung bestätigt. Für den verwendeten Datensatz bestehen sämtliche Prüfungen.

Mit dieser Validierung steht ein Datensatz zur Verfügung, dessen Sollzustand, Fehlerbild und Erzeugung vollständig bekannt und nachvollziehbar sind und der damit die kontrollierte Vergleichsgrundlage für die weitere Untersuchung bildet. Zugleich ist die dem Vorgehen innewohnende Einschränkung zu benennen: Ein synthetischer Datensatz kann die Vielfalt und die unvorhergesehenen Unregelmäßigkeiten realer Datenbestände nur annähern, weshalb die aus ihm gewonnenen Ergebnisse unter diesem

Vorbehalt zu interpretieren und im Rahmen der Limitationen in Kapitel 10 erneut aufzugreifen sind [37, 38]. Auf der so geschaffenen Grundlage kann im folgenden Kapitel 6 die technische Umsetzung der Verarbeitungsumgebung beschrieben werden, welche die hier erzeugten Daten sowohl der klassischen Pipeline als auch den LLM-gestützten Verfahren zuführt.

6. Implementierung

Das vorangegangene Kapitel 4 hat das Untersuchungsdesign entworfen und der Datensatz aus Kapitel 5 stellt die unveränderliche Datengrundlage bereit. Dieses Kapitel beschreibt, wie dieses Design in ein lauffähiges Softwaresystem überführt wurde, mit dem sich die vollständige Matrix aus Modell, Prompting-Strategie und ETL-Aufgabe automatisiert durchführen und einheitlich bewerten lässt. Zunächst wird die modulare Systemarchitektur und die Versuchsumgebung dargestellt (Abschnitt 6.1), gefolgt von den eingesetzten Technologien und Bibliotheken (Abschnitt 6.2). Anschließend werden die Umsetzung der klassischen Vergleichspipeline (Abschnitt 6.3), die Integration der LLMs über eine einheitliche Schnittstelle (Abschnitt 6.4) sowie die konkrete Realisierung der Prompting-Strategien (Abschnitt 6.5) beschrieben. Den Abschluss bildet das automatisierte Mess- und Auswertungsverfahren (Abschnitt 6.6), das die in Abschnitt 4.5 definierten Kriterien technisch operationalisiert und damit die Grundlage für die Durchführung in Kapitel 7 schafft.

6.1. Systemarchitektur und Versuchsumgebung

Das System ist als modulares Python-Projekt aufgebaut, dessen Komponenten entlang der einzelnen Verarbeitungsschritte getrennt sind und über klar umrissene Schnittstellen zusammenwirken. Die Ausführung der Versuche übernimmt eine eigene Komponente, im Folgenden als Experiment-Runner bezeichnet, welche Konfiguration, Modellaufruf, Ausführung und Bewertung je Lauf zusammenführt. Diese Trennung folgt dem Ziel, die einzelnen Faktoren des Versuchsdesigns unabhängig voneinander austauschen zu können, ohne den übrigen Ablauf zu verändern, und erleichtert zugleich die Nachvollziehbarkeit der Ergebnisse. Abbildung 6.1 zeigt die sechs Module und den Weg, den ein einzelner Lauf durch sie nimmt.

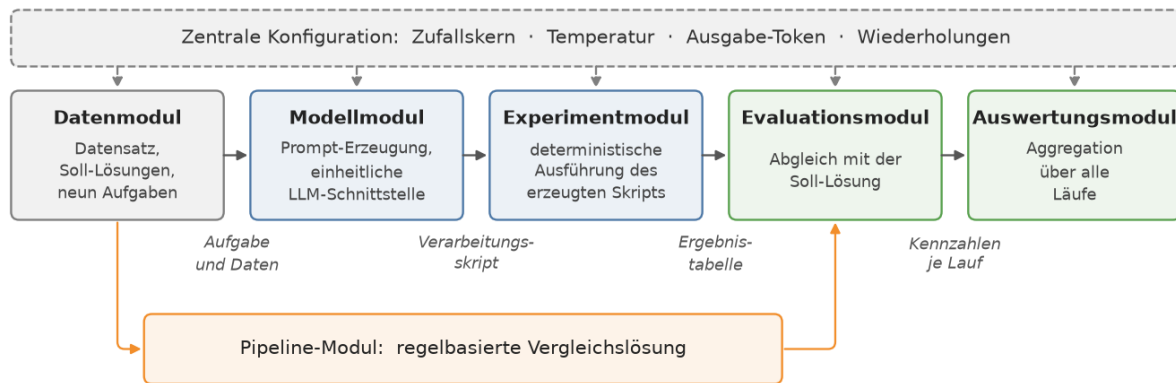


Abbildung 6.1.: Die sechs Module des Versuchsaufbaus und der Weg, den ein einzelner Lauf durch sie nimmt. An jedem Übergang ist das übergebene Artefakt benannt. Die regelbasierte Vergleichspipeline arbeitet auf derselben Datengrundlage und mündet in dieselbe Bewertung.

Entsprechend der in Abschnitt 4.1 getroffenen Gestaltungsentscheidung erzeugt das Modell dabei nicht das Ergebnis unmittelbar, sondern ein ausführbares Verarbeitungsskript. Sämtliche steuerbaren Randbedingungen sind in einer zentralen Konfiguration gebündelt, die unter anderem den Zufallskern für die Datenerzeugung, die konstant niedrig gehaltene Temperatur, die Obergrenze der Ausgabe-Token sowie die Anzahl der Wiederholungen je Konfiguration festlegt. Diese zentrale und explizit hinterlegte Parametrisierung ist eine wesentliche Voraussetzung für die in Abschnitt 4.5 als querschnittliche Dimension eingeführte Reproduzierbarkeit, da sie die Bedingungen jedes Durchlaufs eindeutig dokumentiert. Die Ausführung erfolgt durchgängig auf demselben Rechner unter identischer Laufzeitumgebung. Die konkreten Angaben dazu finden sich in Abschnitt 7.2.

6.2. Eingesetzte Technologien und Bibliotheken

Als Implementierungssprache dient Python: Sämtliche untersuchten Modellanbieter stellen offizielle Python-Schnittstellen bereit, und die für die Datenverarbeitung benötigten Bibliotheken, allen voran Pandas, liegen dort in ausgereifter Form vor, wie auch die einschlägigen Übersichtsarbeiten zur automatisierten Datenverarbeitung durchgehend voraussetzen. Die eigentliche Datenverarbeitung stützt sich sowohl in der klassischen Pipeline als auch im vom Modell erzeugten Code auf die Bibliothek Pandas, die sich als De-facto-Standard für tabellarische Verarbeitung etabliert hat. Ihre einheitliche Verwendung stellt sicher, dass die regelbasierte und die KI-gestützte Lösung auf derselben technischen Grundlage arbeiten und damit unmittelbar vergleichbar bleiben. Sämtliche Zwischen- und Ergebnistabellen werden im spaltenorientierten Parquet-Format ausgetauscht, da dieses die Datentypen der Spalten mitführt und so einen typerhaltenden Abgleich zwischen erzeugter Ausgabe und Soll-Lösung ermöglicht, wie er für das Konsistenzkriterium aus Abschnitt 4.5.2 benötigt wird.

Die Anbindung der Modelle erfolgt über die jeweils offiziellen Softwarebibliotheken der Anbieter für die geschlossenen Modelle von OpenAI, Anthropic und Google sowie über

eine lokale Laufzeitumgebung für das offen verfügbare Modell der Llama-Reihe. Diese heterogenen Schnittstellen werden, wie in Abschnitt 6.4 beschrieben, hinter einer einheitlichen Abstraktion zusammengeführt, sodass der Versuchsablauf unabhängig vom konkreten Anbieter bleibt. Ergänzend kommen etablierte Hilfsbibliotheken zum Einsatz: eine typsichere Konfigurationsverwaltung zur Bündelung der Versuchsparameter, eine Auszeichnungssprache zur versionierten Ablage der Prompt-Vorlagen sowie Werkzeuge zur Visualisierung und interaktiven Auswertung, mit denen die in Kapitel 8 dargestellten Tabellen und Abbildungen erzeugt werden.

6.3. Umsetzung der klassischen ETL-Pipeline

Die in Abschnitt 4.6 als Vergleichsmaßstab definierte klassische Pipeline ist als regelbasierte Lösung umgesetzt, die dieselben neun Aufgaben ausschließlich mit fest vorgegebener Verarbeitungslogik und ohne Zugriff auf ein Sprachmodell löst. Für die klar formalisierbaren Aufgaben, etwa das Entfernen exakter Duplikate oder die Typkonvertierung, nutzt sie dieselben deterministischen Routinen, die auch der Erzeugung der Soll-Lösung zugrunde liegen. Für diese Aufgaben ist die explizit formulierte Regel zugleich die korrekte Lösung, sodass die Pipeline hier das exakte Soll-Ergebnis liefert, in Tabelle 8.1 durchgängig 100 Prozent. Dieser Wert ist Konstruktionsmerkmal und nicht Messergebnis: Er folgt daraus, dass dieselben Routinen die Soll-Lösung erzeugen.

Von besonderer Bedeutung für die Aussagekraft des Vergleichs ist die bewusste Gestaltung der Pipeline bei den semantisch geprägten Aufgaben. Für die Vereinheitlichung der Länderangaben greift sie auf eine absichtlich begrenzte Zuordnungstabelle zurück, die ISO-Codes in zwei- und dreistelliger Form, IOC-Kürzel sowie deutsche und englische Ländernamen abdeckt, jedoch weder Tippfehler noch umgangssprachliche Bezeichnungen berücksichtigt. Nicht auflösbare Werte werden einheitlich als unbekannt gekennzeichnet. Für die Erkennung unscharfer Duplikate bildet sie einen normalisierten Schlüssel aus Name und E-Mail-Bestandteil und entfernt darüber Mehrfachnennungen, kann damit aber nur solche Abweichungen erfassen, die sich durch einfache Normalisierung angleichen lassen. Diese Heuristiken sind realistisch für ein rein regelbasiertes Vorgehen, stoßen jedoch genau dort an ihre Grenzen, wo ein inhaltliches Verständnis der Werte erforderlich wäre. Da die Pipeline vollständig deterministisch arbeitet und identische Eingaben stets zu identischen Ausgaben führen, dient sie im weiteren Verlauf als stabiler und nachvollziehbarer Referenzpunkt, gegen den sich die Ergebnisse der Modelle einordnen lassen.

6.4. Integration der LLMs in die Pipeline

Um die in Abschnitt 4.2 ausgewählten Modelle trotz ihrer unterschiedlichen Schnittstellen einheitlich ansprechen zu können, werden sie hinter einer gemeinsamen Abstraktion gekapselt. Jeder Anbieter implementiert dieselbe abstrakte Schnittstelle, die einen Aufruf entgegennimmt und eine standardisierte Antwort zurückliefert, in der neben dem erzeugten Text auch die exakte Modellkennung, der Verbrauch an Eingabe- und Ausgabe-Token sowie die Antwortzeit enthalten sind. Eine vorgeschaltete Erzeugungsinstanz liefert anhand eines Anbieternamens die passende Implementierung, sodass der Versuchsablauf vollständig unabhängig vom konkreten Modell bleibt und die

Faktorachse Modell ohne Eingriffe in die übrige Logik variiert werden kann. Diese standardisierte Antwort ist zugleich die Datenbasis für die spätere Effizienzbewertung nach Abschnitt 4.5.3, da sie die dafür benötigten Kennzahlen bei jedem Aufruf unmittelbar mitführt.

Für den eigentlichen Verarbeitungsschritt folgt dem Ansatz der Code-Generierung: Aus der Modellantwort wird der enthaltene Python-Code extrahiert und anschließend in einem eigenen Betriebssystemprozess mit fester Zeitbegrenzung ausgeführt. Ein Durchlauf gilt nur dann als erfolgreich, wenn der Prozess fehlerfrei endet und tatsächlich die erwartete Ausgabedatei erzeugt wurde. Andernfalls wird das Ergebnis als fehlgeschlagene Ausführung protokolliert. Da an dieser Stelle von einem Modell erzeugter Code ausgeführt wird, ist die Ausführung in einen separaten Prozess mit Zeitlimit ausgelagert. Dieses Vorgehen ist für die kontrollierte, lokale Durchführung der Experimente vertretbar, stellt jedoch bewusst keine vollständige Sicherheitsisolation dar und wäre für einen unbeaufsichtigten Produktivbetrieb entsprechend abzusichern [6, 46].

Ein zentrales Element der Integration ist die Behandlung von Fehlern, die aufgrund der probabilistischen Natur der Modelle und der Nutzung externer Dienste unvermeidlich auftreten. Dabei wird bewusst zwischen zwei Arten von Fehlern unterschieden: Vorübergehende Störungen des Dienstes, die sich etwa in bestimmten Statuscodes oder Verbindungsabbrüchen äußern, werden als transient eingestuft und der betreffende Aufruf mit exponentiell wachsender Wartezeit mehrfach wiederholt. Ein inhaltlich fehlerhaftes, aber technisch wohlgeformtes Ergebnis, etwa ein Skript, das mit einem Programmfehler abbricht, wird hingegen nicht wiederholt, da es einen gültigen Messwert über die Fähigkeit des Modells darstellt und eine Wiederholung das Ergebnis verfälschen würde.

Ergänzend werden anbieterspezifische Besonderheiten aktueller Modelle abgefangen: Lehnt ein Modell einen Sampling-Parameter wie die Temperatur ab, wird dieser automatisch weggelassen und die Anpassung im Ergebnisdatensatz vermerkt, sodass solche Abweichungen bei der Interpretation der Reproduzierbarkeit sichtbar bleiben. Für jeden Aufruf werden schließlich sämtliche relevanten Angaben, von der Modellkennung über den Token-Verbrauch und die Kosten bis hin zum erzeugten Code und dem Ausführungsergebnis, vollständig protokolliert und bilden so die lückenlose Grundlage für die anschließende Auswertung.

Der in Abschnitt 4.1 eingeführte zweite Ausführungsmodus, die direkte In-Context-Verarbeitung, ist über dieselbe einheitliche Modellschnittstelle umgesetzt, unterscheidet sich jedoch im Ablauf. Hier werden die Eingabedaten unmittelbar in den Prompt eingebettet und das Modell angewiesen, die transformierte Ergebnistabelle direkt in einem klar abgegrenzten Format zurückzugeben, das anschließend eingelesen und gegen die Soll-Lösung geprüft wird. Die Auswertung der Antwort ist dabei bewusst robust gegenüber am Ausgabelimit abgeschnittenen Antworten gestaltet, sodass auch unvollständige Ergebnisse noch verarbeitet und als solche erkannt werden.

Da aktuelle Modelle standardmäßig einen internen Denkprozess ausführen, der das für die Ergebnistabelle verfügbare Ausgabebudget aufzehren würde, bevor die eigentlichen Daten erzeugt werden, wird dieses Verhalten für den Direkt-Modus

deaktiviert und das Ausgabelimit angehoben [32]. Anders als bei der wiederholten Code-Erzeugung wird jede Konfiguration in diesem Modus nur einmal ausgeführt, um den durch die eingebetteten Daten erhöhten Token-Verbrauch zu begrenzen. Eine Wiederholung erfolgt ausschließlich bei einem technischen Fehlschlag wie einem transienten Anbieterfehler oder einer leeren beziehungsweise nicht auswertbaren Antwort, nicht hingegen bei einem wohlgeformten, aber inhaltlich abweichenden Ergebnis, das einen gültigen Messwert darstellt.

6.5. Umsetzung der Prompting-Strategien

Die in Abschnitt 4.4 konzipierten Strategien sind als versionierte Vorlagen umgesetzt, die getrennt vom Programmcode in einem eigenen, menschenlesbaren Format abgelegt sind. Jede Vorlage besteht aus einer eindeutigen Kennung, einer Systemanweisung und einer Nutzervorlage mit klar benannten Platzhaltern, in die zur Laufzeit die Aufgabenbeschreibung sowie je nach Ausführungsmodus die Ein- und Ausgabepfade oder die eingebetteten Daten eingesetzt werden. Diese versionierte und vom Code entkoppelte Ablage stellt sicher, dass jede eingesetzte Prompt-Variante eindeutig dokumentiert, unverändert wiederverwendbar und damit nachvollziehbar bleibt, was eine wesentliche Voraussetzung für die Vergleichbarkeit und Reproduzierbarkeit der Ergebnisse ist [29, 32].

Alle drei Vorlagen verwenden dieselben Platzhalter für die eigentliche Aufgabenbeschreibung und die Pfade. Die Zero-Shot-Vorlage enthält keine Demonstration. Die als Few-Shot bezeichnete Vorlage ergänzt genau ein ausgearbeitetes Beispiel einer anderen Aufgabe, sie ist damit eine One-Shot-Operationalisierung, die vor allem Lösungsstil und Format vermittelt. Die dritte Vorlage verlangt eine schrittweise Lösung mit Kommentaren im Code und verändert dazu auch die Systemanweisung. Gemessen wird daher der Effekt dieser konkreten Vorlagen als Ganzes. Unterschiede lassen sich nicht ausschließlich der Beispielzahl oder dem Umfang des Kontexts zuschreiben. Dass die Auswahl und Darstellung von Beispielen selbst relevant sind, wurde auch in aufgabenbezogenen Experimenten untersucht [8].

Für die Direktverarbeitung existiert zu jeder der drei Strategien eine eigene Vorlage, die dieselbe Hilfestellung bietet, aber statt eines Ausgabepfads die Eingabedaten selbst aufnimmt und statt eines Skripts die Ergebnistabelle anfordert. Diese Spiegelung ist Voraussetzung dafür, den Vergleich der beiden Ausführungsmodi in Kapitel 8 mit inhaltlich korrespondierenden Vorlagen zu führen. Die unterschiedlichen Ausgabeanforderungen und Reasoning-Einstellungen bleiben dabei Teil der jeweiligen Ausführungskonfiguration.

6.6. Mess- und Auswertungsverfahren

Die in Abschnitt 4.5 definierten Kriterien werden durch eine automatisierte Bewertungsroutine operationalisiert, die die erzeugte Ausgabedatei mit der bekannten Soll-Lösung abgleicht. Um zu berücksichtigen, dass programmatisch erzeugte Ergebnisse korrekte Werte häufig in abweichender Zeilenreihenfolge liefern, werden Tabellen ohne eindeutigen Schlüssel nach allen Soll-Spalten sortiert. Bei eindeutigem Schlüssel er-

folgt die Zuordnung über den Schlüssel und einen Vorkommensindex. Die Genauigkeit wird als Anteil der zellweise übereinstimmenden Inhaltswerte bestimmt, die Vollständigkeit aus dem Vorhandensein der erwarteten Spalten und der korrekten Datensatzanzahl abgeleitet und die Konsistenz über die Übereinstimmung der Datentypen mit dem Zielschema erfasst [16].

Entsprechend der in Abschnitt 4.5.1 getroffenen Unterscheidung sind beide Ausprägungen der Genauigkeit implementiert: Die strenge Variante bricht den Abgleich ab, sobald Spaltenmenge oder Datensatzanzahl abweichen. Die abgeglichene Variante, die der Auswertung zugrunde liegt, verlangt lediglich ein gültiges Zielschema, ordnet die Tabellen über ihre Schlüssel beziehungsweise die sortierte Reihenfolge einander zu und verbucht überzählige oder fehlende Datensätze über den Nenner $\max(N_{\text{Soll}}, N_{\text{Ist}})$ als Abweichung. Numerische Spalten werden mit $\text{rtol} = 10^{-5}$ und $\text{atol} = 0,01$ verglichen, Textspalten exakt. Fehlt die strukturelle Grundlage vollständig, weil eine erwartete Spalte gar nicht vorhanden ist, wird das Ergebnis als Schema-Bruch gekennzeichnet und getrennt von einem Programmabbruch ausgewiesen. Die Effizienzkennzahlen aus Zeit, Token-Verbrauch und Kosten stammen unmittelbar aus der standardisierten Modellantwort und werden ergänzend zur inhaltlichen Bewertung erfasst [26, 32].

Jeder einzelne Durchlauf wird als eigenständiger, unveränderlicher Ergebnisdatensatz abgelegt. Er umfasst die vollständige Konfiguration des Laufs (das Modell mit seiner exakten Versionskennung, die Prompting-Strategie, die Aufgabe mit Kategorie und Schwierigkeitsgrad, den Zufallskern, die Nummer der Wiederholung und die Temperatur), die erhobenen Qualitäts- und Effizienzkennzahlen sowie das erzeugte Artefakt, im Modus der Code-Generierung den vollständigen Quelltext, im Direkt-Modus die zurückgegebene Ergebnistabelle im typerhaltenden Parquet-Format. Für die verkettete Ausführung kommt je Schritt hinzu, welche Eingaben er erhalten hat, wie viele Versuche bis zu einer verwertbaren Ausgabe nötig waren und wie lange Modellaufruf und Ausführung gedauert haben. Diese lückenlose Protokollierung erlaubt es, jede spätere Auswertung ohne erneute Modellaufrufe aus den gespeicherten Datensätzen nachzuvollziehen, und bildet zugleich die Grundlage für die Erhebung der Reproduzierbarkeit, die sich aus der Streuung der Kennzahlen über die wiederholten Ausführungen einer Konfiguration ergibt.

Eine gesonderte Auswertungsschicht führt die einzelnen Datensätze zusammen und klassifiziert die Ergebnisse nach ihrem Zustand, um technische Fehlschläge von inhaltlichen Abweichungen zu trennen. Da jeder Lauf mit einem Zeitstempel im Dateinamen abgelegt wird und ein wiederholter Aufruf derselben Konfiguration, etwa nach einem Abbruch der Gesamtausführung, somit einen zweiten Datensatz erzeugt, statt den ersten zu überschreiben, wird je Konfiguration einschließlich ihrer Wiederholungsnummer der jeweils jüngste Datensatz herangezogen. Es handelt sich dabei ausdrücklich nicht um eine Auswahl nach Ergebnisqualität: Unterschieden wird allein nach dem Zeitpunkt der Ausführung, sodass eine unvollständig abgebrochene Gesamtausführung keine veralteten Werte in die Auswertung trägt. Auf dieser Grundlage entstehen die in Kapitel 8 dargestellten Vergleiche und Abweichungsanalysen. Damit ist die technische Umsetzung abgeschlossen. Durchführung und Messdaten behandelt Kapitel 7.

7. Durchführung der Experimente

Nachdem Kapitel 6 die technische Umsetzung des Untersuchungssystems beschrieben hat, dokumentiert dieses Kapitel die eigentliche Durchführung der Experimente. Es legt offen, in welchem Ablauf die einzelnen Läufe erfolgten (Abschnitt 7.1) und unter welchen Bedingungen die Durchführung stattfand, um die Nachvollziehbarkeit und Reproduzierbarkeit der Ergebnisse sicherzustellen (Abschnitt 7.2). Damit schafft es die Grundlage für die anschließende Auswertung in Kapitel 8.

7.1. Versuchsablauf

Ausgangspunkt jeder Durchführung ist der in Kapitel 5 beschriebene synthetische Datensatz sowie die zu jeder Aufgabe vorab bestimmte Soll-Lösung. Auf dieser gemeinsamen Grundlage werden die Experimente vollständig automatisiert über den in Abschnitt 6.1 dargestellten Experiment-Runner ausgeführt, sodass jede Konfiguration unter identischen Bedingungen und ohne manuelle Eingriffe durchläuft. Der Ablauf durchmustert die vollständige Faktormatrix systematisch, indem er für jede Kombination aus Modell, Prompting-Strategie und Aufgabe nacheinander die vorgesehenen Wiederholungen ausführt und das Ergebnis unmittelbar festhält.

Vorauszuschicken ist eine Entscheidung, die den Umfang der ausgewerteten Datenbasis bestimmt. Die Erprobung des Untersuchungsdesigns erfolgte auf einem ersten Datensatz: An ihm wurden die Prompts formuliert, der Aufgabenzuschnitt festgelegt und die Auswertungslogik entwickelt. Zwei der in Kapitel 8 berichteten Prompt-Bestandteile, die Angabe der Eingabe-Datentypen und die Anweisung zum Umgang mit akademischen Graden, sind unmittelbare Reaktionen auf dort beobachtete Fehler. Die an diesem Datensatz gemessenen Werte sind damit nicht unbefangen: Sie beschreiben die Leistung auf Daten, gegen die die Konfiguration justiert wurde, und überschätzen die auf unbekannten Daten zu erwartende Leistung in unbekanntem Umfang. Diese Gefahr ist in der methodischen Literatur zur Bewertung von LLMs als zentrale Fehlerquelle beschrieben.

Nach Abschluss sämtlicher Anpassungen wurde deshalb ein zweiter, bis dahin nicht verwendeter Datensatz mit dem Startwert 2 erzeugt und ohne jede weitere Änderung an Prompts, Aufgabenstellungen oder Auswertungscode durchlaufen. Er folgt derselben Erzeugungsvorschrift, enthält jedoch andere Namen, andere Adressen und eine andere Zuordnung der eingebauten Abweichungen. Für die verkettete Ausführung im Direkt-Modus kommt zusätzlich seine halbierte Fassung mit dem Startwert 102 zum Einsatz, da die vollständige Fassung das Ausgabefenster der Modelle überschreitet. Sämtliche in Kapitel 8 berichteten Ergebnisse stammen ausschließlich aus diesen beiden Fassungen. Der erste Datensatz diente allein der Entwicklung und wird nicht ausgewertet.

Entsprechend den beiden in Abschnitt 4.1 eingeführten Ausführungsmodi und den dort festgelegten Untersuchungsebenen gliedert sich die Durchführung in mehrere Teilläufe. Im primären Modus der Code-Generierung wird für jede Konfiguration ein Verarbeitungsskript erzeugt, ausgeführt und bewertet. Aus vier Modellen, drei Prompting-

Strategien, neun Aufgaben und je zwei Wiederholungen ergeben sich 216 Einzelläufe der Faktormatrix. Im ergänzenden Direkt-Modus löst jedes der drei kommerziellen Modelle dieselben neun Aufgaben einmalig unmittelbar auf den eingebetteten Daten, woraus sich 27 weitere Läufe ergeben. Zusätzlich löst die regelbasierte Vergleichspipeline dieselben neun Aufgaben in je einem deterministischen Durchlauf. Jeder einzelne Lauf wird direkt nach seiner Ausführung als eigenständiger Ergebnisdatensatz gespeichert, sodass der Gesamtablauf jederzeit unterbrochen und fortgesetzt werden kann, ohne dass bereits erhobene Ergebnisse verloren gehen oder überschrieben werden. Transiente Fehlaufäufe werden dabei, wie in Abschnitt 6.4 beschrieben, automatisch wiederholt. Inhaltlich fehlerhafte, aber technisch vollständige Ergebnisse bleiben als gültige Messwerte erhalten.

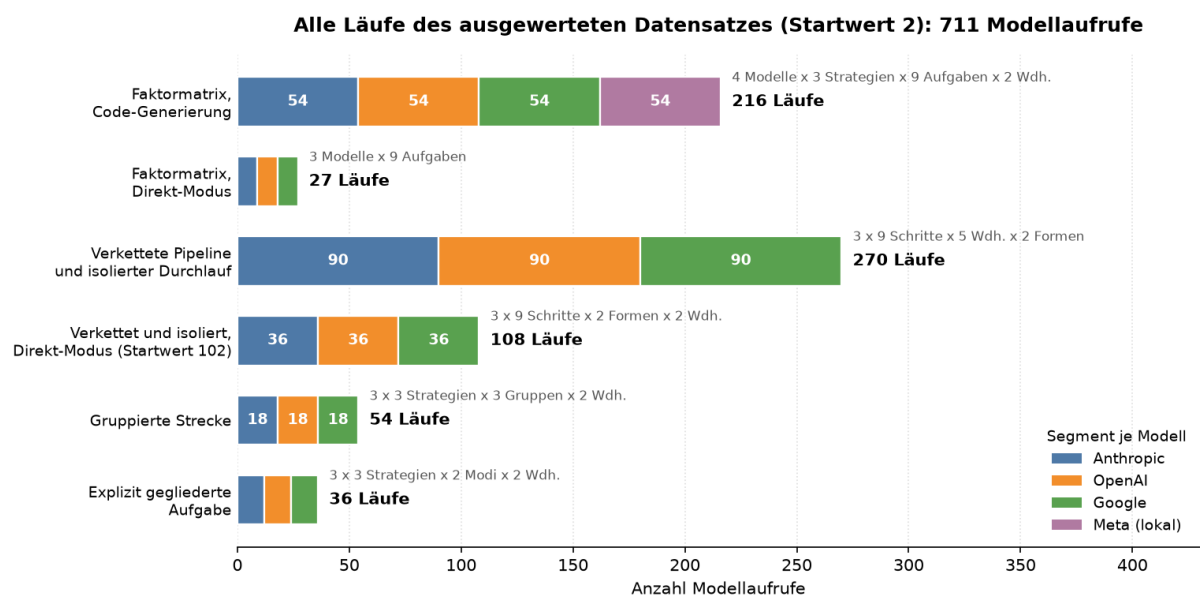
Von den folgenden Teilläufen ausgenommen ist das lokal betriebene Modell. Diese Einschränkung war im Untersuchungsdesign aus Kapitel 4 nicht vorgesehen: Dort ist das lokale Modell als gleichberechtigter vierter Faktor geführt, und die Durchführung begann mit der Absicht, es auf allen Untersuchungsebenen mitzuführen. Erst der Verlauf der Faktormatrix machte sichtbar, dass diese Absicht nicht tragfähig war. Von seinen 54 Läufen brachen 25 mit einem Programmfehler ab, das Modell erzeugte also in nahezu der Hälfte der Fälle schon keinen lauffähigen Code (Abschnitt 8.2). Ursächlich dafür ist die in Abschnitt 7.2 beschriebene Begrenzung des verfügbaren Grafikspeichers, die Modellgröße und Kontextfenster festlegt und damit unmittelbar bestimmt, ob ein Schritt überhaupt eine strukturell verwertbare Ausgabe liefert.

In der Faktormatrix, in der jede Aufgabe unabhängig gestellt ist, bleibt ein solcher Ausfall folgenlos und wird als gültiger Messwert ausgewiesen. In einer Kette dagegen, in der jeder Schritt seinen Nachfolger mit Daten versorgt, würde eine Ausfallquote dieser Größenordnung die Ketten regelmäßig vorzeitig abbrechen und die Mittelwerte in einer Weise verschieben, die sich nicht mehr der jeweils untersuchten Einflussgröße zurechnen ließe: Gemessen würde dann die Hardwareausstattung des ausführenden Rechners und nicht der Aufgabenzuschnitt oder der Ausführungsmodus. Die verkettete Ausführung, die Ebenen zum Aufgabenzuschnitt und der Modusvergleich beschränken sich deshalb auf die drei kommerziell betriebenen Modelle. Das lokale Modell wird ausschließlich in der Faktormatrix geführt, dort aber vollständig ausgewiesen. Da es sich um eine während der Durchführung getroffene und nicht um eine vorab geplante Abgrenzung handelt, ist sie hier offengelegt und in Abschnitt 10.2 als Einschränkung der Reichweite aufgegriffen.

Auf die Faktormatrix folgt die verkettete Ausführung, die dieselben Aufgaben als durchgängige Strecke aus neun aufeinander aufbauenden Schritten durchläuft. Sie wird je Modell fünfmal wiederholt und nach jedem Schritt unmittelbar gegen die zugehörige Soll-Zwischentabelle geprüft, sodass sich sowohl der Qualitätsverlauf über die Kette als auch dessen Streuung über die Wiederholungen bestimmen lässt. Da ein Schritt seine Nachfolger mit Daten versorgt, muss er strukturell verwertbar sein: Bricht der erzeugte Code ab oder verletzt seine Ausgabe das erwartete Schema, wird der Versuch bis zu dreimal wiederholt und die Kette andernfalls an dieser Stelle abgebrochen, wobei die abhängigen Schritte als blockiert protokolliert werden. Diese Wiederholung dient allein der Herstellung einer ausführbaren Kette und ist nicht mit der zur Reproduzierbarkeit erhobenen Mehrfachausführung zu verwechseln.

Ergänzend wird die Kette in einem zweiten, sonst identischen Durchlauf so ausgeführt, dass jeder Schritt die Soll-Ausgabe seines Vorgängers erhält. Beide Durchläufe zusammen erlauben in Kapitel 8 die Trennung von eigener Schrittleistung und geerbtem Fehler. Beide Formen werden auch für die regelbasierte Pipeline ausgeführt, sodass sich die Fehlerfortpflanzung zwischen KI-gestützter und deterministischer Verarbeitung vergleichen lässt. Im Direkt-Modus wird dieselbe Strecke auf der halbierten Datensatzfassung verkettet und isoliert je zweimal durchlaufen.

Zwei weitere Teilläufe untersuchen den Zuschnitt der Aufgabenstellung als eigenständige Einflussgröße. Im ersten werden die neun Pipeline-Schritte zu drei Gruppen von je drei Schritten zusammengefasst, die jeweils die Soll-Ausgabe ihrer Vorgängergruppe erhalten. Ausgeführt werden die drei kommerziellen Modelle, drei Strategien, drei Gruppen und zwei Wiederholungen, also 54 geplante Läufe. Im zweiten wird die zusammengesetzte Aufgabe aus Verknüpfung, Bereinigung und Aggregation erneut in einem einzigen Arbeitsschritt gestellt, diesmal jedoch mit sechs ausdrücklich benannten Teilschritten und einer Soll-Lösung, die diesen Schritten wörtlich folgt und aus den Eingabedaten allein erreichbar ist. Sie wird für die drei kommerziellen Modelle in beiden Ausführungsmodi, unter allen drei Strategien und in je zwei Wiederholungen gestellt, was 36 weitere geplante Läufe ergibt. Abbildung 7.1 stellt die geplanten Versuchseinheiten der Untersuchungsebenen zusammen. Die Zahl der geplanten Einheiten ist von den tatsächlich ausgeführten Modellaufrufen zu unterscheiden, da technische Wiederholungen innerhalb eines Laufs zusätzliche API-Aufrufe erzeugen. Diese Aufrufe, ihre Kosten und die Zahl der Versuche bis zu einer verwertbaren Ausgabe werden in den Ergebnisdatensätzen getrennt protokolliert. Die regelbasierte Vergleichspipeline wurde zusätzlich in 27 deterministischen Durchläufen ausgeführt.



Das lokal betriebene Modell wird ausschließlich in der Faktormatrix geführt. Hinzu kommen 27 deterministische Durchläufe der regelbasierten Vergleichspipeline.

Abbildung 7.1.: Alle Läufe des ausgewerteten Datensatzes mit dem Startwert 2, aufgeschlüsselt nach Untersuchungsebene und Modell. Die Laufzahl je Ebene ist das Produkt der an ihr genannten Faktoren. Sichtbar wird zugleich, dass das lokal betriebene Modell ausschließlich in der Faktormatrix geführt wird.

7.2. Reproduzierbarkeit und Versuchsbedingungen

Die Reproduzierbarkeit wurde bereits im Untersuchungsdesign als querschnittliche Dimension verankert (Abschnitt 4.5) und schlägt sich unmittelbar in den Versuchsbedingungen nieder. Um vergleichbare Bedingungen zu schaffen, werden alle steuerbaren Einflussgrößen über sämtliche Läufe hinweg konstant gehalten: Der Datensatz wird über einen festen Zufallskern reproduzierbar erzeugt und unverändert verwendet, und die Temperatur wird durchgehend auf einen niedrigen Wert festgelegt, um die nicht durch das Modell selbst bedingte Variabilität zu minimieren [5, 29]. Da sich das Feld der Modelle mit hoher Geschwindigkeit weiterentwickelt und die genaue Modellfassung bei den kommerziell betriebenen Anbietern nicht selbst kontrolliert werden kann, wird die exakte Versionskennung jedes Modells bei jedem Aufruf protokolliert. Im Rahmen dieser Arbeit kamen je ein aktueller Vertreter von Anthropic, OpenAI und Google sowie ein lokal ausgeführtes Modell der Llama-Reihe zum Einsatz [5, 26].

Um die probabilistische Variabilität der Modelle nicht nur zu kontrollieren, sondern selbst messbar zu machen, wird sie auf zwei Ebenen erhoben. In der Faktormatrix wird jede Konfiguration zweifach ausgeführt, was anzeigt, ob ein beobachteter Unterschied zwischen Modellen oder Strategien überhaupt Bestand hat. Die vollständige Pipeline wird darüber hinaus in beiden Formen je fünfmal durchlaufen, sodass sich die Streuung auf Ebene einer ganzen Verarbeitungsstrecke beziffern lässt, jener Ebene, auf der sie sich praktisch auswirkt.

Der ergänzende Direkt-Modus wird demgegenüber mit weniger Wiederholungen ausgeführt, um den durch die eingebetteten Daten deutlich erhöhten Token-Verbrauch zu begrenzen: als Einzelaufgabe einfach, als Kette zweifach. Die dort erhobenen Werte sind entsprechend zurückhaltender zu lesen, tragen aber, wie Kapitel 8 zeigt, Unterschiede von mehreren Größenordnungen und damit weit jenseits der beobachteten Streuung. Zu berücksichtigen ist ferner, dass einige aktuelle Modelle bestimmte Sampling-Parameter nicht mehr akzeptieren oder standardmäßig einen internen Denkprozess ausführen. Solche anbieterspezifischen Abweichungen werden, wie in Abschnitt 6.4 beschrieben, abgefangen und im Ergebnisdatensatz vermerkt.

Sämtliche Läufe wurden auf demselben Rechner unter Windows 11 mit Python 3.12 ausgeführt. Für die Einordnung der Ergebnisse ist davon eine einzige Angabe erheblich: Die verwendete Grafikeinheit verfügt über 8 Gigabyte Video Random Access Memory (VRAM). Für die kommerziellen Modelle ist das ohne Belang, da dort lediglich Netzwerkaufrufe abgesetzt und die erzeugten Skripte lokal ausgeführt werden. Für das lokal betriebene Modell ist es die bestimmende Randbedingung. Der verfügbare Videospeicher begrenzt sowohl die Modellgröße, eingesetzt wurde die Acht-Milliarden-Parameter-Variante, als auch das Kontextfenster, das auf 8.192 Token festgelegt werden musste, da größere Zwischenspeicher die Verarbeitung auf die deutlich langsamere CPU zurückfallen lassen. Diese Begrenzung ist bei der Einordnung der Ergebnisse in Abschnitt 8.2 zu berücksichtigen.

Eine bitgenaue Reproduktion bleibt dennoch nicht garantierbar: Die Modelle arbeiten probabilistisch, ihre serverseitige Fassung ist nicht kontrollierbar, und auch bei fester Temperatur wirkt sich die Reihenfolge der Gleitkommaoperationen auf das Ergebnis

aus. Die protokollierten Versionskennungen und die konstanten Bedingungen sichern jedoch eine weitgehende Nachvollziehbarkeit.

8. Ergebnisse und Auswertung

Die in Kapitel 7 beschriebene Durchführung hat eine geschlossene Datenbasis erzeugt, deren Auswertung Gegenstand dieses Kapitels ist. Es folgt der Struktur des Untersuchungsdesigns aus Kapitel 4: zunächst die Ergebnisse je ETL-Aufgabe (Abschnitt 8.1), dann der Vergleich der Modelle (Abschnitt 8.2) und der Prompting-Strategien (Abschnitt 8.3), darauf die verkettete Pipeline mit der Trennung von eigener Schrittleistung und Fehlerfortpflanzung (Abschnitt 8.4) und die querschnittliche Reproduzierbarkeit (Abschnitt 8.5). Die Abschnitte 8.6 und 8.7 behandeln Ausführungsmodi und Aufgabenzuschnitt als eigenständige Gestaltungsgrößen. Abschnitt 8.8 beziffert den Aufwand, Abschnitt 8.9 wertet ihn unter Nachhaltigkeitsgesichtspunkten aus, und Abschnitt 8.10 stellt den Vergleich mit der regelbasierten Pipeline an, bevor die Abschnitte 8.11 bis 8.13 die Befunde bündeln, in den Stand der Forschung einordnen und an den Hypothesen spiegeln.

Für die Lektüre gelten dabei drei Festlegungen. Erstens stammen sämtliche berichteten Werte aus dem Datensatz mit dem Startwert 2. Allein die verkettete und die isolierte Ausführung im Direkt-Modus beruhen auf dessen halbiertem Fassung mit dem Startwert 102. Die Beschränkung ist in Abschnitt 7.1 begründet. Zweitens meint Genauigkeit stets die in Abschnitt 4.5.1 definierte abgeglichenen Ausprägung. Läufe ohne auswertbare Ausgabe gehen mit dem Wert null in alle Mittelwerte ein. Drittens weist die Erfolgsquote den strengerem Anteil derjenigen Läufe aus, die Zielschema *und* erwartete Datensatzanzahl einhalten. Die mittlere Genauigkeit kann sie deshalb übersteigen. Der Abstand beider Größen wird nur deskriptiv berichtet und lässt keine direkte Bestimmung der Zahl strukturell abweichender Läufe zu.

8.1. Ergebnisse je ETL-Aufgabe

Über alle 216 Läufe der Faktormatrix hinweg erreichen die vier Modelle eine mittlere Genauigkeit von 67,5 Prozent bei einer Erfolgsquote von 69,4 Prozent. Diese Gesamtwerte sind rein deskriptiv, weil sie die später als methodisch ungeeignet erkannte Join-/Aggregationsaufgabe enthalten. Sie werden daher nicht als belastbarer Modell- oder Komplexitätsvergleich interpretiert. Von den 216 Läufen führten 150 zu einer verwertbaren Ausgabe, 48 brachen mit einem Programmfehler ab und 18 lieferten eine lauffähige, aber strukturell abweichende Ausgabe. Der gesamte Aufwand belief sich auf 2,80 US-Dollar bei rund 103.000 Eingabe- und 277.000 Ausgabe-Token sowie einer mittleren Antwortzeit von 12,0 Sekunden je Aufruf. Diese Mittelwerte verdecken jedoch eine erhebliche Spreizung, die erst in der Aufschlüsselung nach Aufgaben sichtbar wird. Für Schlussfolgerungen zu Aufgaben- und Modellunterschieden werden die acht erreichbaren Einzelaufgaben und die separat ausgewertete explizite Variante herangezogen.

Abbildung 8.1 zeigt diese Spreizung im Überblick. Aufgetragen ist je Aufgabe das bes-

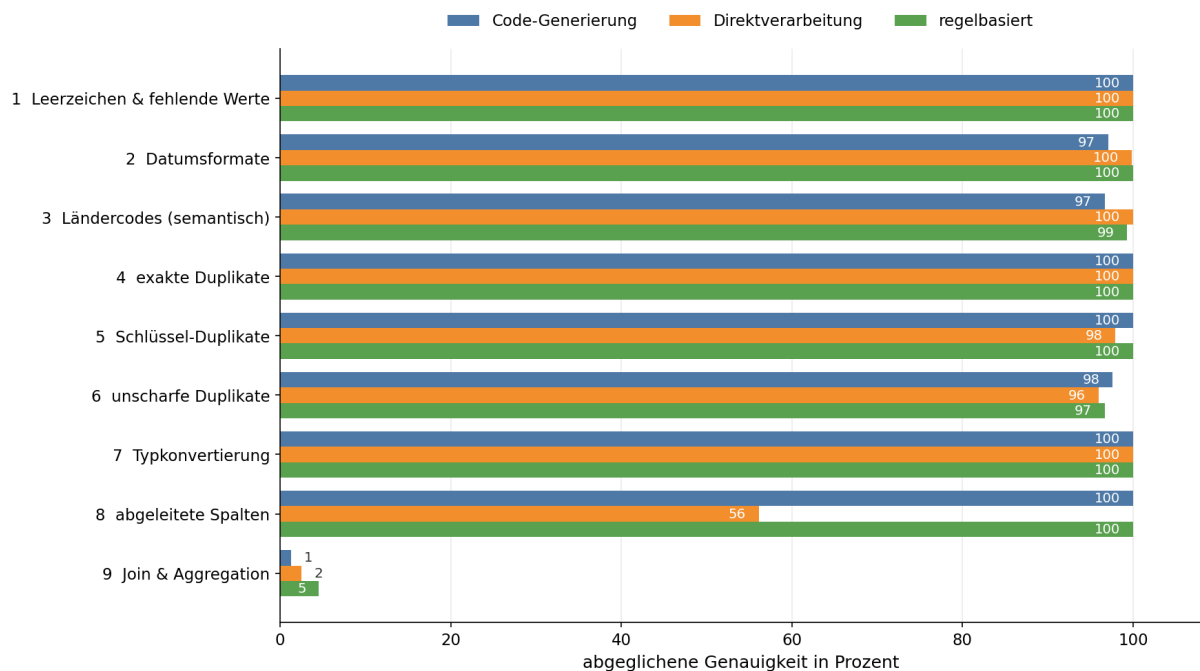


Abbildung 8.1.: Bestes je Aufgabe erreichtes Ergebnis der drei Ansätze. Für die Code-Generierung ist der beste Wert über Modelle, Strategien und Wiederholungen ausgewiesen, für die Direktverarbeitung der beste Wert über die Modelle. Deren Rückstand bei den abgeleiteten Spalten beruht auf einer am Ausgabelimit abgeschnittenen Antwort und nicht auf inhaltlichen Fehlern. Abschnitt 8.6 weist dies nach.

te erreichte Ergebnis der drei untersuchten Ansätze, sodass der Vergleich das jeweils Erreichbare und nicht den Durchschnitt einer beliebig herausgegriffenen Konfiguration abbildet. Drei Muster treten unmittelbar hervor: Die formal beschreibbaren Aufgaben werden von allen drei Ansätzen nahezu fehlerfrei gelöst, die beiden anspruchsvollen Entdoppelungsaufgaben trennen die Ansätze deutlich, und die zusammengesetzte Aufgabe aus Verknüpfung, Bereinigung und Aggregation scheitert bei allen Ansätzen, einschließlich der regelbasierten Pipeline.

Tabelle 8.1 beziffert dieses Bild je Aufgabe. Ausgewiesen sind die über alle Modelle, Strategien und Wiederholungen gemittelte Genauigkeit, die zugehörige Erfolgsquote sowie zum Vergleich das Ergebnis der regelbasierten Pipeline. Die Reihenfolge folgt den drei Schwierigkeitsstufen aus Abschnitt 4.3, sodass sich der Einfluss der Aufgabenkomplexität unmittelbar ablesen lässt.

Bemerkenswert ist, dass selbst die einfachsten Aufgaben im Mittel deutlich unter der Marke von 100 Prozent bleiben, obwohl jedes einzelne kommerzielle Modell sie in seinem besten Lauf fehlerfrei löst. Die Spalte „Genau.“ mittelt über alle vier Modelle und alle drei Strategien und ist deshalb nicht als Aussage über die erreichbare Qualität zu lesen, sondern über die Verlässlichkeit einer beliebig herausgegriffenen Konfiguration. Ihr Abstand zur Spalte „Cloud“ geht auf das lokal betriebene Modell zurück. Wie sich die Werte auf die einzelnen Modelle verteilen und warum diese Verteilung je nach Aufgabe eine andere ist, behandelt Abschnitt 8.2.

Tabelle 8.1.: Mittlere Genauigkeit und Erfolgsquote je Aufgabe über alle vier Modelle, drei Strategien und zwei Wiederholungen der Faktormatrix, gegenübergestellt der regelbasierten Pipeline. Die vorletzte Spalte weist zusätzlich das Mittel der drei kommerziellen Modelle aus.

Kategorie	Aufgabe	Genau.	Erfolg	Cloud	Regel
Bereinigung	Einfach: Leerzeichen und fehlende Werte	79,2 %	79,2 %	100,0 %	100,0 %
Bereinigung	Mittel: Datumsformate	85,9 %	91,7 %	96,7 %	100,0 %
Bereinigung	Schwer: ISO-Ländercodes	84,2 %	91,7 %	94,8 %	99,3 %
Transformation	Einfach: Typkonvertierung	85,0 %	91,7 %	95,4 %	100,0 %
Transformation	Mittel: abgeleitete Spalten	83,3 %	83,3 %	100,0 %	100,0 %
Transformation	Schwer: Join und Aggregation	0,1 %	0,0 %	0,1 %	4,5 %
Duplikate	Einfach: exakte Duplikate	100,0 %	100,0 %	100,0 %	100,0 %
Duplikate	Mittel: Schlüssel-Duplikate	53,0 %	45,8 %	37,8 %	100,0 %
Duplikate	Schwer: unscharfe Duplikate	37,2 %	41,7 %	49,5 %	96,7 %

8.1.1. Datenbereinigung

Die Datenbereinigung ist die Kategorie mit der höchsten und zugleich stabilsten Ergebnisqualität. Das Entfernen überflüssiger Leerzeichen und die Kennzeichnung fehlender Werte lösen die drei kommerziellen Modelle in jedem Lauf vollständig korrekt, das lokale in seinem besten. Die Vereinheitlichung der vier Datumsformate erreicht bei den kommerziellen Modellen 96,7 Prozent im Mittel. Beide Aufgaben sind, wie in Abschnitt 5.3 beschrieben, durch eine endliche Menge syntaktischer Regeln vollständig beschreibbar, und genau in diesem Bereich verhält sich der erzeugte Code so zuverlässig wie eine manuell geschriebene Lösung [12, 17].

Gerade hier zeigt sich jedoch ein Risiko, das für die Beurteilung der Eignung wesentlich ist: ein Fehler, der weder eine Fehlermeldung noch eine Schemaverletzung erzeugt und ohne bekannte Soll-Lösung im laufenden Betrieb unbemerkt bliebe. Skript 8.1 stellt zwei Lösungen derselben Aufgabe gegenüber, die beide fehlerfrei durchlaufen. Die erste wählt die zu bereinigenden Spalten über deren Datentyp aus, ein über Jahre korrektes Programmiermuster. In der eingesetzten Pandas-Version bleibt diese Auswahl leer, sodass die Bereinigung wirkungslos bleibt. Die zweite benennt die Spalten explizit. Beobachtet wurde dies am Entwicklungsdatensatz (vgl. Abschnitt 7.1). Die Darstellung ist qualitativer Natur und trägt keine der berichteten Kennzahlen.

```

1 # still wirkungslos: die Spaltenauswahl liefert keine Treffer
2 for col in df.select_dtypes(include=['object']).columns:
3     df[col] = df[col].str.strip()
4
5 # wirksam: die betroffenen Spalten werden explizit benannt
6 for col in ["full_name", "email", "country", "registered_at"]:
7     df[col] = df[col].astype(str).str.strip()

```

Skript 8.1: Zwei lauffähige Lösungen derselben Bereinigungsaufgabe: Die obere Variante bleibt in der eingesetzten Pandas-Version wirkungslos, weil Textspalten nicht mehr als Objektspalten geführt werden. Die untere benennt die Spalten explizit.

Das Versagen ist systematisch und nicht zufällig: Es tritt hier bei der Verwendung eines Programmiermusters auf, das in der eingesetzten Laufzeitumgebung nicht das beabsichtigte Verhalten zeigt. Behoben wurde es nicht durch einen allgemeinen Hinweis auf Versionsunterschiede, sondern erst dadurch, dass dem Modell die Datentypen der Eingabespalten explizit mitgeteilt wurden.

Die semantische Vereinheitlichung der Länderangaben erreicht unter den kommerziellen Modellen 94,8 Prozent im Mittel und 96,7 Prozent im besten Lauf, bleibt damit aber hinter der regelbasierten Pipeline mit 99,3 Prozent zurück. Der Grund für diese Lücke lässt sich anhand der erzeugten Skripte exakt benennen: Alle drei Modelle lösen die Aufgabe, indem sie im Code eine Zuordnungstabelle aufbauen und die Rohwerte darüber nachschlagen. Bei den beobachteten Lösungen begrenzt die Abdeckung dieser Zuordnungstabelle die Ergebnisqualität. Das ist eine Eigenschaft der erzeugten Skripte, keine grundsätzliche Einschränkung der Code-Generierung: Andere Programme könnten beispielsweise zusätzliche Wissensquellen oder geeignetere Abgleichverfahren nutzen. Der Direkt-Modus, in dem das Modell jeden Wert einzeln beurteilt, statt eine verallgemeinernde Vorschrift zu formulieren, erreicht bei derselben Aufgabe 97,1 bis 100 Prozent und übertrifft damit erstmals die regelbasierte Lösung. Abschnitt 8.6 greift diesen Modusunterschied systematisch auf.

8.1.2. Datentransformation

Typkonvertierung und abgeleitete Spalten fügen sich in das Bild der formal beschreibbaren Aufgaben ein: Beide werden von den kommerziellen Modellen im besten Lauf vollständig korrekt gelöst, die abgeleiteten Spalten sogar in jeder Strategie und jeder Wiederholung.

Die zusammengesetzte Aufgabe aus Verknüpfung, Bereinigung und Aggregation ist dagegen die einzige der neun Aufgaben, die von keinem Modell und in keiner Strategie gelöst wurde. Auch die regelbasierte Pipeline erreicht nur 4,5 Prozent. Ihre Analyse zeigt zwei wiederkehrende Fehlerbilder. Im ersten aggregiert das Ergebnis lediglich 478 der 2.000 Bestellungen, weil die Verknüpfung Bestellungen ohne passenden Kundendatensatz verwirft. Im zweiten werden zwar alle 2.000 Bestellungen erfasst, die Gruppierung erfolgt jedoch über 41 bis 42 verschiedene Rohschreibweisen der Länderangabe, deren Zahl je nach Lauf schwankt, weil unterschiedliche Schreibvarianten zusammenfallen, statt über die zehn ISO-Codes, sodass etwa DE, Deutschland, Germany, GER und Deutshc land als fünf getrennte Länder erscheinen. Da die bereinigte Länderangabe zugleich Gruppierungsschlüssel ist, wirkt sich jede unvollständige Bereinigung auf sämtliche aggregierten Werte aus, sodass kein einziger Kennzahlenwert korrekt bleibt.

Dieser Nullbefund verlangt jedoch eine Einordnung, die ihn wesentlich relativiert, und diese Einordnung ist selbst ein Ergebnis. Die Aufgabenstellung verlangt zwar, die Daten „so weit wie nötig“ zu bereinigen, benennt die erforderliche Vereinheitlichung der Länderangaben aber an keiner Stelle ausdrücklich. Sie ist lediglich implizit über das Zielschema angedeutet. Hinzu tritt ein Umstand der Soll-Lösung: In den Rohdaten fehlt bei 38 der 540 Kunden die Landangabe vollständig, und auf diese Kunden entfallen 144 der 2.000 Bestellungen. Die Soll-Lösung ordnet sie dennoch den zehn realen

Ländern zu, weil ihr die Erzeugungsvorschrift des Datensatzes zugrunde liegt. Aus den Eingabedaten allein ist dieses Ergebnis folglich gar nicht erreichbar, und zwar für kein Verfahren, was auch erklärt, warum die regelbasierte Pipeline hier ebenfalls scheitert.

Die Kennzahl misst an dieser Stelle somit weder die Fähigkeit zur Aggregation noch die zur Komposition mehrerer Schritte, sondern in erster Linie die Passung zwischen Aufgabentext und Soll-Lösung. Ein Nullwert, den alle Verfahren gleichermaßen erreichen, trennt nichts mehr und ist als Leistungsaussage nicht verwertbar. Die ursprüngliche Aufgabe wird deshalb als methodisch ungeeigneter Benchmarkfall separat behandelt. Abschnitt 8.7.2 löst die Vermengung auf, indem er dieselbe Verarbeitung mit ausdrücklich benannten Teilschritten und einer aus den Eingabedaten tatsächlich erreichbaren Soll-Lösung erneut stellt. Die Modelle erreichen dort zwischen 54,5 und 75,8 Prozent. Dieser Vergleich ist eine explorative Sensitivitätsanalyse, kein isolierter Nachweis eines Effekts der Zerlegung, weil sich Aufgabenformulierung und erreichbare Referenz gleichzeitig ändern [7].

8.1.3. Duplikaterkennung

Die Duplikaterkennung weist die stärkste Spreizung aller drei Kategorien auf. Exakte Duplikate werden von allen vier Modellen und in jeder Strategie fehlerfrei entfernt. Die Aufgabe ist mit einem einzigen Funktionsaufruf lösbar und trennt die Ansätze nicht. Die schlüsselbasierte Entdoppelung anhand der E-Mail-Adresse fällt dagegen auf 53,0 Prozent im Mittel, die unscharfen Duplikate auf 37,2 Prozent.

Bei näherer Betrachtung sind diese niedrigen Mittelwerte weniger auf ein Unvermögen der Modelle als auf die Zweiseitigkeit der Aufgabe zurückzuführen. Anders als bei der Bereinigung, bei der jede Zeile für sich betrachtet wird, verlangt die Entdoppelung eine Entscheidung über das Verhältnis von Datensätzen zueinander, bei der sowohl zu wenig als auch zu viel entfernt werden kann. Ein zu enger Ähnlichkeitsschwellenwert übersieht echte Duplikate, ein zu weiter führt unbeteiligte Datensätze zusammen. Am Entwicklungsdatensatz trat dieser zweite Fehlertyp in besonders klarer Form auf (vgl. Abschnitt 7.1): Werden akademische Grade und Anreden nicht als solche erkannt, sondern als Namensbestandteile behandelt, entstehen Vergleichsschlüssel, unter denen mehrere verschiedene Kunden zusammenfallen. Erst die explizite Anweisung, Titel nicht als Teil des Namens zu behandeln und Übereinstimmung sowohl im Namen als auch in der E-Mail-Adresse zu verlangen, löste dieses Problem auf.

Dass die Aufgabe grundsätzlich lösbar ist, belegen die Bestwerte: Bei den unscharfen Duplikaten erreichen Claude 97,6 und Gemini 97,4 Prozent und liegen damit über der regelbasierten Pipeline mit 96,7 Prozent. Bei den Schlüssel-Duplikaten erreichen GPT und das lokale Modell 100 Prozent. Der Abstand zwischen Mittel und Bestwert ist in dieser Kategorie mit gut 60 Prozentpunkten der größte der Faktormatrix. Genau hier, und nur hier, entfaltet auch die Wahl der Prompting-Strategie eine messbare Wirkung, wie Abschnitt 8.3 zeigt.

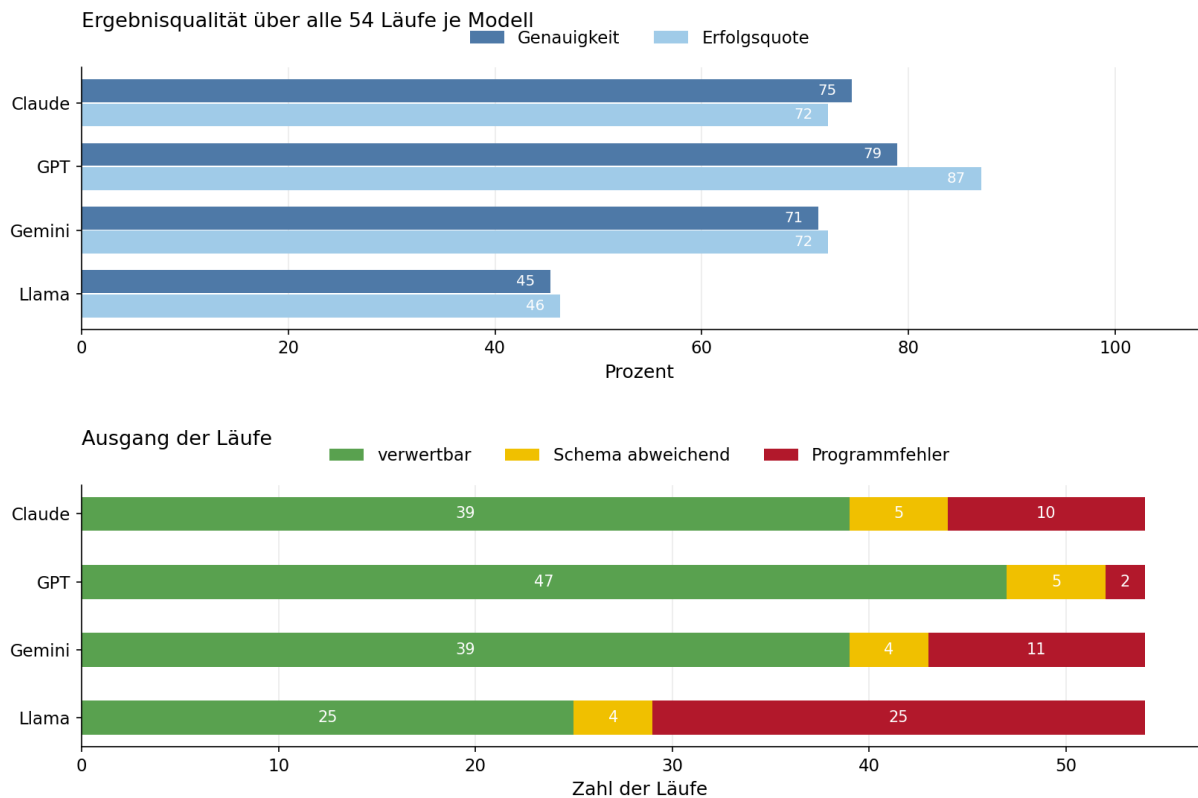


Abbildung 8.2.: Modellvergleich über die je 54 Läufe der Faktormatrix: oben Genauigkeit und Erfolgsquote, unten der Ausgang der Läufe nach verwertbarem Ergebnis, abweichendem Schema und Programmfehler.

8.2. Einfluss der Modellwahl

Abbildung 8.2 fasst den Modellvergleich über die 216 Läufe zusammen. Unter den kommerziell betriebenen Modellen liegen die Ergebnisse eng beieinander: Das Modell von OpenAI erreicht mit einer Genauigkeit von 78,9 Prozent bei einer Erfolgsquote von 87,0 Prozent den höchsten Wert, gefolgt von Anthropic mit 74,5 und 72,2 Prozent sowie Google mit 71,3 und 72,2 Prozent. Die Abstände zwischen diesen drei Modellen sind mit rund acht Prozentpunkten deutlich geringer als die zuvor beobachteten Unterschiede zwischen den Aufgaben, die von 0 bis 100 Prozent reichen. Für die Frage der Eignung ist damit die Wahl der Aufgabe erheblich folgenreicher als die Wahl des Anbieters, ein Befund, der die in der Literatur verbreitete Fokussierung auf Modellranglisten relativiert.

Dass diese Rangfolge nicht über alle Aufgaben trägt, zeigt der Blick auf die Einzelwerte. Bei den Schlüssel-Duplikaten erreicht GPT 100 Prozent, Claude 80,0 und Gemini 0,0 Prozent. Gemini erzeugt hier in beiden Wiederholungen und allen drei Strategien keine verwertbare Ausgabe. Bei den unscharfen Duplikaten kehrt sich das Verhältnis um: Dort liegen Claude mit 97,6 und Gemini mit 97,4 Prozent vor GPT mit 90,0 Prozent. Genau diese aufgabenweise Umkehrung ist die Ursache dafür, dass die kommerziellen Modelle bei den Schlüssel-Duplikaten in Tabelle 8.1 schlechter abschneiden als das Gesamtmittel. Eine über alle Aufgaben gemittelte Modellrangfolge verdeckt somit mehr, als sie zeigt.

Das untere Bild der Abbildung macht einen zweiten Unterschied sichtbar, der sich in der Genauigkeit nur teilweise niederschlägt. GPT brach lediglich 2 seiner 54 Läufe mit einem Programmfehler ab, Claude 10 und Gemini 11. Diese Trennung ist praktisch bedeutsam, weil sie einen qualitativen Unterschied markiert: Ein inhaltlich abweichendes Ergebnis lässt sich prüfen und korrigieren, ein nicht lauffähiges Skript hingegen erzwingt einen erneuten Aufruf und verschiebt das Problem lediglich.

Deutlich abgesetzt ist das lokal betriebene Modell der Llama-Reihe mit einer Genauigkeit von 45,4 Prozent und einer Erfolgsquote von 46,3 Prozent. Von seinen 54 Läufen brachen 25 mit einem Programmfehler ab. Das lokale Modell scheitert damit weit überwiegend nicht an der inhaltlichen Aufgabe, sondern daran, überhaupt lauffähigen Code zu erzeugen. Dass es die Aufgaben durchaus versteht, belegen seine Bestwerte: Die einfache Bereinigung, die exakten Duplikate, die abgeleiteten Spalten und die Schlüssel-Duplikate löst es mit 100 Prozent und liegt bei letzteren damit über zwei der drei kommerziellen Modelle.

Der gemessene Abstand ist keine Aussage über lokal betreibbare Modelle im Allgemeinen, sondern über dasjenige Modell, das sich auf der für diese Arbeit verfügbaren Hardware sinnvoll ausführen ließ. Die Größe eines lokal betriebenen Modells ist unmittelbar durch den verfügbaren Grafikspeicher begrenzt, und mit ihr die Fähigkeit, längere Anweisungen zu verarbeiten und syntaktisch fehlerfreien Code zu erzeugen. Die kommerziellen Modelle unterliegen dieser Beschränkung nicht, da die Ausführung beim Anbieter stattfindet. Ein Vergleich zwischen beiden misst somit nicht allein die Modellqualität, sondern zu einem erheblichen Teil die Ausstattung des ausführenden Rechners [5, 26].

Der lokale Betrieb ist zugleich die einzige der untersuchten Optionen, bei der keine Daten das eigene Netz verlassen. Die häufigen Programmabbrüche beschreiben einen Fehlertyp, erlauben aber keine isolierte Aussage über Aufgabenverständnis oder Hardwareeinfluss. Ob eine andere lokale Modellkonfiguration den Abstand verringert, müsste gesondert untersucht werden [5, 26].

8.3. Vergleich der drei Prompt-Vorlagen

Über alle vier Modelle und Aufgaben hinweg erzielt das Few-Shot-Prompting mit 70,6 Prozent die höchste Genauigkeit, gefolgt vom Chain-of-Thought-Prompting mit 69,0 und dem Zero-Shot-Prompting mit 63,0 Prozent. Beschränkt man die Auswertung auf die drei kommerziellen Modelle, bleibt die Reihenfolge erhalten, bei durchweg höherem Niveau: 77,9 gegenüber 75,1 und 71,8 Prozent. Auch beim lokalen Modell bleibt der Zero-Shot-Prompt mit 36,6 Prozent deutlich hinter den beiden aufwändigeren Strategien zurück, die mit 50,6 (Chain-of-Thought) und 48,9 Prozent (Few-Shot) nahezu gleichauf liegen. Der Befund geht damit nicht auf das Verhalten eines einzelnen Modells zurück, sondern zeigt sich über alle vier hinweg.

Der Aufwand wächst allerdings deutlich stärker als der Nutzen: Das Few-Shot-Prompting benötigt im Mittel 875 Eingabe-Token annähernd das Dreifache des Zero-Shot-Prompts mit 302 Token, um unter den kommerziellen Modellen einen

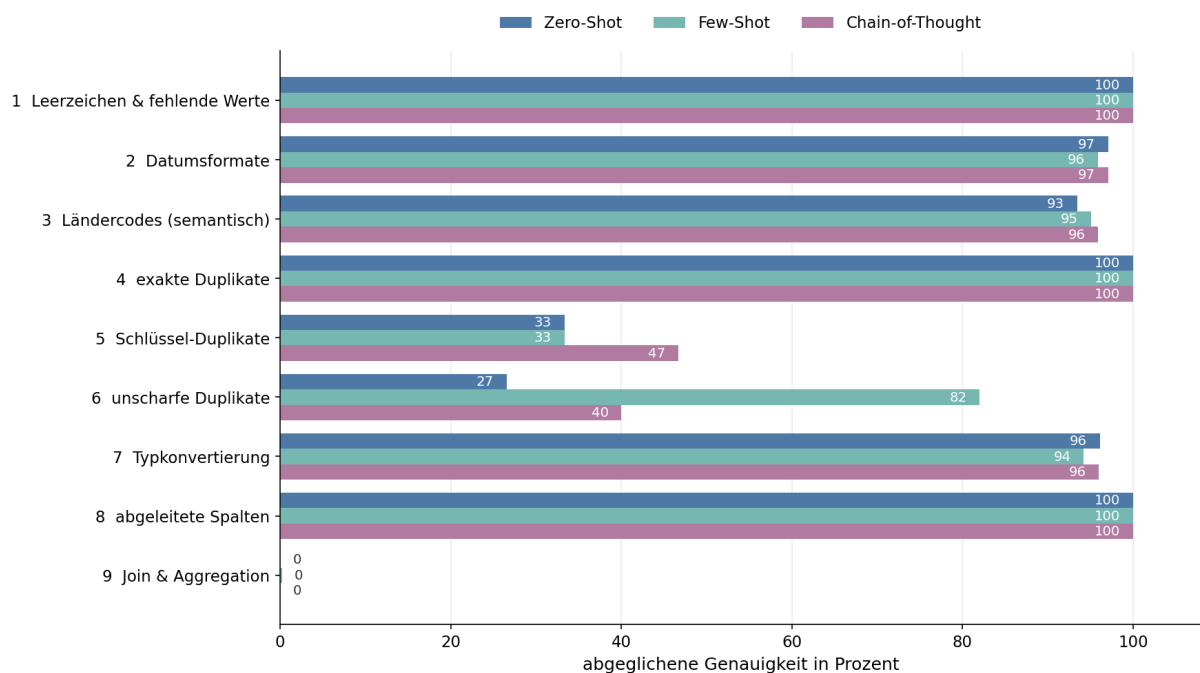


Abbildung 8.3.: Abgeglichene Genauigkeit der drei Prompting-Strategien je Aufgabe, gemittelt über die drei kommerziellen Modelle und beide Wiederholungen.

Vorsprung von 6,1 Prozentpunkten zu erzielen. Für einen Dauerbetrieb, in dem die Aufgabenbeschreibung bei jedem Aufruf erneut übertragen wird, ist diese Rechnung bei den heutigen Preisen dennoch unkritisch, da der Prompt gegenüber der erzeugten Antwort kaum ins Gewicht fällt.

Aussagekräftig ist der Strategievergleich jedoch erst auf Aufgabenebene, und dort zeigt Abbildung 8.3 ein sehr klares Bild: Sieben der neun Aufgaben unterscheiden zwischen den Strategien praktisch nicht. Die drei Bereinigungsaufgaben, die exakten Duplikate, die Typkonvertierung und die abgeleiteten Spalten werden von jeder Strategie nahezu fehlerfrei gelöst, die zusammengesetzte Aufgabe von keiner. Der gesamte messbare Unterschied konzentriert sich auf die beiden anspruchsvollen Entdoppelungsaufgaben, und dort mit klarer Zuordnung: Bei den unscharfen Duplikaten führt das Few-Shot-Prompting mit 82,0 gegenüber 40,0 Prozent für Chain-of-Thought und 26,6 für Zero-Shot, bei den Schlüssel-Duplikaten hingegen das Chain-of-Thought-Prompting mit 46,7 gegenüber jeweils 33,3 Prozent. Beispiele wirken damit dort, wo eine Ähnlichkeitsentscheidung zu treffen ist, für die eine verbale Beschreibung schlecht taugt. Die Aufforderung zur schrittweisen Herleitung wirkt dort, wo eine mehrstufige Auswahlregel umzusetzen ist [25, 28].

Die über alle Aufgaben gemittelte Rangfolge ist damit kein eigenständiger Befund, sondern eine Folge dieser einen Aufgabe: Der Vorsprung des Few-Shot-Promptings von 6,1 Prozentpunkten unter den kommerziellen Modellen entsteht nahezu vollständig aus den 55 Prozentpunkten, die es bei den unscharfen Duplikaten gewinnt. Wer aus der Gesamtrangfolge die Regel ableitet, Beispiele seien generell hilfreich, überträgt eine Beobachtung aus einem Neuntel des Aufgabenraums auf das Ganze. Die praktisch verwertbare Aussage lautet stattdessen: Der Nutzen zusätzlicher Anleitung ist nicht

allgemein, sondern an die Eigenart der Aufgabe gebunden. Wo eine Aufgabe in einem einzigen gedanklichen Schritt lösbar ist, stiftet sie keinen Mehrwert und verleitet zu unnötig komplexem und damit fehleranfälligerem Code. Wo eine Ermessensgrenze zu setzen oder eine mehrstufige Entscheidung zu treffen ist, wirkt sie [29, 32].

Wie stark diese Bindung an die Aufgabe tatsächlich ist, zeigt Abschnitt 8.7.1: Wird derselbe Verarbeitungsinhalt anders zugeschnitten, verschwindet der Strategieunterschied vollständig.

8.4. Ergebnisse der verketteten Pipeline

Die bisher dargestellten Ergebnisse betrachten jede Aufgabe für sich. Die in Abschnitt 4.1 eingeführte Untersuchungsebene der verketteten Ausführung prüft, wie sich dieses Bild verändert, wenn dieselben Aufgaben als durchgängige Strecke ausgeführt werden, in der jeder Schritt auf dem Ergebnis seines Vorgängers aufsetzt. Beide Durchläufe, der verkettete und der isolierte, wurden je Modell fünfmal wiederholt. Sämtliche Werte dieses Abschnitts sind Mittel über diese fünf Wiederholungen.

8.4.1. Qualitätsverlauf über die Kette

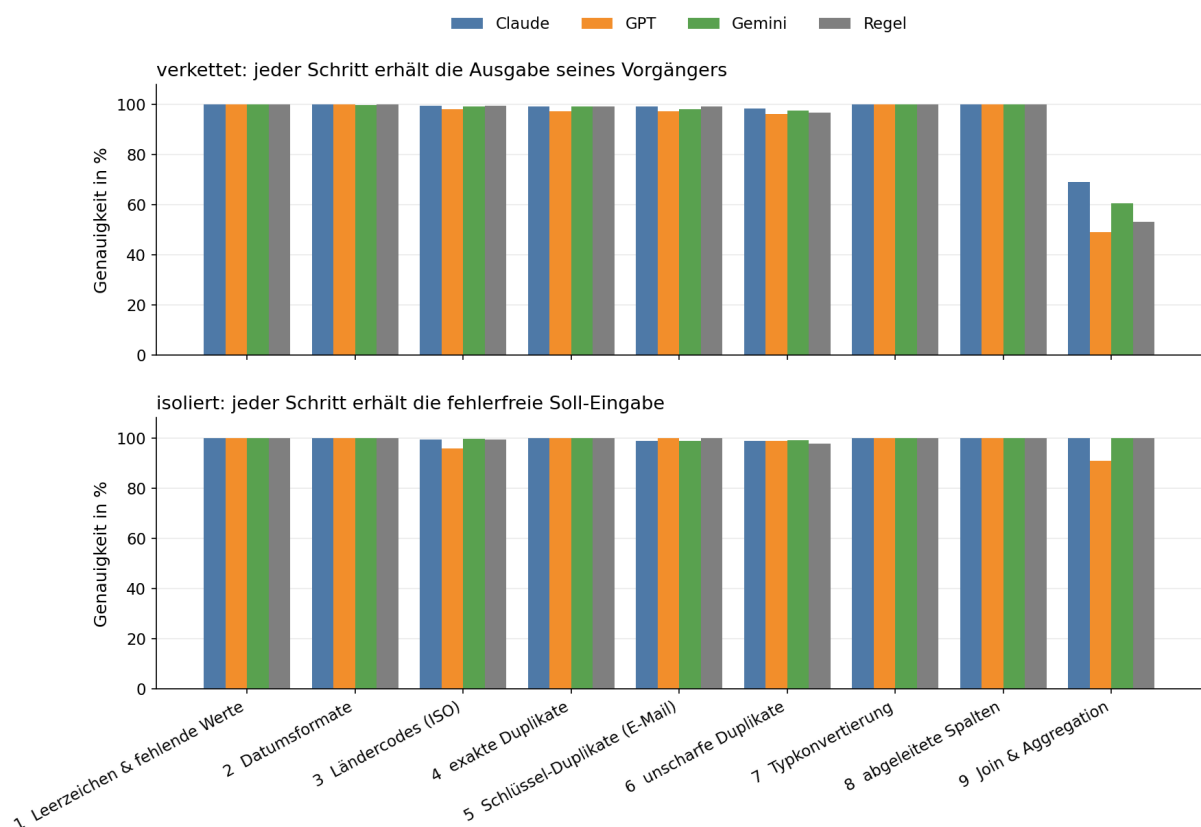


Abbildung 8.4.: Abgeglicheene Genauigkeit je Schritt, gemittelt über fünf Wiederholungen. Oben die verkettete Ausführung, in der jeder Schritt die Ausgabe seines Vorgängers erhält. Unten die isolierte Ausführung mit fehlerfreier Soll-Eingabe je Schritt.

Das obere Bild in Abbildung 8.4 zeigt den Verlauf der neun Schritte für die drei Modelle und die regelbasierte Pipeline. Alle drei Modelle durchlaufen die Kette in jeder Wiederholung vollständig und erreichen in den Schritten 1 bis 8 durchweg Werte zwischen 96,0 und 100 Prozent. Über alle neun Schritte gemittelt erzielt Claude 96,1 Prozent, Gemini 94,9 und GPT 93,1, gegenüber 94,1 Prozent für die regelbasierte Vergleichspipeline. Der Aufwand einer vollständigen Strecke ist dabei gering: Er beträgt je Modell zwischen 0,117 und 0,216 US-Dollar sowie 102 bis 135 Sekunden, gegenüber 2,80 US-Dollar für die vollständige Faktormatrix. Die regelbasierte Pipeline benötigt für dieselbe Strecke 0,18 Sekunden und keine Nutzungskosten.

Der Endschrift bricht dagegen bei allen Teilnehmern ein: Claude erreicht 69,1 Prozent, Gemini 60,6 und GPT 49,1, die regelbasierte Pipeline 53,0 Prozent. Bemerkenswert ist zunächst, dass diese Werte weit über dem Ergebnis derselben Aufgabe als Einzellauf liegen, wo sie bei 0,1 Prozent lag. Die Ursache ist dieselbe wie die in Abschnitt 8.1.2 beschriebene: In der Kette erhält der Endschrift bereits bereinigte Länderangaben und muss die semantische Vereinheitlichung nicht mehr selbst erbringen. Zugleich prüft ihn die Kettenreferenz gegen eine Soll-Lösung, die aus seinen tatsächlichen Eingabedaten erreichbar ist. Der verbleibende Verlust entsteht ausschließlich aus den Daten, die er verarbeitet: Es bleiben Bestellungen ohne zugehörigen Kundendatensatz zurück, weil in der vorgelagerten Entdoppelung jeweils der andere der beiden unscharfen Datensätze behalten wurde. Da die Aggregation über Land und Kategorie gruppiert, macht jede solche Bestellung zwei Gruppen in beiden Kennzahlen falsch, sodass wenige fehlplatzierte Datensätze einen überproportionalen Ausschlag erzeugen.

Der Vergleich beider Betrachtungsweisen zeigt bereits für sich genommen einen deutlichen Unterschied: Für dieselben neun Aufgaben liegt das Ergebnis in der Kette im Mittel weit über dem des unabhängigen Einzellaufs. In der Kette erhält jeder Schritt bereits vorbereitete Daten und eine eng umrissene Teilaufgabe, während er im Einzellauf zusätzlich alle vorgelagerten Mängel mitbehandeln muss. Die Zerlegung einer Strecke in kleine, klar abgegrenzte Schritte verbessert das Ergebnis somit unabhängig vom eingesetzten Modell, ein Befund, den Abschnitt 8.7.1 systematisch prüft und der unmittelbar auf die praktische Gestaltung KI-gestützter ETL-Strecken durchschlägt [1, 3].

8.4.2. Trennung von eigener Leistung und Fehlerfortpflanzung

Der bis hierhin gemessene Verlauf lässt offen, welcher Anteil eines schwachen Schrittergebnisses dem Schritt selbst und welcher den Vorgängerschritten zuzurechnen ist. Der isolierte Durchlauf, in dem jeder Schritt die Soll-Ausgabe seines Vorgängers erhält, beantwortet diese Frage. Das untere Bild in Abbildung 8.4 stellt ihn dem verketteten gegenüber.

Mit fehlerfreier Eingabe erreichen die Modelle über alle neun Schritte 99,7 (Claude), 99,7 (Gemini) und 98,4 Prozent (GPT). Den Endschrift lösen Claude und Gemini in jeder einzelnen Wiederholung vollständig korrekt, GPT erreicht konstant 90,9 Prozent. Der in der Kette gemessene Verlust im Endschrift von 30,9 (Claude), 39,4 (Gemini) und 41,8 Prozentpunkten (GPT) ist damit fast vollständig geerbt und nicht dem Endschrift anzulasten. Als eigene, nicht geerbte Schwäche verbleiben genau die drei semantisch

geprägten Schritte. Die Länder-Vereinheitlichung lösen die Modelle auch mit fehlerfreier Eingabe nur zu 99,3, 99,6 und 95,9 Prozent, und zwar aus dem in Abschnitt 8.1.1 belegten Grund der unvollständig aufgezählten Zuordnungstabelle. Die schlüsselbasierte und die unscharfe Entdoppelung bleiben mit 98,8 bis 99,0 Prozent knapp darunter. Alle übrigen sechs Schritte werden mit fehlerfreier Eingabe von jedem Modell vollständig korrekt gelöst. Die verbleibenden Verluste konzentrieren sich damit vollständig auf jene Schritte, die eine Ermessens- oder Bedeutungsentscheidung verlangen.

Den unmittelbarsten Vergleich liefert dabei die regelbasierte Pipeline, die für diese Arbeit ebenfalls in beiden Formen ausgeführt wurde. Ihre Werte sind allerdings asymmetrisch zu lesen: In den formal beschreibbaren Schritten verwendet sie dieselben Routinen, aus denen auch die Kettenreferenz komponiert ist, sodass ein Ergebnis von 100 Prozent dort definitionsgemäß eintritt und keine Leistung ausweist. Aussagekräftig sind die beiden Schritte mit begrenzten Heuristiken, die sie mit 99,3 und 97,6 Prozent löst, und vor allem der Endschritt: Auch die Regel löst ihn mit fehlerfreier Eingabe vollständig korrekt und fällt in der Kette auf 53,0 Prozent. Ihr Verlust von 47,0 Prozentpunkten übersteigt damit den jedes einzelnen Modells. Die Fehlerfortpflanzung ist folglich keine Eigenheit KI-gestützter Verarbeitung, sondern eine Eigenschaft der verketteten Architektur, die das deterministische Verfahren in gleichem, hier sogar stärkerem Maße trifft [2, 16].

In verketteten Strecken entscheidet die Qualität der frühen, semantisch geprägten Schritte über das Endergebnis, und ein Fehler wird umso teurer, je weiter vorn er entsteht. Welche Konsequenz daraus für die Gestaltung solcher Strecken folgt, behandelt Abschnitt 9.3 [16].

8.5. Reproduzierbarkeit der Ergebnisse

Die in Abschnitt 4.1 als querschnittliche Dimension verankerte Reproduzierbarkeit lässt sich auf zwei Ebenen bestimmen: über die zwei Wiederholungen jeder Konfiguration der Faktormatrix und über die fünf Wiederholungen der vollständigen Pipeline. Beide Ebenen liefern ein sehr unterschiedliches Bild, und gerade deren Gegenüberstellung ergibt den Einzelbefund mit der größten Tragweite für dieses Kapitel.

Auf Ebene der Einzelaufgabe beträgt die mittlere Standardabweichung über die zwei Wiederholungen jeder Konfiguration 3,7 Prozentpunkte. Bei 77,8 Prozent der Konfigurationen wurde derselbe Genauigkeitswert beobachtet. Das bedeutet nicht notwendigerweise identischen Code oder identische Ergebnistabellen. Die unscharfen Duplikate weisen mit 19,9 Prozentpunkten die höchste beobachtete Streuung auf, gefolgt von den Schlüssel-Duplikaten mit 4,7. Bei den abgeleiteten Spalten und exakten Duplikaten waren die Kennzahlen in beiden Wiederholungen gleich. Zwei Wiederholungen liefern allerdings nur einen ersten Hinweis auf die Variabilität, insbesondere lassen sie seltene Fehler und die Streuung künftiger Ausführungen nicht zuverlässig abschätzen. Da keine alternative Temperatur experimentell verglichen wurde, kann ihr Einfluss nicht aus diesen Messungen bestimmt werden.

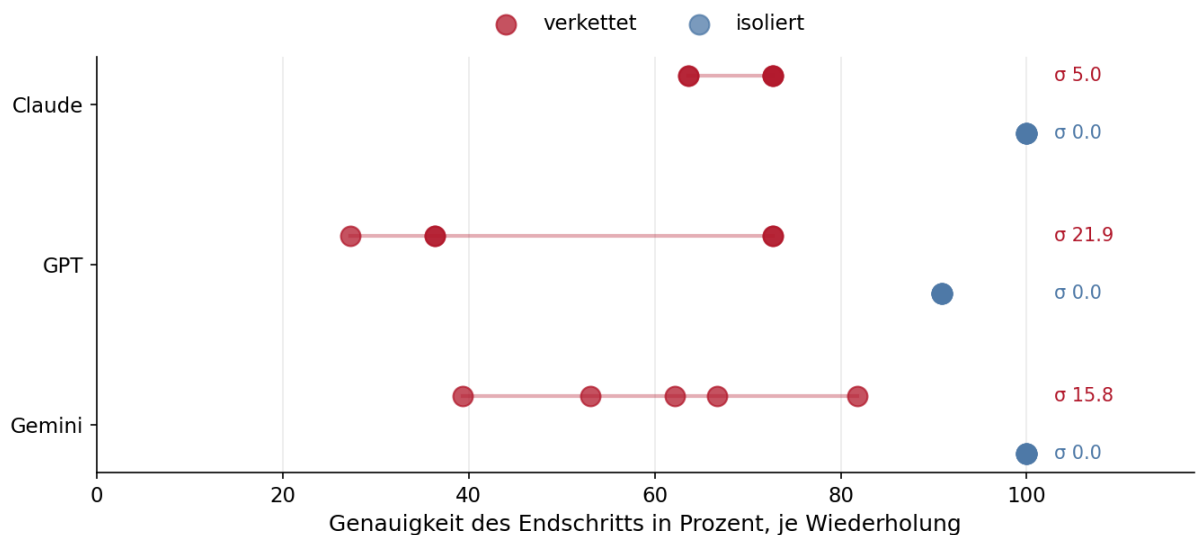


Abbildung 8.5.: Genauigkeit des Endschriffs in jeder der fünf Wiederholungen, verkettet und isoliert. Jeder Punkt ist ein vollständiger Durchlauf. Rechts ist die Standardabweichung in Prozentpunkten angegeben.

Auf Ebene der vollständigen Strecke ändert sich dieses Bild grundlegend, und Abbildung 8.5 zeigt worin. Der Gesamtwert der Strecke wirkt zunächst ebenfalls stabil: Die Standardabweichung über die fünf verketteten Wiederholungen beträgt 0,6 Prozentpunkte bei Claude, 2,0 bei Gemini und 3,2 bei GPT. Der Endschrift zeigt jedoch ein völlig anderes Verhalten. Dort schwankt das Ergebnis desselben Modells bei identischem Prompt und identischen Eingabedaten zwischen 27,3 und 72,7 Prozent bei GPT sowie zwischen 39,4 und 81,8 Prozent bei Gemini. Lediglich Claude bleibt mit 63,6 bis 72,7 Prozent in einem engeren Band. Eine einzelne Messung des Endergebnisses ist damit praktisch nicht aussagekräftig: Je nach Wiederholung ließe sich dasselbe Modell als klar besser oder klar schlechter als die regelbasierte Pipeline darstellen.

Im isolierten Durchlauf wurde bei allen drei Modellen über die fünf Wiederholungen derselbe Genauigkeitswert des Endschriffs gemessen. Die Standardabweichung des über die Schritte gemittelten Gesamtwerts liegt dort bei 0,08 bis 0,62 Prozentpunkten. Das ist mit der Erklärung vereinbar, dass Unterschiede vorgelagerter Ausgaben in der verketteten Verarbeitung verstärkt werden: Die Wahl des Duplikat-Repräsentanten beeinflusst, welche Bestellungen im Endschrift einen Kundendatensatz finden. Modellbedingte Variation und ihre Wirkung innerhalb der Pipeline sind deshalb gemeinsam zu betrachten. Ein von der Modellgenerierung unabhängiger kausaler Anteil der Architektur wurde nicht bestimmt.

Für die Praxis spricht dies dafür, sowohl einzelne Schritte als auch die gesamte Strecke wiederholt zu prüfen. Im Code-Generierungs-Modus kann ein geprüfter Code-Stand versioniert und erneut ausgeführt werden, ohne die Generierung bei jedem Lauf zu wiederholen. Das reduziert diese Quelle von Variation. Ebenso sollten die Eingabestände für Vergleichsläufe dokumentiert werden. Bei neuen Daten bleiben jedoch erneute inhaltliche Prüfungen erforderlich, identische Kennzahlen in wenigen Wiederholungen garantieren weder identische Ausgaben noch künftige Fehlerfreiheit.

8.6. Vergleich der Ausführungsmodi

Die Direktverarbeitung wurde auf drei Ebenen erhoben: als Einzelaufgabe, als vollständige Kette und in der zusammengesetzten Aufgabe des vorangegangenen Abschnitts. Da die vollständigen Tabellen im Prompt mitgeführt werden müssen, kommt für die Kette die halbierte Fassung des Datensatzes mit dem Startwert 102 zum Einsatz. Die Werte sind entsprechend nicht zeilengleich mit denen der Code-Generierung, wohl aber strukturgleich.

Der erste und praktisch folgenreichste Befund betrifft nicht die inhaltliche Qualität, sondern das Ausgabebudget. Die Direktverarbeitung wurde mit einem Ausgabelimit von 32.000 Token je Aufruf ausgeführt, das gegenüber den übrigen Läufen bewusst angehoben, aber begrenzt gehalten wurde, um die durch die eingebetteten Daten ohnehin hohen Kosten nicht unbegrenzt wachsen zu lassen. Von den 27 Einzelaufgaben lieferten sieben eine unvollständige Tabelle. Tabelle 8.2 weist die fünf gravierendsten aus.

Tabelle 8.2.: Die fünf gravierendsten unvollständigen Ergebnistabellen der Direktverarbeitung. Vier davon enden am gesetzten Ausgabelimit von 32.000 Token. (Die Soll-Zeilenzahl folgt aus den 540 Rohdaten abzüglich der zu entfernenden Duplikate: 520 bei den Schlüssel-Duplikaten, 500 bei den unscharfen, die beide Duplikat-klassen auflösen, vgl. Abschnitt 5.3.)

Aufgabe	Anbieter	Zeilen	Soll	Ausgabe-Token
Abgeleitete Spalten	Anthropic	1.123	2.000	32.000
Abgeleitete Spalten	Google	718	2.000	31.996
Schlüssel-Duplikate	Anthropic	247	520	32.000
Unscharfe Duplikate	Google	319	500	31.996
Abgeleitete Spalten	OpenAI	100	2.000	3.333

Vier dieser Läufe enden exakt am gesetzten Limit, ein weiterer bricht bereits bei einem Zehntel davon von selbst ab. Aufschlussreich ist, wie präzise sich der Abbruch in der Genauigkeit abbildet: Die abgeglichene Genauigkeit entspricht nahezu exakt dem gelieferten Zeilenanteil. Was geliefert wurde, war also inhaltlich korrekt. Verloren ging ausschließlich, was gar nicht mehr ausgegeben wurde. Die bestimmende Grenze dieses Modus ist damit keine der Ergebnisqualität, sondern des Ausgabebudgets, und dieses schlägt unmittelbar in Kosten um (Abschnitt 8.8). Verschärfend wirkt, dass Modelle mit internem Denkprozess dessen Token dem Ausgabebudget zurechnen, so dass sich der Bedarf nicht aus der erwarteten Tabellengröße abschätzen lässt [26, 32].

Wo die Antwort vollständig ausfiel, ist die Qualität dagegen konkurrenzfähig. Die drei Bereinigungsaufgaben löst die Direktverarbeitung mit 96,0 bis 100 Prozent, die semantische Länder-Vereinheitlichung mit 97,1 bis 100 Prozent und damit als einziger Ansatz über der regelbasierten Pipeline. Bei den unscharfen Duplikaten erreicht sie 93,7 bis 96,0 Prozent und bei den Schlüssel-Duplikaten bis zu 97,9 Prozent, sofern die Antwort nicht abbrach. In der verketteten Ausführung erreicht sie in den Schritten 1 bis 8 zwischen 95,9 und 100 Prozent und liegt damit auf dem Niveau der Code-Generierung.

Der Endschrift bricht jedoch auf 2,7 bis 7,6 Prozent ein und damit weit unter jeden anderen Ansatz. Entscheidend ist, dass der isolierte Durchlauf hier nichts rettet: Mit fehlerfreier Eingabe erreicht der Endschrift lediglich 1,5 bis 4,5 Prozent. Anders als bei der Code-Generierung ist dieser Einbruch also gerade nicht geerbt, sondern eigenes Versagen, und die in Abschnitt 8.7.2 berichtete Zerlegung benennt seine Ursache genau: Das Modell muss die Aggregation über mehrere Dutzend Gruppen selbst rechnen, und diese arithmetische Zusammenfassung mehrerer hundert Einzelwerte gelingt ihm nicht.

In der Zusammenschau ergibt sich ein klares Einsatzprofil. Als durchgängiger Verarbeitungsmodus für eine vollständige ETL-Strecke ist die Direktverarbeitung nicht zu begründen: Sie ist an das Ausgabefenster gebunden, arithmetisch unzuverlässig und um mehr als eine Größenordnung teurer je verarbeiteter Zeile. Sinnvoll einsetzbar ist sie allein als gezielt vorgeschalteter Einzelschritt für Teilaufgaben, die Einzelfallverständnis verlangen, auf überschaubaren Datenmengen arbeiten und keine Aggregation enthalten, eine Konstellation, die in dieser Untersuchung genau auf die semantische Vereinheitlichung zutrifft.

8.7. Zuschnitt der Aufgabenstellung

Neben Aufgabe, Modell und Prompting-Strategie ist wiederholt eine vierte Einflussgröße sichtbar geworden: der Zuschnitt der Aufgabenstellung, also die Frage, wie viele Verarbeitungsentscheidungen in einem Arbeitsschritt zusammengefasst werden und wie ausdrücklich sie benannt sind. Als Untersuchungsebene ist sie in Tabelle 4.1 vorgesehen. Die ausdrücklich gegliederte Fassung der zusammengesetzten Aufgabe in Abschnitt 8.7.2 wurde erst nach Sichtung der Ergebnisse aus Abschnitt 8.1.2 ergänzt und ist entsprechend als explorative Nachuntersuchung zu lesen, nicht als vorab geplanter Vergleich.

8.7.1. Gruppierte Strecke

Die in Abschnitt 8.3 berichteten Strategieunterschiede unterliegen einer Einschränkung, die aus dem Aufbau der Faktormatrix folgt. Jede Aufgabe wird dort auf den unveränderten Rohdaten gestellt, sodass die anspruchsvollen Stufen zusätzlich alle vorgelagerten Mängel mitbehandeln müssen. Die Messung ist deshalb genau bei jenen Aufgaben unscharf, bei denen ein Strategieeffekt am ehesten zu erwarten wäre.

Um diese Blindstelle zu schließen, wurde eine Untersuchungsebene ergänzt, die zwischen der Einzelaufgabe und der vollständigen Kette aus neun Schritten liegt. Die neun Pipeline-Schritte werden zu drei Gruppen von je drei Schritten zusammengefasst: Bereinigung, Deduplizierung und Transformation. Jede Gruppe erhält die Soll-Ausgabe ihrer Vorgängergruppe als Eingabe. Die Anweisungen der drei zugehörigen Schritte werden dabei wörtlich aus der Pipeline übernommen und nummeriert in einem Prompt zusammengefasst, sodass die Gruppe inhaltlich dasselbe verlangt wie die drei Einzelschritte. Damit bleibt die Komposition mehrerer Verarbeitungsschritte als Anforderung erhalten, genau jene Eigenschaft, an der sich eine Strategie wie das Chain-of-Thought-Prompting bewähren sollte, ohne dass ein Fehler aus einer früheren Gruppe

das Ergebnis überdeckt. Ausgeführt wurden 3 Strategien $\times$ 3 Modelle $\times$ 3 Gruppen $\times$ 2 Wiederholungen, also 54 Läufe.

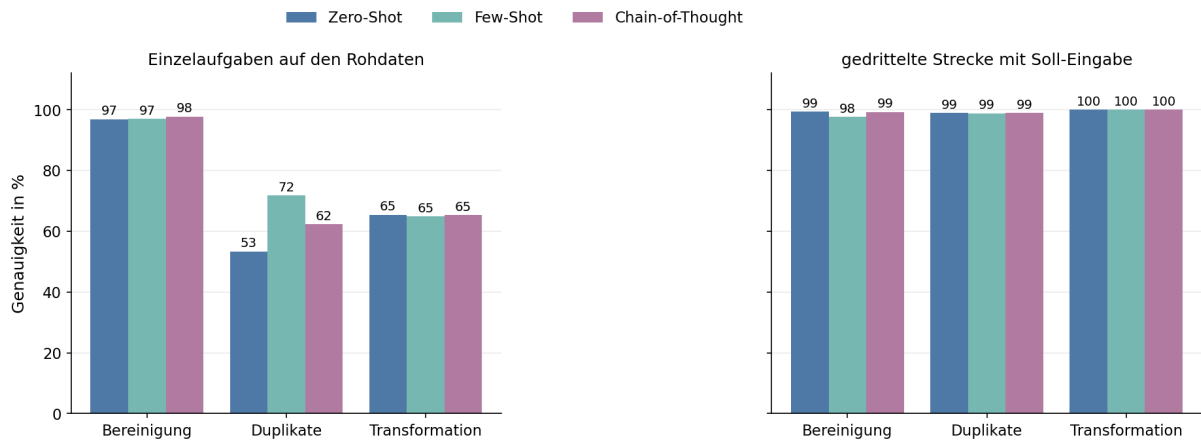


Abbildung 8.6.: Dieselben Verarbeitungsschritte als unabhängige Einzelaufgaben auf den Rohdaten (links) und als gruppierte Strecke mit Soll-Eingabe je Gruppe (rechts), gemittelt über die drei kommerziellen Modelle. Der Strategieunterschied bei den Duplikaten verschwindet vollständig.

Sämtliche 54 Läufe lieferten ein verwertbares Ergebnis, und die Genauigkeit liegt durchgängig zwischen 95,3 und 100 Prozent. Über alle Gruppen und Wiederholungen gemittelt erreichen das Zero-Shot- und das Chain-of-Thought-Prompting jeweils 99,4 Prozent und das Few-Shot-Prompting 98,8, eine Spanne von 0,6 gegenüber 6,1 Prozentpunkten, die dieselben drei Strategien in der Faktormatrix trennten. Auch zwischen den Modellen ist mit 99,0 bis 99,3 Prozent im Rahmen der erhobenen Wiederholungen kein Unterschied mehr erkennbar, und beide Wiederholungen liefern mit 99,3 und 99,1 Prozent nahezu denselben Gesamtwert. Besonders deutlich zeigt sich das an der Transformationsgruppe: Sie enthält als letzten Schritt genau jene Aggregation, die als Einzelaufgabe in allen Läufen scheiterte, und wird hier von jedem Modell, in jeder Strategie und in beiden Wiederholungen vollständig korrekt gelöst.

Abbildung 8.6 stellt beide Betrachtungsweisen unmittelbar gegenüber und macht damit die eigentliche Konsequenz sichtbar. Der auffälligste Strategieunterschied der Faktormatrix lag bei der Duplikaterkennung mit 53, 72 und 62 Prozent. Er verschwindet vollständig, sobald dieselben Schritte mit sauberer Eingabe und in einem klar abgegrenzten Zuschnitt gestellt werden. Was in der Matrix als Wirkung der Prompt-Gestaltung erscheint, ist demnach zu einem erheblichen Teil eine Wirkung des Aufgabenzuschnitts: Beispiele und schrittweise Herleitung helfen einem Modell, eine überladene Aufgabe zu bewältigen, sind aber entbehrlich, sobald die Aufgabe angemessen zugeschnitten ist. Für die Beantwortung von FF2 ist das ein Befund von eigenständigem Gewicht, denn er kehrt die übliche Reihenfolge der Gestaltungsentscheidungen um: Nicht der Prompt ist zuerst zu optimieren, sondern der Zuschnitt der Aufgabe, und ist dieser einmal richtig gewählt, verliert die Wahl der Prompting-Strategie ihre Bedeutung weitgehend [1, 29].

8.7.2. Explizite statt impliziter Anforderungen

Die gruppierte Strecke verändert zwei Dinge gleichzeitig: Sie verkleinert den Zuschnitt *und* stellt fehlerfreie Eingabedaten bereit. Offen bleibt damit, welcher der beiden Eingriffe wirkt. Ein zweites Experiment trennt sie, indem es den Zuschnitt bewusst *nicht* verkleinert: Gestellt wird erneut die vollständige zusammengesetzte Aufgabe aus Abschnitt 8.1.2 auf den unveränderten Rohdaten, in einem einzigen Arbeitsschritt, jedoch mit sechs ausdrücklich benannten und nummerierten Teilschritten anstelle der pauschalen Aufforderung, „so weit wie nötig“ zu bereinigen. Insbesondere wird die Vereinheitlichung der Länderangaben nun ausdrücklich verlangt, einschließlich der Vorgabe, nicht zuzuordnende Werte als UNKNOWN zu kennzeichnen. Die Soll-Lösung folgt diesen sechs Schritten wörtlich und ist damit, anders als jene der Faktormatrix, aus den Eingabedaten allein erreichbar. Die Aufgabe wurde in beiden Ausführungsmodi, unter allen drei Prompting-Strategien und in je zwei Wiederholungen gestellt, sodass der Modusvergleich nicht länger einen optimierten gegen einen festen Prompt stellt.

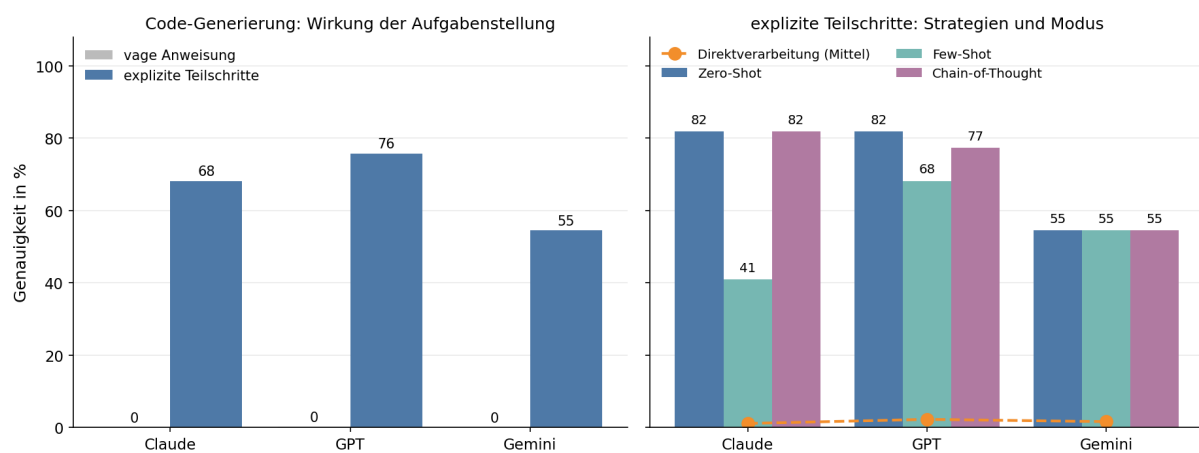


Abbildung 8.7.: Links dieselbe Verarbeitung mit vager und mit ausdrücklich benannter Anforderung im Code-Generierungs-Modus. Rechts das Ergebnis der expliziten Fassung nach Strategie sowie das Mittel der Direktverarbeitung.

Abbildung 8.7 fasst den deutlichsten Einzelbefund dieses Kapitels zusammen. Im Code-Generierungs-Modus steigt die Genauigkeit von 0,1 Prozent auf 75,8 Prozent bei GPT sowie 68,2 und 54,5 Prozent bei Claude und Gemini. Entscheidender als die Zahlen ist die Diagnose dahinter: In der ursprünglichen Fassung gruppierten die Modelle über 41 oder 42 Rohschreibweisen der Länderangabe, in der expliziten Fassung über die vorgesehenen elf Werte, also die zehn ISO-Codes zuzüglich UNKNOWN für nicht zuordenbare Angaben, in nahezu jedem Lauf, unabhängig von Modell und Strategie. Die Modelle können die Komposition mehrerer Verarbeitungsschritte also durchaus leisten. Sie hatten in der ursprünglichen Fassung schlicht eine Anforderung nicht erraten, die nie ausgesprochen wurde.

Was als Fehler verbleibt, ist der bereits aus Abschnitt 8.1.1 bekannte Mechanismus, nun jedoch ohne störende Überlagerung messbar. Der beste Lauf trifft 54 der 66 Ergebniszeilen auf den Cent genau und erfasst alle 2.000 Bestellungen. Die Gesamtsumme des Umsatzes stimmt exakt. Die zwölf abweichenden Zeilen gehen sämtlich

darauf zurück, dass die erzeugte Zuordnungstabelle das Kürzel `SUI` für die Schweiz nicht enthält, sodass 47 Bestellungen in die Gruppe `UNKNOWN` wandern. Der Fehler der Code-Generierung ist damit systematisch und summenerhaltend: Er verschiebt Datensätze zwischen Gruppen, ohne zu rechnen.

Die Direktverarbeitung liefert das exakte Gegenbild. Sie vereinheitlicht die Länderangaben in nahezu allen Läufen korrekt, liefert die richtige Zahl von Ergebniszeilen und trifft die Gesamtsumme der Bestellungen, und erreicht dennoch nur 1,2 bis 2,3 Prozent, in beiden Wiederholungen nahezu unverändert. Im besten Lauf ist keine einzige der 66 Umsatzzellen auf den Cent genau, die typische Abweichung beträgt rund 4.800 Euro bei Werten um 20.000 Euro, während die Randsummen näherungsweise stimmen. Das Modell erzeugt plausibel wirkende Zahlen, statt zu rechnen. Damit sind die beiden Fähigkeiten innerhalb dieser Untersuchung an derselben Aufgabe und unter demselben Prompt getrennt gemessen: Die semantische Zuordnung beherrscht die Direktverarbeitung besser, die Arithmetik über viele Datensätze beherrscht allein die Code-Generierung [6, 46].

Der Strategievergleich bestätigt hier das Bild aus Abschnitt 8.7.1: Zero-Shot und Chain-of-Thought liegen mit 72,7 und 71,2 Prozent gleichauf, das Few-Shot-Prompting kommt auf 54,5 Prozent. Ob dieser Abstand einen reproduzierbaren Strategieeffekt darstellt, bleibt angesichts der beiden Wiederholungen offen: Er entsteht vollständig aus einem einzigen Lauf, in dem der Few-Shot-Prompt bei Anthropic auch nach drei Versuchen keinen lauffähigen Code erzeugte, während dieselbe Konfiguration in der zweiten Wiederholung 81,8 Prozent erreichte. Der Ausfall bleibt Bestandteil der berichteten Mittelwerte. Seine Bedeutung zeigt, wie empfindlich die Rangfolge bei wenigen Wiederholungen auf einen einzelnen Lauf reagiert, eine gemeinsame Spannweite allein belegt keine Gleichwertigkeit der Strategien.

Genau darin liegt der eigentliche Befund dieses Abschnitts zur Prompt-Gestaltung: Die Streuung zwischen zwei Wiederholungen derselben Konfiguration übersteigt hier den Unterschied zwischen den Strategien. Anthropic schwankte zwischen 54,5 und 81,8 Prozent, während Google in beiden Wiederholungen und allen drei Strategien konstant 54,5 Prozent lieferte. Wer aus einer einzelnen Messung eine Strategieempfehlung ableitet, kann unter diesen Bedingungen zufällige Schwankungen überbewerten, dieselbe Schlussfolgerung, die Abschnitt 8.5 für die verkettete Strecke gezogen hat, nun auf der Ebene einer Einzelaufgabe.

8.8. Aufwand und Kosten

Preisgrundlage der dargestellten Auswertung. Die nachstehenden Kostenbeträge und die Kostenabbildungen beruhen auf den in Tabelle 4.2 ausgewiesenen Listenpreisen des jeweiligen Anbieters. Sie bilden ausschließlich geschätzte Modellnutzungskosten ab, Entwicklungs-, Prüf-, Wartungs- und Hardwarekosten wurden nicht vollständig erhoben.

Abbildung 8.8 und Tabelle 8.3 beziffern den Aufwand. Da die Direktverarbeitung nur die halbierte Fassung des Datensatzes verarbeitet, sind die Kennzahlen zusätzlich auf je 1.000 verarbeitete Zeilen normiert und damit unmittelbar vergleichbar. Auf dieser

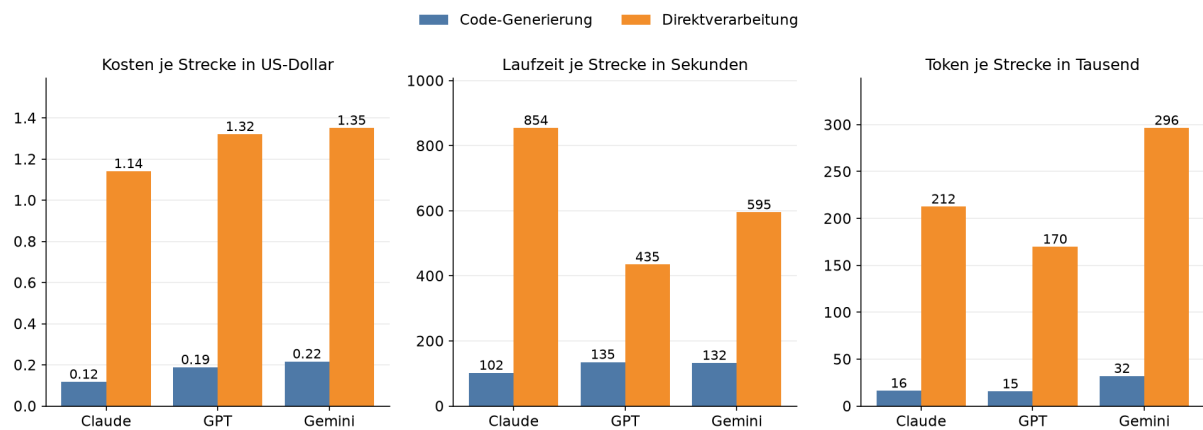


Abbildung 8.8.: Kosten, Laufzeit und Token-Verbrauch einer vollständigen Strecke aus neun Schritten je Ausführungsmodus, gemittelt über die Wiederholungen. Die Direktverarbeitung arbeitet dabei auf dem halbierten Datensatz.

Tabelle 8.3.: Aufwand einer vollständigen Strecke aus neun Schritten, absolut und normiert auf 1.000 verarbeitete Zeilen. Die Normierung ist erforderlich, weil die Direktverarbeitung auf der halbierten Fassung des Datensatzes mit 1.361 statt 2.640 Zeilen ausgeführt wurde. Angegeben ist jeweils die Spanne über die drei kommerziellen Modelle.

Kennzahl	Code-Gen.	Direkt	Regelbasiert
Kosten je Strecke (US-Dollar)	0,12–0,22	1,14–1,35	0,00
Laufzeit je Strecke (Sekunden)	102–135	435–854	0,2
Modell-Token je Strecke	15.000–32.000	170.000–296.000	–
Kosten je 1.000 Zeilen (US-Dollar)	0,04–0,08	0,84–0,99	0,00
Laufzeit je 1.000 Zeilen (Sekunden)	39–51	319–628	0,06

Grundlage kostet der Direkt-Modus rund das Fünfzehnfache, dauert rund elfmal so lange und setzt rund zwanzigmal so viele Token um. Der Grund liegt in der Struktur der Verarbeitung: Im Code-Generierungs-Modus durchläuft nur die Aufgabenbeschreibung das Modell, während die Daten lokal verarbeitet werden. Im Direkt-Modus wandert die vollständige Tabelle in jedem der neun Schritte einmal hinein und einmal wieder heraus.

Zwischen den kommerziellen Modellen fallen die Unterschiede dagegen deutlich geringer aus als zwischen den Ausführungsmodi. Über die je 54 Läufe eines Modells liegen die Nutzungskosten zwischen 0,67 und 1,10 US-Dollar, das günstigste Modell kostet damit etwa zwei Drittel des teuersten. Die Antwortzeiten unterscheiden sich mit 10,8 bis 13,6 Sekunden je Aufruf nur moderat. Gegenüber dem Abstand zwischen den Aufgaben und zwischen den Ausführungsmodi bleibt dieser Kostenunterschied klein, sodass die Modellwahl sinnvoller der Zuverlässigkeit bei der vorliegenden Aufgabenart folgt als dem Preis. Die Auswahl folgt vielmehr der Zuverlässigkeit bei der konkret vorliegenden Aufgabenart.

Auch der Zuschnitt der Aufgabenstellung wirkt auf den Aufwand. Da jeder Lauf der gruppierten Strecke drei Verarbeitungsschritte umfasst, decken die 54 Läufe der grup-

pierten Strecke dieselben 162 Aufgabenausführungen ab wie die 162 Läufe der kommerziellen Modelle in der Faktormatrix, und kosteten dabei 2,21 gegenüber 2,80 US-Dollar. Vor allem aber lieferten sie ausnahmslos ein verwertbares Ergebnis, 49 davon im ersten Versuch, während in der Faktormatrix ein erheblicher Teil der Läufe ohne Ergebnis blieb. Der kleinteiligere Zuschnitt ist damit nicht nur genauer, sondern zugleich günstiger und zuverlässiger, eine Kombination, die in dieser Untersuchung sonst an keiner Stelle auftritt.

Der lokale Betrieb verursacht keine nutzungsabhängigen Kosten. Dieser Vergleich erfasst allerdings nur eine Seite der Rechnung: Anschaffung und Betrieb der erforderlichen Hardware wurden hier nicht bilanziert, sodass sich aus den erhobenen Daten kein vollständiger Kostenvergleich, sondern allein die Aussage ableiten lässt, dass die laufenden Kosten mit steigendem Verarbeitungsvolumen zugunsten des lokalen Betriebs skalieren [5, 26].

8.9. Ressourcenverbrauch und Nachhaltigkeitsaspekte

Die im vorangegangenen Abschnitt erhobenen Aufwandskennzahlen lassen sich nicht nur wirtschaftlich, sondern auch unter dem Gesichtspunkt der Nachhaltigkeit lesen. Da jeder Modellaufruf Rechenleistung in einem Rechenzentrum beansprucht, ist die Wahl zwischen regelbasierter und modellgestützter Verarbeitung zugleich eine Entscheidung über den Ressourceneinsatz einer ETL-Strecke [26, 32]. Dieser Abschnitt wertet die vorliegenden Messungen entlang der drei Dimensionen der ökologischen, der ökonomischen und der sozialen Nachhaltigkeit aus.

Vorauszuschicken ist der Umfang dessen, was die Messungen dazu hergeben. Energieverbrauch und Treibhausgasemissionen wurden nicht erhoben, und sie lassen sich für die kommerziell betriebenen Modelle auch nicht erheben, da die Anbieter weder Auslastung noch Hardware ihrer Rechenzentren offenlegen [5, 26]. Die folgenden Aussagen stützen sich deshalb auf den Token-Durchsatz als Näherungsgröße für die aufgewendete Rechenleistung. Die gemessene Laufzeit ist dafür ausdrücklich *kein* geeigneter Maßstab: Sie enthält bei den Cloud-Modellen Netz- und Wartezeiten und beschreibt damit die Antwortzeit, nicht den Rechenaufwand. Die berichteten Größenordnungen sind entsprechend als Verhältnisse zu lesen, nicht als absolute Verbrauchswerte.

8.9.1. Ökologische Dimension

Der erste und deutlichste Befund ist die Spreizung des Rechenaufwands zwischen den drei untersuchten Ansätzen für ein identisches Verarbeitungsergebnis. Die regelbasierte Pipeline löst die vollständige Strecke aus neun Schritten ohne jeden Modellaufruf. Der Code-Generierungs-Modus setzt dafür 15.000 bis 32.000 Token um, der Direkt-Modus 170.000 bis 296.000 und, auf die verarbeitete Zeilenzahl normiert, rund das Zwanzigfache des Code-Generierungs-Modus (Abschnitt 8.8). Die Rangfolge des Ressourcenverbrauchs entspricht damit exakt der Rangfolge der Kosten, was nicht überrascht, da die Nutzungsgebühren je Token abgerechnet werden und den Rechenaufwand insoweit abbilden.

Bemerkenswert ist die Ursache dieser Spreizung, denn sie ist eine Eigenschaft der Architektur und nicht der Aufgabe. Im Code-Generierungs-Modus durchläuft allein die Aufgabenbeschreibung das Modell, während die Daten lokal verarbeitet werden. Im Direkt-Modus wandert die vollständige Tabelle in jedem der neun Schritte einmal in das Modell hinein und einmal wieder heraus. Der Ressourcenbedarf des Direkt-Modus wächst deshalb linear mit der Datenmenge, jener des Code-Generierungs-Modus bleibt bei festem Prompt davon unberührt. Für wachsende Datenbestände verschiebt sich das Verhältnis somit weiter zugunsten der Code-Generierung.

Der zweite Befund betrifft eine Aufwandsform, die in Kostenbetrachtungen üblicherweise untergeht: verworfene Rechenleistung. Ein Lauf, der kein verwertbares Ergebnis liefert, hat die Rechenleistung dennoch vollständig beansprucht. In der Faktormatrix brachen 48 der 216 Läufe mit einem Programmfehler ab, weitere 18 lieferten eine strukturell abweichende Ausgabe; nur 150 Läufe führten zu einem verwertbaren Ergebnis. Rund ein Fünftel des gesamten Rechenaufwands dieser Untersuchungsebene ist damit ohne Ertrag geblieben. Im Direkt-Modus tritt derselbe Effekt in besonders klarer Form auf: Vier der 27 Einzelaufgaben endeten exakt am gesetzten Ausgabelimit von 32.000 Token, das heißt, das Modell hat das volle Ausgabebudget verbraucht und dennoch eine unvollständige und damit unbrauchbare Tabelle geliefert (Tabelle 8.2). Verschärfend wirkt, dass Modelle mit internem Denkprozess auch dessen Token dem Ausgabebudget zurechnen, sodass Rechenleistung aufgewendet wird, die im Ergebnis nicht erscheint [32].

Daraus folgt der für die Praxis wesentliche Zusammenhang: Zuverlässigkeit und Ressourcenschonung fallen hier zusammen. Ein Ansatz, der häufiger abbricht, verbraucht nicht nur mehr Rechenleistung je gelungenem Ergebnis, sondern erzwingt zusätzlich Wiederholungsaufrufe. Am deutlichsten zeigt das der Vergleich der Aufgabenzuschnitte. Die 54 Läufe der gruppierten Strecke decken dieselben 162 Aufgabenausführungen ab wie die 162 Läufe der kommerziellen Modelle in der Faktormatrix, kosteten dabei 2,21 gegenüber 2,80 US-Dollar und lieferten *ausnahmslos* ein verwertbares Ergebnis, 49 davon im ersten Versuch. Der kleinteiligere Zuschnitt ist damit gleichzeitig genauer, günstiger und ressourcenschonender. Dies ist der einzige Punkt der Untersuchung, an dem keine Abwägung zwischen Ergebnisqualität und Aufwand erforderlich ist.

Für das lokal betriebene Modell ist die Bilanz uneindeutig und ausdrücklich offen. Es verursacht keine nutzungsabhängigen Kosten und keinen Datenabfluss, doch die Herstellung und der Betrieb der erforderlichen Hardware wurden hier nicht bilanziert. Hinzu tritt, dass 25 seiner 54 Läufe mit einem Programmfehler abbrachen: Ein hoher Anteil verworfener Läufe entwertet den Vorteil des lokalen Betriebs, weil dieselbe Aufgabe mehrfach gerechnet werden muss. Aus den erhobenen Daten lässt sich daher nicht ableiten, welche Betriebsform ökologisch vorzuziehen ist, sondern allein, dass die laufenden Nutzungsgebühren mit steigendem Verarbeitungsvolumen zugunsten des lokalen Betriebs skalieren [5, 26].

8.9.2. Ökonomische und soziale Dimension

Die ökonomische Dimension ist in Abschnitt 8.8 beziffert und hier nur um ihre Grenze zu ergänzen. Erfasst sind ausschließlich die Nutzungsgebühren der Modelle. Der Auf-

wand für die Erstellung und Prüfung der Anforderungen, für die Wartung der Strecke und für die erforderliche Hardware wurde nicht erhoben. Gerade dieser Aufwand ist es jedoch, der den Einsatz von LLMs wirtschaftlich motiviert (Abschnitt 1.1). Eine tragfähige Wirtschaftlichkeitsrechnung setzt seine Erhebung voraus und ist auf Grundlage dieser Arbeit nicht möglich. Belastbar ist allein die Aussage, dass die Nutzungsgebühren für die untersuchte Strecke gering ausfallen, mit 0,12 bis 0,22 US-Dollar je vollständigem Durchlauf im Code-Generierungs-Modus, und dass die regelbasierte Ausführung dort, wo sie möglich ist, sowohl kostenfrei als auch um Größenordnungen schneller bleibt.

Die soziale Dimension berührt die Arbeit an zwei Stellen, die sich aus den Befunden unmittelbar ergeben. Die erste ist die Verschiebung der Tätigkeit von Datenfachkräften. Die Ergebnisse legen nahe, dass der Beitrag der Modelle nicht in der Ausführung, sondern in der Erstellung der Verarbeitungsvorschrift liegt (Abschnitt 9.1). Damit verlagert sich die Tätigkeit vom Formulieren von Regeln zum Prüfen erzeugter Ergebnisse. Diese Verlagerung entlastet nicht ohne Weiteres: Der in Skript 8.1 dokumentierte Fall eines Codes, der fehlerfrei durchläuft, schemakonform ausgibt und dennoch nichts bewirkt, zeigt, dass die Prüfung inhaltlich zu erfolgen hat und weder durch Fehlermeldungen noch durch eine Sichtprüfung ersetzbar ist. Die Verantwortung für das Ergebnis bleibt bei der prüfenden Person, während die Nachvollziehbarkeit des Zustandekommens abnimmt. Ein durchgängig autonomer Einsatz, wie er in dieser Arbeit gemessen wurde, ist auf dieser Grundlage nicht zu verantworten [6, 7].

Die zweite Stelle ist der Verbleib der Daten. Die Direktverarbeitung überträgt die vollständigen Datensätze an einen externen Anbieter, die Code-Generierung überträgt lediglich die Aufgabenbeschreibung, und der lokale Betrieb überträgt nichts. Für personenbezogene Bestände, wie sie das hier verwendete Szenario nachbildet, ist dieser Unterschied erheblich und tritt neben die Qualitäts- und Kostenbetrachtung. Er wurde in dieser Arbeit nicht rechtlich bewertet (Abschnitt 4.1), verschiebt die Empfehlung aus Abschnitt 9.3 aber in dieselbe Richtung wie die Ressourcenbetrachtung: Je weniger Schritte ein Modell benötigen, desto geringer sind Rechenaufwand und übertragene Datenmenge zugleich.

Tabelle 8.4 fasst diese Auswertung zusammen. Für die Gesamtaussage der Arbeit ist dabei bedeutsam, dass die nachhaltigkeitsbezogene Betrachtung zu derselben Empfehlung führt wie die qualitätsbezogene und nicht zu einer konkurrierenden. Die in Abschnitt 9.3 abgeleitete hybride Architektur führt sieben der neun Schritte regelbasiert aus und benötigt ein Modell allein für die Länder-Vereinheitlichung und die unscharfe Duplikaterkennung. Sie senkt damit die Zahl der Modellaufrufe je Strecke von neun auf zwei und mit ihr den Rechenaufwand, die Nutzungsgebühren und die übertragene Datenmenge, ohne die Ergebnisqualität an den beiden semantisch geprägten Schritten aufzugeben. Der sparsame Einsatz der Modelle ist hier folglich nicht der Preis für Nachhaltigkeit, sondern die Folge derselben Analyse, die auch die Qualität optimiert [2, 40]. Zu betonen bleibt, dass diese Variante abgeleitet und nicht als Ganzes gemessen wurde.

Tabelle 8.4.: Nachhaltigkeitsbezogene Auswertung der erhobenen Kennzahlen entlang der drei Dimensionen. Die letzte Spalte nennt den jeweils tragenden Messwert beziehungsweise die Stelle, an der er erhoben wurde.

Dimension	Befund	Beleg
Ökologisch	Rechenaufwand für dasselbe Ergebnis spreizt sich um Größenordnungen; die regelbasierte Ausführung benötigt keinen Modellaufruf	15.000–32.000 gegen 170.000–296.000 Token je Strecke, normiert rund Faktor 20
Ökologisch	Verworfen Rechenleistung ist eine eigene Aufwandsform; Zuverlässigkeit und Ressourcenschonung fallen zusammen	48 Programmabbrüche und 18 Schemabbrüche von 216 Läufen; 4 Läufe verbrauchen das volle Ausgabebudget ohne verwertbares Ergebnis
Ökologisch	Kleinteiliger Zuschnitt senkt den Aufwand und erhöht gleichzeitig die Qualität	54 Läufe der gruppierten Strecke: 2,21 gegen 2,80 US-Dollar bei gleicher Zahl an Aufgabenausführungen, alle verwertbar
Ökonomisch	Nutzungsgebühren sind gering, die entscheidende Größe wurde jedoch nicht erhoben	0,12–0,22 US-Dollar je Strecke; Entwicklungs-, Prüf- und Wartungsaufwand nicht gemessen
Sozial	Tätigkeit verschiebt sich vom Formulieren der Regel zum Prüfen des Ergebnisses, ohne die Verantwortung zu verlagern	stilles Versagen nach Skript 8.1: fehlerfreier Durchlauf ohne Wirkung
Sozial	Die Ausführungsform bestimmt, welche Daten das eigene Netz verlassen	Direkt-Modus überträgt vollständige Tabellen, Code-Generierung nur die Aufgabenbeschreibung, lokaler Betrieb nichts

8.10. Vergleich mit der regelbasierten Pipeline

Im Mittel der kommerziellen Modelle bleibt die regelbasierte Pipeline bei keiner der neun Aufgaben zurück. Dieser Befund ist jedoch differenziert zu lesen. Für die sechs formal beschreibbaren Aufgaben ist er weitgehend definitorisch bedingt: Wie in Abschnitt 6.3 dargelegt, verwendet die regelbasierte Pipeline für diese Aufgaben dieselben deterministischen Routinen, die auch der Erzeugung der Soll-Lösung zugrunde liegen. Die explizit formulierte Regel *ist* dort die korrekte Lösung. Ein Ergebnis von 100 Prozent ist folglich kein Leistungsnachweis, sondern die zu erwartende Untergrenze für ein Verfahren, dessen Vorschrift mit der Aufgabendefinition zusammenfällt. Interessant ist an diesen Aufgaben nicht, dass die Regel gewinnt, sondern dass die Modelle in ihren besten Läufen gleichauf liegen, ohne dass die Regel zuvor formuliert werden musste.

Aussagekräftig ist der Vergleich daher erst bei den drei verbleibenden Aufgaben, von denen zwei die regelbasierte Pipeline bewusst nur mit begrenzten Heuristiken abdeckt. Bei der Länder-Vereinheitlichung erreicht sie 99,3 Prozent: Ihre Zuordnungstabelle deckt ISO-Codes in zwei- und dreistelliger Form, IOC-Kürzel sowie deutsche und englische Ländernamen ab, lässt aber einzelne Schreibweisen offen. Genau hier liegt die Trennlinie zwischen beiden Verfahrensarten: Eine Zuordnungstabelle kann nur erfassen, was zuvor jemand aufgezählt hat, während ein Sprachmodell erschließt, dass „Holland“ umgangssprachlich für die Niederlande steht. Dass die Code-Generierung diesen Vorteil dennoch nicht ausspielt und mit 96,7 Prozent zurückbleibt, lässt sich hier auf Lücken in den erzeugten Zuordnungstabellen zurückführen. Daraus folgt keine grundsätzliche Begrenzung aller codebasierten Lösungen auf diese Form des Nachschlagens. Erst die Direktverarbeitung, in der jeder Wert einzeln beurteilt wird, übertrifft die regelbasierte Lösung mit bis zu 100 Prozent.

Bei den unscharfen Duplikaten kehrt sich das Verhältnis erstmals um: Die regelbasierte Pipeline erreicht 96,7 Prozent, Claude und Gemini erreichen im besten Lauf 97,6 und 97,4 Prozent. Der Abstand ist gering, aber er markiert den einzigen Punkt der Untersuchung, an dem die Code-Generierung die ausformulierte Regel schlägt, und zwar bei genau jener Aufgabe, deren Lösung sich am schlechtesten in einer Vorschrift fassen lässt. Zugleich macht diese Konstellation eine Grenze des Untersuchungsdesigns sichtbar, die in Kapitel 10 aufzugreifen ist: Die eingebauten Abweichungen entstammen einer bekannten, endlichen Menge von Mustern, was regelbasierte Verfahren systematisch begünstigt. Wie sich dieses Verhältnis in realen Beständen mit unbekannten Abweichungsmustern verändert, bleibt empirisch zu prüfen [16, 17].

Der dritte aussagekräftige Fall ist die zusammengesetzte Aufgabe, bei der auch die regelbasierte Pipeline mit 4,5 Prozent nahezu vollständig scheitert, und zwar, wie in Abschnitt 8.1.2 dargelegt, weil die Soll-Lösung aus den Eingabedaten nicht erreichbar ist. Die Aufgabe trennt somit nicht zwischen den Ansätzen. In der verketteten Strecke hingegen lag die regelbasierte Pipeline mit 53,0 Prozent im Endschrift hinter Claude und Gemini, weil sie mehr Bestellungen ohne zugehörigen Kunden zurückließ, und verlor gegenüber ihrem eigenen isolierten Durchlauf mit 47,0 Prozentpunkten mehr als jedes Modell.

Zusammengefasst ergibt sich kein Verfahren, das über alle Aufgaben hinweg vorzuziehen wäre. Für formal beschreibbare Aufgaben ist die regelbasierte Umsetzung der KI-gestützten mindestens ebenbürtig und in Kosten, Geschwindigkeit und Determiniertheit deutlich überlegen, 0,18 Sekunden ohne Nutzungskosten gegenüber gut zwei Minuten und rund 0,17 US-Dollar für dieselbe Strecke. Ihr Vorteil endet jedoch genau dort, wo die Verarbeitungsvorschrift nicht mehr vollständig vorab formulierbar ist, und in diesem Bereich liegt der Beitrag der Modelle weniger im Ersetzen als im Ergänzen der Regel [17, 40].

8.11. Stärken, Schwächen und Risiken

Die deutlichste Stärke der untersuchten Modelle liegt nicht in der erreichten Genauigkeit, sondern im Aufwand, mit dem sie erreicht wird. Die hohe Ergebnisqualität der regelbasierten Vergleichspipeline ist das Resultat einer vorgelagerten Spezifikation, die sämtliche Verarbeitungsvorschriften einschließlich Zuordnungstabelle und Normalisierungsregel vorwegnimmt. Die Modelle erzeugen bei den formal beschreibbaren Aufgaben auf Grundlage der formulierten Anforderungen in Sekunden erste Lösungen mit teilweise vergleichbarer Ergebnisqualität. Der Aufwand für die Erstellung und Prüfung dieser Anforderungen wurde nicht gemessen, eine Entwicklungszeiterparnis lässt sich daher nicht beziffern. Hinzu tritt ein Vorteil bei den semantisch geprägten Aufgaben, der sich je nach Ausführungsmodus unterschiedlich artikuliert: In der Direktverarbeitung wendet das Modell sein Kontextwissen unmittelbar auf jeden einzelnen Datenwert an, was bei der Länder-Vereinheitlichung Spitzenwerte von bis zu 100 Prozent ermöglicht. Im Code-Generierungs-Modus wählten die Modelle für diese Aufgabe hingegen überwiegend das Muster statischer Nachschlagetabellen, wodurch Lücken in der Aufzählung die Genauigkeit begrenzten. Dass Codegenerierung nicht prinzipiell auf solche Aufzählungen beschränkt ist, belegt die unscharfe Duplikaterkennung, bei der generierte Ähnlichkeitsvergleiche die regelbasierte Referenz im besten Lauf knapp übertrafen [6, 7].

Die praktisch größte Stärke ist jedoch die Zerlegbarkeit der Aufgabenstellung, wie sie Abbildung 8.6 zeigt: Bei kleinteilig gestellten Schritten steigt nicht nur das Ergebnisniveau, es verschwinden auch die Unterschiede zwischen Strategien und Modellen vollständig. Begrenzend wirkt demnach nicht die Schwierigkeit der einzelnen Operation, sondern die Zahl der Entscheidungen, die in einem Arbeitsschritt gleichzeitig zu treffen sind und die dabei unausgesprochen bleiben, beides Eigenschaften der Aufgabenformulierung und damit unmittelbar gestaltbar.

Dem stehen vier Risiken gegenüber. Das gravierendste ist das stille Versagen: Der in Skript 8.1 dokumentierte Fall zeigt einen Code, der fehlerfrei durchläuft, schema-konform ausgibt und dennoch nichts bewirkt. Weder Erfolgsquote noch Schema- oder Sichtprüfung hätten ihn aufgedeckt. Sichtbar wurde er allein im Abgleich mit einer Soll-Lösung, die im produktiven Einsatz gerade fehlt. Der beobachtete Fehler ist mit einem inzwischen unpassenden Programmiermuster erklärbar. Ob dessen Verwendung auf die Zusammensetzung der Trainingsdaten zurückgeht, wurde nicht untersucht. Jede Produktivnutzung setzt folglich eine Ergebnisprüfung voraus, die nicht auf der Abwesenheit von Fehlermeldungen beruhen darf [6, 7].

Das zweite Risiko ist die Fehlerfortpflanzung, deren Ausmaß der isolierte Durchlauf bezieht: Bis zu 41,8 Prozentpunkte des im Endschrift gemessenen Verlusts sind geerbt. Entscheidend ist dabei nicht der Umfang, sondern die Art der verbliebenen Abweichung: Wenige typfremde Restwerte schaden mehr als viele gleichartige. Die Kennzahl eines einzelnen Schritts zeigt seine Verwendbarkeit im nachfolgenden daher nicht verlässlich an. Das dritte Risiko ist die begrenzte Reproduzierbarkeit, die sich ausgerechnet bei den schwierigsten Aufgaben am stärksten ausprägt und entlang der Kette auf über 45 Prozentpunkte verstärkt. Hinzu tritt die Abhängigkeit von serverseitig gepflegten Modellen, die sich durch die Protokollierung der Versionskennung dokumentieren, nicht aber beheben lässt [5, 26]. Viertens ist die Erwartung zu korrigieren, mehr Kontext im Prompt führe zu besseren Ergebnissen: Dieselbe Few-Shot-Strategie, die bei den unscharfen Duplikaten 55 Prozentpunkte gewinnt, bleibt in sieben von neun Aufgaben trotz dreifachen Token-Aufwands ohne Wirkung [29, 32].

8.12. Einordnung in den Stand der Forschung

Die Ergebnisse bestätigen die in Kapitel 3 referierte grundsätzliche Einschätzung, dass LLMs für Aufgaben der Datenbereinigung und -integration nutzbar sind, präzisieren sie jedoch in einem für die Praxis wesentlichen Punkt. Ergänzend zu den empirischen Untersuchungen von Narayan et al. und Zhang et al. betrachtet die vorliegende Arbeit das Zusammenspiel aus Modell, Aufgabentyp und Ausführungskonfiguration [2, 6]. Die Spreizung zwischen den Aufgaben übersteigt jene zwischen den Modellen um ein Vielfaches, was der verbreiteten Orientierung an Modellranklisten eine begrenzte Aussagekraft für konkrete ETL-Vorhaben bescheinigt [26, 27].

Über die beiden Hypothesen hinaus liefert die verkettete Betrachtung einen Beitrag, der in der Literatur bislang wenig ausgeprägt ist. Die getrennte Messung von eigener Schrittleistung und geerbtem Fehler zeigt, dass ein erheblicher Teil der in Einzelbetrachtungen berichteten Schwächen gar nicht dem bewerteten Schritt zuzurechnen ist: Der Endschrift der Kette, der in der verketteten Ausführung zwischen 49,1 und 69,1 Prozent erreicht, ist mit fehlerfreier Eingabe von zwei der drei Modelle vollständig lösbar. Evaluierungen, die einzelne ETL-Operationen isoliert betrachten, überschätzen die Leistungsfähigkeit in Bezug auf reale Strecken daher ebenso, wie rein verkettete Betrachtungen sie unterschätzen. Erst die Gegenüberstellung beider Betrachtungsweisen erlaubt eine belastbare Zuordnung [2].

Ein zweiter Befund betrifft die wiederholte Messung ganzer Strecken. Der Endschrift derselben Konfiguration schwankte über fünf Wiederholungen um bis zu 45 Prozentpunkte, während im isolierten Durchlauf jeweils derselbe Genauigkeitswert beobachtet wurde. Unter den hier untersuchten Bedingungen kann eine einzelne Ausführung daher zu einer anderen Einschätzung führen als mehrere Wiederholungen. Dieser Befund beschreibt die vorliegende Strecke und begründet die Empfehlung, ihre Schwankungen bei einer praktischen Erprobung mitzuerheben.

Ein dritter Beitrag betrifft die Gestaltung von Bewertungsaufgaben selbst. Die zusammengesetzte Aufgabe der Faktormatrix ergab für sämtliche Verfahren einschließlich des regelbasierten einen Wert nahe null und wirkte damit wie ein starker Negativbefund über die Kompositionsfähigkeit von LLMs. Die gezielte Wiederholung derselben

Verarbeitung mit ausdrücklich benannten Teilschritten und einer aus den Eingabedaten erreichbaren Soll-Lösung hob dieses Ergebnis auf bis zu 75,8 Prozent. Ein Nullbefund, den alle Verfahren gleichermaßen erreichen, ist folglich zunächst als Hinweis auf die Aufgabengestaltung und erst danach als Aussage über die geprüften Verfahren zu lesen, eine Vorsichtsregel, die für die wachsende Zahl von Vergleichsstudien in diesem Feld unmittelbar praktisch ist [7].

Auf dieser Grundlage lassen sich im folgenden Kapitel 9 konkrete Empfehlungen für die Auswahl zwischen klassischem, KI-gestütztem und hybridem Vorgehen je Aufgabentyp ableiten, wie sie FF1 einfordert, ebenso wie Empfehlungen zur Gestaltung der Prompts im Sinne von FF2.

8.13. Beantwortung der Forschungsfragen und Hypothesen

FF1: Die erste Forschungsfrage richtete sich auf die Eignung aktueller LLMs für die Bearbeitung typischer ETL-Teilaufgaben im Vergleich zu einer klassischen, regelbasierten Pipeline, auf die Abhängigkeit dieser Eignung von Aufgabentyp und Ausführungsform sowie auf das Vorgehen, das die Ergebnisse je Aufgabentyp nahelegen.

Auf den ersten Teil der Frage lautet die Antwort, dass die Eignung nicht pauschal, sondern ausschließlich je Aufgabentyp und Ausführungsmodus zu beurteilen ist. Für die sechs formal beschreibbaren Aufgaben erreichen die kommerziellen Modelle in ihren besten Läufen das Niveau der regelbasierten Pipeline, ohne dieses zu übertreffen. Ihr Beitrag liegt dort nicht im Ergebnis, sondern darin, dass die Verarbeitungsvorschrift nicht zuvor formuliert werden musste. Bei der Länder-Vereinheitlichung übertrifft die Direktverarbeitung im besten Lauf die regelbasierte Referenz, bei den unscharfen Duplikaten gelingt dies auch einzelnen Code-Generierungen. Diese kleinen Vorteile beziehen sich auf die gewählte einfache Vergleichspipeline. Für die zusammengesetzte Aufgabe versagen alle untersuchten Ansätze einschließlich der regelbasierten Pipeline, solange sie ungeteilt gestellt wird. Die in H1 formulierte Erwartung ist damit in ihrem Kern bestätigt: starke Abhängigkeit vom Aufgabentyp, Ebenbürtigkeit bei formalisierbaren Aufgaben, Zuverlässigkeitsverlust bei zunehmender struktureller Komplexität, kein durchgängig überlegener Ansatz. In einem Teilaspekt ist sie jedoch einzuschränken. Der erwartete Vorteil bei semantischen Aufgaben tritt nicht durchgängig ein, sondern hängt von Aufgabe, Ausführungskonfiguration und Vergleichsverfahren ab.

Zum zweiten Teil der Frage, der Abhängigkeit von der Ausführungsform, liefern die Ergebnisse Hinweise auf einen Einfluss des Modus, dessen Größe jedoch nicht unabhängig von Aufgabenformulierung und Referenz bestimmt werden kann. Die Direktverarbeitung beherrscht die semantische Einzelfallentscheidung besser, während die Code-Generierung die arithmetischen Operationen über viele Datensätze zuverlässiger ausführt. Der Zuschnitt wirkt ebenfalls relevant, der Vergleich der ursprünglichen und der expliziten zusammengesetzten Aufgabe bleibt wegen der gleichzeitig veränderten Referenz explorativ. Hypothese H1a wird durch diese Befunde daher nicht allein anhand des 0,1-zu-100-Prozent-Vergleichs bestätigt.

Auf den dritten Teil der Frage folgt die Empfehlung eines hybriden Vorgehens, das in Abschnitt 9.3 ausgeführt ist. Es sieht regelbasierte Ausführung für die syntaktisch beschreibbaren Schritte vor, ein Modell im Direkt-Modus für die wenigen semantisch geprägten Schritte und regelbasierte oder codegenerierte Ausführung für die abschließende Aggregation. Alle Schritte sind dabei kleinteilig zugeschnitten und werden einzeln inhaltlich geprüft. Für die untersuchte Strecke bedeutet dies, dass sieben der neun Schritte regelbasiert auszuführen wären und lediglich zwei ein Modell benötigen. Ein durchgängiger LLM-Einsatz ist auf Grundlage dieser Ergebnisse nicht zu empfehlen, ein durchgängiger Verzicht ebenso wenig. Diese hybride Variante ist aus den gemessenen Befunden abgeleitet, nicht als Ganzes implementiert und gemessen. Sie ist eine begründete Empfehlung, kein Ergebnis.

FF2: Die zweite Forschungsfrage richtete sich auf den Einfluss der Prompting-Strategie auf die Ergebnisqualität und auf die daraus abzuleitende Empfehlung.

Der Einfluss ist erheblich, aber ungleich verteilt und stark vom Aufgabenzuschnitt abhängig. Bei sieben der neun Aufgaben wurde in den ausgewerteten Mittelwerten der untersuchten Vorlagen kein Unterschied beobachtet. Bei den beiden anspruchsvollen Entdoppelungsaufgaben entschied die Wahl über bis zu 55 Prozentpunkte. Wo die Prompt-Gestaltung überhaupt wirkt, ist ihr Gewicht demnach mit dem der Modellwahl vergleichbar, nur wirkt sie eben selten.

Die Empfehlung, die daraus folgt, betrifft weniger die Wahl der Strategie als die Reihenfolge der Entscheidungen. Wird dieselbe Verarbeitung in angemessen zugeschnittenen Gruppen statt als überladene Einzelaufgabe gestellt, liegen alle drei Strategien zwischen 98,8 und 99,4 Prozent, und die Frage nach der besten Strategie erübrigt sich. Zuerst ist daher die Aufgabe zu zerlegen. Bleibt danach eine Aufgabe übrig, die sich nicht weiter teilen lässt, gelten die beiden belegten Zuordnungen: Beispiele bei Ähnlichkeitsentscheidungen, schrittweise Herleitung bei mehrstufigen Auswahlregeln, im Übrigen der einfachste und billigste Prompt. Zusätzlicher Kontext sollte an der Aufgabe ausgerichtet und empirisch geprüft werden. In der Entwicklungsphase beobachtet beispielsweise die Angabe der Eingabe-Datentypen ein stilles Versagen der Spaltenauswahl. Ob vorbeugend ergänzter Kontext ebenfalls nützt, wurde nicht gesondert untersucht.

Hypothesen: Die drei in Abschnitt 4.1 formulierten Hypothesen lassen sich damit abschließend beurteilen.

Hypothese H1 gilt damit in ihrem Kern als bestätigt und in einem Teilaspekt als eingeschränkt bestätigt. Bestätigt ist die starke Abhängigkeit vom Aufgabentyp, die Ebenbürtigkeit bei formal beschreibbaren Aufgaben, der Verlust an Zuverlässigkeit mit zunehmender struktureller Komplexität sowie die Aussage, dass kein Ansatz über alle Aufgaben hinweg überlegen ist. Der erwartete Vorteil bei semantisch geprägten Aufgaben trat hingegen nur eingeschränkt ein: Bei den unscharfen Duplikaten übertrifft die Code-Generierung die Regel um knapp einen Prozentpunkt, bei der Länder-

Vereinheitlichung bleibt sie hinter ihr zurück und wird erst von der Direktverarbeitung übertroffen. Der Vorteil ist damit real, aber an den Ausführungsmodus gebunden.

Genau diese Bindung ist Gegenstand von Hypothese H1a. Für den Ausführungsmodus liefert die erreichbare explizite Variante einen Hinweis auf unterschiedliche Stärken von Code-Generierung und Direktverarbeitung, aber keinen kausal isolierten Beleg, weil Prompt und Referenz gegenüber der ursprünglichen Fassung verändert wurden. Auch der Vergleich der Zuschnitte ist wegen dieser gleichzeitigen Änderungen explorativ. Die ergänzenden Betrachtungsebenen liefern damit wichtige Hinweise für die Gestaltung weiterer Experimente, bestätigen H1a jedoch nicht allein anhand des 0,1-zu-100-Prozent-Vergleichs [1, 2].

Hypothese H2 wird durch die untersuchten Vorlagen nicht als allgemeiner Zusammenhang gestützt. Unter den kommerziellen Modellen liegt die beispielgestützte Vorlage im Mittel 6,1 Prozentpunkte vor Zero-Shot, dieser Vorsprung konzentriert sich jedoch weitgehend auf eine Aufgabe. Daraus lässt sich weder eine allgemeine Überlegenheit beispielgestützten Promptings noch ein allgemeines Ausbleiben seiner Wirkung ableiten. Insbesondere wurden weder verschiedene Beispielauswahlen noch mehrere Beispielszahlen verglichen. Auch ein abnehmender Grenznutzen wurde nicht durch eine abgestufte Versuchsreihe isoliert. Die Ergebnisse beziehen sich deshalb auf die drei dokumentierten Vorlagen und die jeweiligen Aufgabenbedingungen.

Reproduzierbarkeit: In gut einem Fünftel der Konfigurationen der Faktormatrix wurden in den zwei Wiederholungen unterschiedliche Genauigkeitswerte beobachtet. Größere Schwankungen traten besonders bei den Duplikataufgaben und im Endschritt der verketteten Verarbeitung auf. Die Messungen deuten damit auf eine Wechselwirkung zwischen der Variation einzelner Modellausgaben und der Verarbeitung dieser Ausgaben in nachfolgenden Schritten hin. Sie erlauben weder eine kausale Bewertung der Temperatur noch eine allgemeine Stabilitätsgarantie. Versionierter Code, dokumentierte Eingabestände und wiederholte Referenzläufe unterstützen eine kontrollierte Nutzung.

9. Handlungsempfehlungen für die Praxis

Die in Abschnitt 8.13 gegebenen Antworten auf die Forschungsfragen lassen sich unmittelbar in Empfehlungen für die Gestaltung KI-gestützter ETL-Strecken überführen. Dieses Kapitel leitet sie aus den Befunden ab und ordnet sie den beiden Fragen zu. Sie werden entlang dreier Entscheidungen entwickelt, die in einem Vorhaben nacheinander zu treffen sind: ob ein LLM für eine gegebene Aufgabe überhaupt das geeignete Mittel ist (Abschnitt 9.1), wie Modell, Ausführungsmodus und Prompt zu wählen sind (Abschnitt 9.2) und wie sich beide Verfahrensarten in einer Gesamtarchitektur verbinden lassen (Abschnitt 9.3). Für alle gilt vorab dieselbe Einschränkung, die Abschnitt 10.2 ausführt: Die Empfehlungen sind Ausgangspunkte für eine eigene Erpro-

bung. Weder die konkreten Zahlenwerte noch die vermuteten Wirkungszusammenhänge sind ohne Prüfung auf andere Daten und Systeme übertragbar.

9.1. Eignung von LLMs für unterschiedliche ETL-Szenarien

Der über alle Untersuchungsebenen hinweg stabilste Befund dieser Arbeit ist, dass die Eignung eines LLM weit stärker von der Art der Aufgabe abhängt als von der Wahl des Modells. Zwischen dem besten und dem schwächsten Cloud-Modell liegen rund acht Prozentpunkte, zwischen der am besten und der am schlechtesten gelösten Aufgabe hingegen nahezu hundert. Die erste Frage in einem ETL-Vorhaben lautet daher nicht, welches Modell einzusetzen ist, sondern für welche Teilaufgaben sich der Einsatz überhaupt lohnt [5, 26].

Für diese Entscheidung hat sich in den Ergebnissen eine einzige Unterscheidung als tragfähig erwiesen: ob die Verarbeitungsvorschrift vollständig vorab formulierbar ist oder nicht. Aufgaben der ersten Art (das Entfernen von Leerzeichen, das Vereinheitlichen bekannter Datumsformate, das Entfernen exakter Duplikate, die schlüsselbasierte Entdoppelung, Typkonvertierungen, das Berechnen abgeleiteter Spalten) werden von einer regelbasierten Implementierung definitionsgemäß korrekt gelöst, deterministisch, in Sekundenbruchteilen und ohne laufende Kosten. Ein LLM erreicht hier bestenfalls dasselbe Ergebnis und bringt zusätzlich Nichtdeterminismus, Kosten und Prüfaufwand mit sich. Sein Beitrag liegt bei diesen Aufgaben ausschließlich in der *Erstellung* der Regel, nicht in ihrer Ausführung, ein Unterschied, der in der Diskussion über KI-gestützte Datenverarbeitung häufig verwischt wird [17, 45].

Aufgaben der zweiten Art sind dadurch gekennzeichnet, dass die Vorschrift eine Aufzählung erfordern würde, die niemand vollständig angeben kann. Die Vereinheitlichung der Länderangaben ist der klarste Fall: Eine Zuordnungstabelle erfasst nur, was zuvor jemand eingetragen hat. Die regelbasierte Vergleichspipeline scheiterte an genau drei Schreibweisen, nämlich „Holland“, „England“ und dem Tippfehler *Deutshcland*. Genau hier liegt der originäre Beitrag des Modells, das erschließt, dass „Holland“ umgangssprachlich für die Niederlande steht, ohne dass diese Gleichsetzung je hinterlegt wurde. Ähnlich, wenn auch weniger ausgeprägt, liegt der Fall bei der unscharfen Duplikaterkennung, wo die Code-Generierung im besten Lauf die regelbasierte Lösung übertrifft: Die Entscheidung über zwei konkrete Datensätze ist einem Modell unmittelbar zugänglich, während ihre Verallgemeinerung zu einem Schwellenwert Information vernichtet.

Diese Zuordnung bleibt jedoch unvollständig, solange sie den Ausführungsmodus außer Acht lässt, denn der Vorteil bei semantischen Aufgaben tritt nur dann ein, wenn das Modell sein Wissen auf den einzelnen Wert anwenden kann. In generierten Code gefasst, fiel dieselbe Aufgabe auf das Niveau der regelbasierten Lösung zurück, weil die erzeugten Skripte ebenfalls unvollständige Zuordnungstabellen verwendeten. Andere Formen generierter Programme sind damit nicht ausgeschlossen. Die Eignungsfrage ist deshalb nicht mit „LLM oder Regel“ beantwortet, sondern erst mit der Angabe des Modus. Tabelle 9.1 fasst sie zusammen.

Tabelle 9.1.: Empfohlenes Verfahren je Aufgabentyp auf Grundlage der Ergebnisse aus Kapitel 8. Die letzte Spalte nennt den jeweils tragenden Messwert.

Aufgabentyp	Merkmal	Empfehlung	Beleg
Syntaktisch regelbar	Vorschrift vollständig aufzählbar	Regelbasiert ausführen, LLM allenfalls zur Erstellung des Codes	Regelbasiert 100 %, bestes LLM ebenfalls 100 %
Semantisch, wertweise	Entscheidung je Einzelwert, Verallgemeinerung verlustbehaftet	LLM im Direkt-Modus, gezielt als einzelner Schritt	Ländercodes: Direkt 97,1–100 % gegen Code-Generierung 94,8 %
Rechnerisch aggregierend	viele Datensätze, exakte Arithmetik	Regelbasiert oder LLM im Code-Generierungs-Modus, Direkt-Modus ungeeignet	Abgeleitete Spalten: Code-Generierung 100 %, Direkt-Modus am Ausgabelimit abgebrochen
Zusammengesetzt	mehrere Entscheidungen in einem Schritt	In Einzelschritte zerlegen, dann je Schritt entscheiden	Einzelaufgabe 0,1 %, zerlegt vollständig gelöst, ausdrücklich gegliedert bis 75,8 %

Die letzte Zeile benennt den folgenreichsten Gestaltungshebel. Die zusammengesetzte Aufgabe aus Verknüpfung, Bereinigung und Aggregation löste als Einzelaufgabe kein Modell. Zerlegt und mit fehlerfreier Eingabe lösten dieselben Modelle sie vollständig, ausdrücklich in Teilschritte gegliedert zu bis zu drei Vierteln. Begrenzend wirkt also nicht die Schwierigkeit der einzelnen Operation, sondern die Zahl der gleichzeitig und der unausgesprochen zu treffenden Entscheidungen. Bevor ein Vorhaben ein Modell auswählt oder einen Prompt optimiert, sollte es seine Aufgabenstellung zerlegen [1, 3].

9.2. Empfehlungen zur Modell- und Prompt-Auswahl

Steht fest, dass ein LLM eingesetzt werden soll, sind drei Festlegungen zu treffen. Sie werden hier in der Reihenfolge ihrer Wirkung behandelt, die der verbreiteten Aufmerksamkeitsverteilung genau entgegengesetzt ist: Der Ausführungsmodus wirkt am stärksten, die Prompt-Gestaltung deutlich, die Modellwahl am wenigsten.

Ausführungsmodus: Die Wahl zwischen Code-Generierung und Direktverarbeitung ist keine technische Detailfrage, sondern bestimmt, welche Fähigkeit des Modells überhaupt zur Wirkung kommt. Für den Regelbetrieb ist der Code-Generierungs-Modus vorzuziehen, und zwar aus drei unabhängigen Gründen. Er ist um mehr als eine Größenordnung günstiger, 0,117 bis 0,216 US-Dollar gegenüber 1,14 bis 1,35 für dieselbe Strecke auf der halben Datenmenge. Er erzeugt ein prüfbares, versionierbares Artefakt, das sich einmal abnehmen und danach unverändert ausführen lässt, während die Direktverarbeitung bei jedem Lauf neu und potenziell anders entscheidet. Bei festem Prompt ist sein Modellaufruf weitgehend vom Datenvolumen entkoppelt, Laufzeit

und Speicherbedarf des erzeugten Codes hängen dagegen weiterhin von Datenmenge und Algorithmus ab, während der Direkt-Modus durch das Ausgabefenster begrenzt ist und im Rahmen dieser Arbeit eine Halbierung des Datensatzes erzwang. Der Direkt-Modus ist demnach nicht als durchgängiger Verarbeitungsmodus zu empfehlen, sondern gezielt für einzelne, semantisch geprägte Schritte, dort allerdings mit messbarem Gewinn.

Prompt-Gestaltung: Hier ist die wichtigste Empfehlung eine über die Reihenfolge: Der Zuschnitt der Aufgabe ist vor dem Prompt zu optimieren, nicht danach. Die in Abschnitt 8.7.1 berichtete Gegenüberstellung zeigt, dass der auffälligste Strategieunterschied der Faktormatrix, 53 gegenüber 72 Prozent bei der Duplikaterkennung, vollständig verschwindet, sobald dieselben Schritte in angemessenem Zuschnitt und mit sauberer Eingabe gestellt werden: Dort liegen alle drei Strategien zwischen 98,8 und 99,4 Prozent. Beispiele und schrittweise Herleitung helfen einem Modell, eine überladene Aufgabe zu bewältigen. Sie werden entbehrlich, sobald die Aufgabe nicht mehr überladen ist. Wer zuerst den Prompt optimiert, arbeitet daher an einem Symptom.

Für verbleibende Teilaufgaben liefern die vorliegenden Messungen Anhaltspunkte zur Auswahl einer Prompt-Vorlage: Bei den unscharfen Duplikaten erreichte die beispielgestützte Vorlage 82,0 gegenüber 26,6 Prozent für Zero-Shot. Bei der schlüsselbasierten Entdoppelung fiel die Vorlage mit expliziter Schrittanleitung günstig aus. Diese Beobachtungen gelten für die konkret eingesetzten Vorlagen und Wiederholungen. Sie ersetzen weder einen Vergleich alternativer Beispiele noch eine Erprobung auf neuen Daten. Wo kein Vorteil beobachtet wird, ist die einfachere Vorlage ein naheliegender Ausgangspunkt.

Zusätzlicher Kontext sollte gezielt ausgewählt und sein Nutzen geprüft werden. Die Angabe der Eingabe-Datentypen behob in der Entwicklungsphase beispielsweise ein still-les Versagen der Spaltenauswahl. Daraus folgt jedoch nicht, dass Kontext ausschließlich nach einem beobachteten Fehler ergänzt werden darf. Auch vorausschauend formulierte Anforderungen können sinnvoll sein, ihr zusätzlicher Nutzen wurde hier nicht separat erhoben. Die Ergebnisse sprechen für eine aufgabenbezogene Auswahl, nicht für eine allgemeine Rangfolge der Prompting-Verfahren.

Modellwahl: Die drei untersuchten Cloud-Modelle liegen so eng beieinander, dass sich eine allgemeine Empfehlung nicht begründen lässt. Auch die Kosten und Antwortzeiten unterscheiden sich nur moderat. Sinnvoll ist stattdessen eine Auswahl anhand der konkret vorliegenden Aufgabenart, wobei die Modelle in ihren Fehlermustern durchaus unterscheidbar sind. In der gruppierten Strecke, also bei angemessenem Aufgabenzuschnitt, verschwindet auch dieser Unterschied: Dort liegen alle drei zwischen 99,0 und 99,3 Prozent.

Das lokale Modell wurde nur in einer Konfiguration untersucht: als Llama-Modell mit acht Milliarden Parametern und begrenztem Kontextfenster auf der verfügbaren Hardware. Der beobachtete Rückstand lässt sich deshalb nicht eindeutig in Beiträge von Modell, Parametrisierung und Hardware zerlegen und ist keine Aussage über lokale Modelle insgesamt. Jellyfish zeigt, dass auch die aufgabenspezifische Anpassung lokaler Modelle eine relevante Vergleichsdimension ist [9]. Für eine praktische Aus-

wahl sollten geeignete lokale Konfigurationen auf den eigenen Aufgaben geprüft und Anschaffungs-, Betriebs- und Prüfkosten mitberücksichtigt werden.

Als Ausgangspunkt einer solchen Erprobung bieten sich klar abgegrenzte Aufgaben und ein vorab festgelegtes Versuchslimit an. Die beobachteten Bestwerte zeigen, dass die untersuchte lokale Konfiguration einzelne Aufgaben vollständig lösen konnte. Sie belegen jedoch weder ein allgemeines Aufgabenverständnis noch, dass größere Hardware allein die übrigen Fehler beheben würde.

Versionsstand: Die kommerziellen Modelle werden serverseitig gepflegt und lassen sich weder einfrieren noch nach einer Aktualisierung wiederherstellen. Wer eine gemessene Konfiguration produktiv einsetzt, sollte deren Versionskennung protokollieren und die Referenzläufe nach jeder Modellaktualisierung wiederholen.

Messung statt Übernahme: Keine dieser Empfehlungen ersetzt die Messung im eigenen Kontext, und die Anlage dieser Untersuchung zeigt, welche Anforderungen eine solche Messung erfüllen muss. Eine Konfiguration ist an Daten zu bewerten, gegen die sie nicht justiert wurde (Abschnitt 7.1), und mehrfach auszuführen: Der Endschnitt derselben Konfiguration schwankte über fünf identische Wiederholungen zwischen 27,3 und 72,7 Prozent (Abschnitt 8.5). Wer auf dieser Grundlage eine einzelne Messung zur Entscheidung heranzieht, wählt zu einem erheblichen Teil zufällig.

9.3. Hybride Architekturen

Aus den vorstehenden Empfehlungen ergibt sich, dass weder ein rein regelbasiertes noch ein rein KI-gestütztes Vorgehen der jeweils anderen Variante über alle Aufgaben hinweg überlegen ist. Die naheliegende Konsequenz ist eine Architektur, die beide Verfahren nicht als Alternativen, sondern als Bausteine derselben Strecke behandelt und je Schritt dasjenige einsetzt, dessen Stärken zur Aufgabe passen [17, 40].

Der erste Baustein ist die bereits begründete Zerlegung. Sie ist hier nicht nur eine Frage der Übersichtlichkeit, sondern die Voraussetzung dafür, dass je Schritt überhaupt eine Verfahrenswahl möglich wird, und eine mögliche Verbesserung der Bearbeitbarkeit, zusätzlicher Spezifikations- und Prüfaufwand ist dabei zu berücksichtigen.

Der zweite Baustein ist die Zuordnung nach Tabelle 9.1: regelbasierte Ausführung für die syntaktisch beschreibbaren Schritte, ein Modell im Direkt-Modus für die wenigen semantisch geprägten Schritte, und die abschließende Aggregation wieder regelbasiert oder über generierten Code. Für die untersuchte Strecke bedeutet dies konkret, dass sieben der neun Schritte regelbasiert auszuführen wären und lediglich die Länder-Vereinheitlichung sowie die unscharfe Duplikaterkennung ein Modell benötigen. Eine Beschränkung der Modellaufrufe auf diese Schritte könnte die Modellgebühren reduzieren. Die tatsächlichen Kosten und die Gesamtqualität der hybriden Variante wurden jedoch nicht gemessen.

Der dritte und wichtigste Baustein ist die Prüfung nach jedem Schritt, die unmittelbar aus dem Befund zur Fehlerfortpflanzung folgt. Deren Tücke liegt darin, dass die verursachenden Schritte für sich betrachtet unauffällig wirken: Sie erreichten durch-

weg über 96 Prozent, und dennoch genügten die wenigen falsch gewählten Duplikat-Repräsentanten, um das Endergebnis um mehr als ein Drittel zu verschieben. Entscheidend ist zudem, dass diese Prüfung nicht auf der Abwesenheit von Fehlermeldungen beruhen darf, wie der Fall des stillen Versagens aus Abschnitt 8.11 zeigt. Zu prüfen sind daher inhaltliche Eigenschaften des Ergebnisses, etwa dass die Länderspalte ausschließlich gültige zweistellige ISO-Codes oder den ausdrücklich erlaubten Wert `UNKNOWN` enthält, dass die Datensatzanzahl im erwarteten Bereich liegt oder dass keine Bestellung ohne zugehörigen Kunden zurückbleibt [7, 16].

Der vierte Baustein betrifft die kontrollierte Wiederverwendung geprüfter Artefakte. Im Code-Generierungs-Modus kann der geprüfte Code versioniert werden, statt ihn bei jeder Ausführung neu zu erzeugen. Bei einem semantischen Direkt-Schritt können bereits geprüfte Zuordnungen für wiederkehrende Werte gespeichert werden. Neue oder veränderte Eingaben benötigen weiterhin eine Prüfung. Dieses Vorgehen begrenzt wiederholte Generierung und erleichtert die Nachvollziehbarkeit, garantiert aber keine Fehlerfreiheit bei veränderten Daten.

Diese vier Bausteine sind zugleich unter dem Gesichtspunkt des Ressourceneinsatzes zu lesen, wie ihn Abschnitt 8.9 auswertet. Eine Strecke, die ein Modell nur für zwei von neun Schritten benötigt, verbraucht weniger Rechenleistung, verursacht geringere Nutzungsgebühren und überträgt weniger Daten an externe Anbieter als eine durchgängig modellgestützte. Der vierte Baustein, die Wiederverwendung geprüfter Artefakte, wirkt in dieselbe Richtung, weil er die wiederholte Generierung desselben Codes vermeidet. Nachhaltigkeit und Ergebnisqualität führen bei dieser Architektur folglich zu derselben Gestaltungsentscheidung, was ihre Umsetzung erleichtert: Es ist kein Zielkonflikt aufzulösen [26, 32].

Diese Aufteilung beschreibt zugleich die realistische Rolle von LLMs in der Datenintegration, wie sie sich aus den Ergebnissen dieser Arbeit ergibt. Sie ersetzen die klassische Verarbeitung nicht, sondern verschieben die Grenze dessen, was ohne vorherige vollständige Spezifikation automatisierbar ist. Ihr Beitrag liegt in den wenigen Schritten, in denen eine Vorschrift nicht vollständig formulierbar ist, sowie in der Verkürzung der Zeit, die von der Aufgabenstellung bis zu einer lauffähigen ersten Umsetzung vergeht, und beides ist erheblich, ohne die Aufgabe der prüfenden Instanz zu ersetzen [2, 46].

10. Fazit und Ausblick

10.1. Zusammenfassung der Ergebnisse

Die Forschungsfragen und die aus ihnen abgeleiteten Hypothesen werden in Abschnitt 8.13 unmittelbar im Anschluss an die Befunde beantwortet. Dieses Kapitel fasst zusammen, ordnet die Grenzen der Untersuchung ein und benennt den weiteren Forschungsbedarf.

Ziel dieser Arbeit war eine experimentelle Evaluierung, inwieweit sich aktuelle LLMs für ausgewählte ETL-Teilaufgaben eignen. Dazu wurde ein synthetischer Datensatz mit kontrolliert eingebauten Qualitätsproblemen erstellt, neun ETL-Aufgaben in drei Kategorien und drei Schwierigkeitsstufen definiert und eine regelbasierte Vergleichspipeline implementiert, deren Ergebnis als Maßstab dient. Die Modelle wurden auf mehreren Untersuchungsebenen gemessen: in einer vollständig gekreuzten Faktormatrix aus Modell, Prompting-Strategie und Aufgabe, in einer verketteten Pipeline aus neun aufeinander aufbauenden Schritten sowie in zwei Ebenen, die den Zuschnitt der Aufgabenstellung selbst zur Untersuchungsgröße machen. Hinzu kommt ein sonst identischer isolierter Durchlauf, der jedem Schritt fehlerfreie Eingabedaten bereitstellt und damit eigene Leistung von geerbtem Fehler trennt. Ergänzend wurden zwei Ausführungsmodi gegenübergestellt: die Erzeugung von Verarbeitungscode und die unmittelbare Verarbeitung der Daten im Prompt. Sämtliche berichteten Ergebnisse stammen dabei aus einem Datensatz, der zum Zeitpunkt der Festlegung aller Prompts und Auswertungsregeln unbekannt war und anschließend ohne weitere Anpassung mehrfach durchlaufen wurde. Die Entwicklung des Untersuchungsdesigns erfolgte auf einem separaten, nicht ausgewerteten Datensatz.

Über alle Ebenen hinweg dominiert der Aufgabentyp: Welche Teilaufgaben einem Modell übertragen werden, ist für ein ETL-Vorhaben erheblich folgenreicher als die Wahl des Modells. Als zweite Einflussgröße erwies sich der Ausführungsmodus: Bei der Länder-Vereinheitlichung übertraf die Direktverarbeitung im besten Lauf die regelbasierte Referenz, bei unscharfen Duplikaten zeigten auch einzelne Code-Generierungen einen kleinen Vorsprung. Umgekehrt scheiterte die direkte Verarbeitung an der abschließenden Aggregation nahezu vollständig und kostete je Zeile rund das Fünzfache. Die beiden Modi sind folglich nicht besser oder schlechter, sondern für unterschiedliche Aufgabenarten geeignet.

Am stärksten wirkt jedoch der Zuschnitt der Aufgabenstellung, und zwar in beide Richtungen. Die zusammengesetzte Aufgabe aus Verknüpfung, Bereinigung und Aggregation blieb als Einzelaufgabe von jedem Modell und jeder Strategie ungelöst, wurde als Glied einer Kette mit fehlerfreier Eingabe jedoch vollständig korrekt bewältigt und in unzerlegter, aber ausdrücklich in Teilschritte gegliederter Form immerhin zu 75,8 Prozent. Derselbe Eingriff hebt auch den scheinbaren Strategieeffekt auf: Über alle Aufgaben gemittelt liegt Few-Shot bei den kommerziellen Modellen 6,1 Prozentpunkte vor Zero-Shot, doch dieser Vorsprung entsteht nahezu vollständig aus einer einzigen der neun Aufgaben, und in angemessen zugeschnittenen Gruppen liegen alle drei Strategien zwischen 98,8 und 99,4 Prozent. Was in der Einzelaufgaben-Matrix als Wirkung der

Prompt-Gestaltung erscheint, ist demnach zu erheblichem Teil eine Wirkung des Zuschnitts, was die übliche Reihenfolge der Gestaltungsentscheidungen umkehrt: erst zuschneiden, dann formulieren.

Dieselbe Gestaltungsentscheidung entlastet zugleich den Ressourceneinsatz. Für ein identisches Verarbeitungsergebnis spreizt sich der Rechenaufwand um Größenordnungen: Die regelbasierte Ausführung benötigt keinen Modellaufruf, die Code-Generierung 15.000 bis 32.000 Token je Strecke und die Direktverarbeitung, auf die verarbeitete Zeilenzahl normiert, rund das Zwanzigfache davon. Hinzu tritt verworfene Rechenleistung als eigene Aufwandsform, denn rund ein Fünftel der Läufe der Faktormatrix brach mit einem Programmfehler ab und beanspruchte die Rechenleistung dennoch vollständig, während die gruppierte Strecke bei geringeren Kosten ausnahmslos verwertbare Ergebnisse erzeugte. Die abgeleitete hybride Architektur, die ein Modell nur für zwei der neun Schritte benötigt, ist deshalb nicht nur die genauere, sondern auch die ressourcenschonendere und datensparsamere Variante. Ein Zielkonflikt zwischen Ergebnisqualität und Nachhaltigkeit tritt in dieser Untersuchung nicht auf (Abschnitt 8.9).

Zwei weitere Befunde betreffen die Bewertung solcher Strecken selbst. Die Trennung von eigener Schrittleistung und geerbtem Fehler zeigt, dass der Endschrift je nach Modell zwischen 30,9 und 41,8 Prozentpunkte verlor, die ihm nicht zuzurechnen waren. Die verursachenden Schritte erreichten dabei durchweg über 96 Prozent und wirkten unauffällig. Dass die regelbasierte Pipeline an derselben Stelle mit 47,0 Prozentpunkten sogar mehr verlor als jedes Modell, verweist die Ursache in die Architektur der Kette. Evaluierungen einzelner ETL-Operationen überschätzen die Leistungsfähigkeit in realen Strecken daher systematisch, während rein verkettete Betrachtungen sie unterschätzen. Ebenso schwankte der Endschrift derselben Konfiguration über fünf identische Wiederholungen zwischen 27,3 und 72,7 Prozent, lieferte im isolierten Durchlauf aber jedes Mal exakt denselben Wert: Dies spricht für eine Verstärkung vorgelagerter Ergebnisunterschiede entlang der untersuchten Kette. Modellbedingte Variation und die Verarbeitung dieser Ausgaben in der Architektur lassen sich anhand dieser Messungen nicht vollständig voneinander trennen.

10.2. Limitationen der Arbeit

Die berichteten Befunde gelten unter Bedingungen, die im Folgenden benannt werden. Sie begrenzen die Reichweite der Aussagen und markieren zugleich, an welchen Stellen eine Fortführung der Untersuchung den größten Erkenntnisgewinn verspricht.

Die erste und gewichtigste betrifft den Datensatz. Er ist synthetisch erzeugt, und seine Qualitätsprobleme entstammen einer bekannten, endlichen Menge von Mustern. Diese Anlage war notwendig, um eine eindeutige Soll-Lösung zu besitzen und die Ansätze überhaupt vergleichbar messen zu können, begünstigt jedoch systematisch die regelbasierte Vergleichspipeline: Deren Zuordnungstabelle und Normalisierungsregel wurden gegen genau diese Muster entworfen und erreichen deshalb Werte, die in einem realen Bestand mit unbekannten Abweichungsmustern nicht zu erwarten wären. Die gemessenen Werte sind deshalb nicht ohne Weiteres auf reale Bestände übertragbar. Ob und in welchem Umfang sich das Verhältnis der Verfahren bei unbekannten

Abweichungsmustern verändert, bleibt offen. Hinzu kommt die Ausgestaltung der Vergleichspipeline bei der unscharfen Duplikaterkennung. Sie bildet einen normalisierten Schlüssel aus Name und E-Mail-Bestandteil, statt auf die für diese Aufgabe etablierten Ähnlichkeitsmaße wie die Levenshtein- oder die Jaro-Winkler-Distanz zurückzugreifen [18, 19]. Der in Abschnitt 8.10 berichtete knappe Vorsprung der Code-Generierung bei dieser Aufgabe ist deshalb nicht als Aussage über den Stand der Technik zu lesen, sondern über eine bewusst einfach gehaltene Referenzimplementierung.

Die zweite Einschränkung betrifft die Größenordnung. Der Datensatz umfasst einige hundert Kunden und einige tausend Bestellungen, was für die inhaltliche Bewertung ausreicht, aber keine Aussage über das Verhalten bei produktionsstypischen Datenmengen erlaubt. Im Code-Generierungs-Modus durchläuft die Datenmenge zwar nicht das Modell, doch Skalierbarkeit, Laufzeit und Speicherbedarf des erzeugten Codes wurden mit diesem kleinen Datensatz nicht nachgewiesen. Für den Direkt-Modus hingegen ist es die bestimmende Grenze, die im Rahmen dieser Arbeit bereits bei wenigen hundert Datensätzen eine Halbierung des Datensatzes erzwang.

Eng damit verbunden ist eine Einschränkung zum lokal betriebenen Modell. Gemessen wurde bei ihm nicht allein Modellqualität, sondern auch die Ausstattung des ausführenden Rechners. Für lokal betreibbare Modelle liegen deshalb nur die Ergebnisse der Faktormatrix vor; die Beschränkung darauf wurde, wie in Abschnitt 7.1 offengelegt, erst während der Durchführung anhand der beobachteten Ausfallquote getroffen und war nicht Teil des ursprünglichen Designs. Ob ein größeres oder aufgabenspezifisch angepasstes Modell auf geeigneter Hardware den Abstand schließt, bleibt offen. Die Fehlerverteilung beschreibt den beobachteten Fehlertyp, isoliert aber nicht dessen Ursache.

Die dritte Einschränkung betrifft die Momentaufnahme. Untersucht wurden vier Modelle zu einem bestimmten Zeitpunkt in einem Feld, das sich in Monatsabständen verändert. Die exakte Versionskennung wurde bei jeder Ausführung protokolliert, sodass die Läufe dokumentiert bleiben. Da die kommerziellen Modelle jedoch serverseitig gepflegt werden, lässt sich ihr Verhalten weder einfrieren noch nach einer Modellaktualisierung wiederherstellen. Die konkreten Zahlenwerte sind daher nicht als dauerhaft gültig zu betrachten. Auch die vorgeschlagenen Erklärungen und Wirkungszusammenhänge müssen bei anderen Daten, Modellen und Ausführungskonfigurationen erneut geprüft werden. Die daraus folgende Empfehlung steht in Abschnitt 9.2.

Die vierte Einschränkung betrifft die Zahl der Wiederholungen. Für die Faktormatrix wurden zwei Wiederholungen je Konfiguration ausgeführt, für die verkettete Pipeline in beiden Formen je fünf und für die beiden Ebenen zum Aufgabenzuschnitt je zwei. Allein die direkten Einzelaufgaben liegen als einmalige Messung vor. Angesichts der beobachteten Streuung (allein der Endschrift schwankte über fünf Wiederholungen um bis zu 45 Prozentpunkte) ist dies eine begrenzte Grundlage für Aussagen über einzelne Konfigurationen. Die Befunde dieser Arbeit sind deshalb bewusst auf Ebene der Muster formuliert, die über Modelle und Aufgaben hinweg wiederkehren, und nicht auf Ebene einzelner Zahlenwerte.

Die Auswertung arbeitet dementsprechend deskriptiv. Besonders auffällig sind Unter-

schiede, die die in den wenigen Wiederholungen beobachtete Streuung deutlich übersteigen, wie der Abstand zwischen den Ausführungsmodi bei der zusammengesetzten Aufgabe von mehr als einer Größenordnung. Der Abstand zwischen den Modellen von acht Prozentpunkten liegt dagegen in der Größenordnung der Streuung selbst und ist entsprechend zurückhaltend interpretiert worden. Die Arbeit liefert damit die Muster und die Größenordnungen, an denen eine statistisch abgesicherte Folgeuntersuchung ansetzen kann.

Die fünfte Einschränkung betrifft die Rolle der prüfenden Instanz. Die Modelle wurden bewusst autonom arbeiten gelassen und ihre Ergebnisse gegen eine bekannte Soll-Lösung bewertet. Diese Anlage isoliert die Leistung der Modelle von der Leistung der Person, die sie einsetzt, und ist die Voraussetzung dafür, den Vergleich mit der regelbasierten Pipeline überhaupt unter gleichen Bedingungen führen zu können. Der in der Praxis dominierende Einsatz als Assistenz einer Fachkraft, die den erzeugten Code liest, korrigiert und verantwortet, wurde damit nicht abgebildet. Die in dieser Arbeit gemessenen Werte beschreiben folglich, was ohne menschliche Nachbearbeitung entsteht, ohne daraus eine garantierte Untergrenze für den assistierten Einsatz abzuleiten. Menschliche Nachbearbeitung kann sowohl Fehler beheben als auch neue Fehler einführen, ihr Qualitäts- und Aufwandseffekt wurde nicht erhoben.

Die sechste Einschränkung betrifft die Nachhaltigkeitsbetrachtung in Abschnitt 8.9. Sie stützt sich auf den Token-Durchsatz als Näherungsgröße für die aufgewendete Rechenleistung. Energieverbrauch und Treibhausgasemissionen wurden nicht gemessen und lassen sich für die kommerziell betriebenen Modelle auch nicht ermitteln, da Auslastung und Hardware der Rechenzentren nicht offengelegt werden [5, 26]. Die berichteten Verhältnisse sind deshalb als Größenordnungen des Rechenaufwands zu lesen und nicht als Verbrauchsbilanz. Für das lokal betriebene Modell wurden Herstellung und Betrieb der Hardware nicht bilanziert, sodass sich aus den Daten keine Aussage darüber ableiten lässt, welche Betriebsform ökologisch vorzuziehen ist.

10.3. Ausblick auf weitere Forschung

Aus den Ergebnissen und ihren Grenzen ergeben sich sechs Anschlussfragen:

Übertragung auf reale Datenbestände: Der Vorsprung der regelbasierten Pipeline beruht darauf, dass ihre Regeln gegen bekannte Abweichungsmuster entworfen wurden. Wie groß er auf einem gewachsenen Bestand mit unbekannten Mustern bleibt, entscheidet über die wirtschaftliche Tragfähigkeit eines LLM-Einsatzes.

Hybride Architektur: Die in Abschnitt 9.3 skizzierte Kombination aus regelbasierten Schritten, gezielt eingesetzten Direkt-Modus-Schritten und einer Prüfung nach jedem Schritt wurde abgeleitet, aber nicht als Ganzes gemessen. Eine Umsetzung ließe sich unmittelbar gegen die hier gemessenen Varianten stellen.

Kennzahl für die Fehlerfortpflanzung: Entscheidend ist nicht der Umfang, sondern die Art: Abweichungen in Schritten mit über 96 Prozent Genauigkeit verschoben das Endergebnis um mehr als ein Drittel, weil sie die Verknüpfungsschlüssel betrafen. Eine solche Kennzahl wäre für die Überwachung wertvoller als die Genauigkeit je Schritt.

Zwischenform der Ausführungsmodi: Trifft das Modell die semantische Entscheidung je Wert und wird deren Ergebnis anschließend deterministisch weiterverarbeitet, etwa als generierte Zuordnungstabelle statt als generierte Aufzählungslogik, ließe sich der Genauigkeitsvorteil des Direkt-Modus möglicherweise ohne dessen Kostenstruktur erreichen.

Assistierter Einsatz: Gemessen wurde, was Modelle ohne menschliche Nachbearbeitung leisten. Praktisch relevanter ist der Prüfaufwand, den eine Fachkraft benötigt, um ein Ergebnis auf ein verantwortbares Niveau zu bringen, gerade angesichts des dokumentierten stillen Versagens.

Energiebilanz der Ausführungsformen: Der Ressourcenvergleich in Abschnitt 8.9 beruht auf dem Token-Durchsatz als Näherungsgröße. Eine Messung des tatsächlichen Energiebedarfs, für lokal betriebene Modelle unmittelbar möglich und für Cloud-Modelle über veröffentlichte Verbrauchskennzahlen näherungsweise, würde die hier berichteten Verhältnisse in eine belastbare Bilanz überführen und die Frage beantworten, ab welchem Verarbeitungsvolumen sich der lokale Betrieb auch ökologisch trägt.

Literatur

- [1] Bilal Khan u. a. „An Overview of ETL Techniques, Tools, Processes and Evaluations in Data Warehousing“. In: *Journal on Big Data* 6 (2024), S. 1–20. ISSN: 25790048. DOI: 10.32604/jbd.2023.046223. URL: <https://www.proquest.com/docview/3200128360/abstract/ABA55C04CFAC48B2PQ/1> (besucht am 08.05.2026).
- [2] Afef Walha, Faiza Ghazzi und Faiez Gargouri. „Data Integration from Traditional to Big Data: Main Features and Comparisons of ETL Approaches“. In: *The Journal of Supercomputing* 80.19 (1. Dez. 2024), S. 26687–26725. ISSN: 1573-0484. DOI: 10.1007/s11227-024-06413-1. URL: <https://doi.org/10.1007/s11227-024-06413-1> (besucht am 08.05.2026).
- [3] Kushvanth Chowdary Nagabhyru. *Bridging Traditional ETL Pipelines with AI Enhanced Data Workflows: Foundations of Intelligent Automation in Data Engineering*. 24. Nov. 2022. DOI: 10.2139/ssrn.5505199. Social Science Research Network: 5505199. URL: <https://papers.ssrn.com/abstract=5505199> (besucht am 08.05.2026). Vorveröffentlichung.
- [4] Chandran Ravi. „ETL (Extract, Transform & Load) Automation“. In: *International Journal of Emerging Trends in Computer Science and Information Technology* 6.1 (12. Feb. 2025), S. 52–55. ISSN: 3050-9246. DOI: 10.63282/3050-9246.IJETCSIT-V6I1P106. URL: <https://www.ijetcsit.org/index.php/ijetcsit/article/view/37> (besucht am 08.05.2026).
- [5] Wayne Xin Zhao u. a. *A Survey of Large Language Models*. 18. März 2026. DOI: 10.48550/arXiv.2303.18223. arXiv: 2303.18223 [cs]. URL: <http://arxiv.org/abs/2303.18223> (besucht am 08.05.2026). Vorveröffentlichung.
- [6] Hossein Hassani und Emmanuel Sirimal Silva. „The Role of ChatGPT in Data Science: How AI-Assisted Conversational Interfaces Are Revolutionizing the Field“. In: *Big Data and Cognitive Computing* 7.2 (Juni 2023), S. 62. ISSN: 2504-2289. DOI: 10.3390/bdcc7020062. URL: <https://www.mdpi.com/2504-2289/7/2/62> (besucht am 08.05.2026).
- [7] Alhassan Mumuni und Fuseini Mumuni. „Automated Data Processing and Feature Engineering for Deep Learning and Big Data Applications: A Survey“. In: *Journal of Information and Intelligence* 3.2 (1. März 2025), S. 113–153. ISSN: 2949-7159. DOI: 10.1016/j.jiixd.2024.01.002. URL: <https://www.sciencedirect.com/science/article/pii/S2949715924000027> (besucht am 08.05.2026).
- [8] Avanika Narayan u. a. „Can Foundation Models Wrangle Your Data?“ In: *Proceedings of the VLDB Endowment* 16.4 (2022), S. 738–746. DOI: 10.14778/3574245.3574258. URL: <https://www.vldb.org/pvldb/vol16/p738-narayan.pdf> (besucht am 07.09.2026).
- [9] Haochen Zhang u. a. „Jellyfish: Instruction-Tuning Local Large Language Models for Data Preprocessing“. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, S. 8754–8782. DOI: 10.18653/v1/2024.emnlp-main.497. URL: <https://aclanthology.org/2024.emnlp-main.497/> (besucht am 07.09.2026).

-
- [10] Panos Vassiliadis. „A Survey of Extract-Transform-Load Technology“. In: *International Journal of Data Warehousing and Mining* 5.3 (2009), S. 1–27. DOI: 10.4018/jdwm.2009070101.
- [11] Chaimae Boulahia u. a. „The Multi-Criteria Evaluation of Research Efforts Based on ETL Software: From Business Intelligence Approach to Big Data and Semantic Approaches“. In: *Evolutionary Intelligence* 17.4 (1. Aug. 2024), S. 2099–2124. ISSN: 1864-5917. DOI: 10.1007/s12065-023-00899-z. URL: <https://doi.org/10.1007/s12065-023-00899-z> (besucht am 08.05.2026).
- [12] Sandeep Rangineni u. a. „A Review on Enhancing Data Quality for Optimal Data Analytics Performance“. In: *International Journal of Computer Sciences and Engineering* 11 (31. Okt. 2023), S. 51–58. DOI: 10.26438/ijcse/v11i10.5158.
- [13] Sarbaree Mishra. „Automating the Data Integration and ETL Pipelines through Machine Learning to Handle Massive Datasets in the Enterprise“. In: *International Journal of Emerging Research in Engineering and Technology* 1.2 (30. Juni 2020), S. 69–78. DOI: 10.63282/3050-922X.IJERET-V1I2P109. URL: <https://ijeret.org/index.php/ijeret/article/view/231> (besucht am 08.05.2026).
- [14] Richard Y. Wang und Diane M. Strong. „Beyond Accuracy: What Data Quality Means to Data Consumers“. In: *Journal of Management Information Systems* 12.4 (1996), S. 5–33. DOI: 10.1080/07421222.1996.11518099.
- [15] Carlo Batini u. a. „Methodologies for Data Quality Assessment and Improvement“. In: *ACM Computing Surveys* 41.3 (Juli 2009), S. 1–52. DOI: 10.1145/1541880.1541883.
- [16] Fakhitah Ridzuan und Wan Mohd Nazmee Wan Zainon. „A Review on Data Quality Dimensions for Big Data“. In: *Procedia Computer Science. Seventh Information Systems International Conference (ISICO 2023)* 234 (1. Jan. 2024), S. 341–348. ISSN: 1877-0509. DOI: 10.1016/j.procs.2024.03.008. URL: <https://www.sciencedirect.com/science/article/pii/S187705092400365X> (besucht am 08.05.2026).
- [17] Pierre-Olivier Côté u. a. „Data Cleaning and Machine Learning: A Systematic Literature Review“. In: *Automated Software Engineering* 31.2 (11. Juni 2024), S. 54. ISSN: 1573-7535. DOI: 10.1007/s10515-024-00453-w. URL: <https://doi.org/10.1007/s10515-024-00453-w> (besucht am 08.05.2026).
- [18] Ahmed K. Elmagarmid, Panagiotis G. Ipeirotis und Vassilios S. Verykios. „Duplicate Record Detection: A Survey“. In: *IEEE Transactions on Knowledge and Data Engineering* 19.1 (Jan. 2007), S. 1–16. ISSN: 1041-4347. DOI: 10.1109/TKDE.2007.250581.
- [19] Olivier Binette und Rebecca C. Steorts. „(Almost) All of Entity Resolution“. In: *Science Advances* 8.12 (25. März 2022), eabi8021. DOI: 10.1126/sciadv.abi8021.
- [20] Muhammad Fakhrul Safitra u. a. „Advancements in Artificial Intelligence and Data Science: Models, Applications, and Challenges“. In: *Procedia Computer Science. Seventh Information Systems International Conference (ISICO 2023)* 234 (1. Jan. 2024), S. 381–388. ISSN: 1877-0509. DOI: 10.1016/j.procs.2024.03.018. URL: <https://www.sciencedirect.com/science/article/pii/S1877050924003752> (besucht am 08.05.2026).

-
- [21] Jasmin Praful Bharadiya. „The Role of Machine Learning in Transforming Business Intelligence“. In: *International Journal of Computing and Artificial Intelligence* 4.1 (1. Jan. 2023), S. 16–24. ISSN: 27076571, 2707658X. DOI: 10.33545/27076571.2023.v4.i1a.60. URL: <https://www.computersciencejournals.com/ijcai/archives/2023.v4.i1.A.60> (besucht am 08.05.2026).
- [22] Zijing Liang u. a. „A Survey of Multimodal Large Language Models“. In: *Proceedings of the 3rd International Conference on Computer, Artificial Intelligence and Control Engineering*. CAICE '24. New York, NY, USA: Association for Computing Machinery, 6. Aug. 2024, S. 405–409. ISBN: 979-8-4007-1694-2. DOI: 10.1145/3672758.3672824. URL: <https://dl.acm.org/doi/10.1145/3672758.3672824> (besucht am 08.05.2026).
- [23] Ashish Vaswani u. a. „Attention Is All You Need“. In: *Advances in Neural Information Processing Systems*. Bd. 30. Curran Associates, 2017, S. 5998–6008. arXiv: 1706.03762.
- [24] Long Ouyang u. a. „Training Language Models to Follow Instructions with Human Feedback“. In: *Advances in Neural Information Processing Systems*. Bd. 35. Curran Associates, 2022, S. 27730–27744. arXiv: 2203.02155.
- [25] Pengfei Liu u. a. „Pre-Train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing“. In: *ACM Comput. Surv.* 55.9 (16. Jan. 2023), 195:1–195:35. ISSN: 0360-0300. DOI: 10.1145/3560815. URL: <https://dl.acm.org/doi/10.1145/3560815> (besucht am 08.05.2026).
- [26] Stanford Institute for Human-Centered Artificial Intelligence. *The 2026 AI Index Report*. 2026. URL: <https://hai.stanford.edu/ai-index/2026-ai-index-report> (besucht am 08.05.2026).
- [27] Artificial Analysis. *Comparison of AI Models across Intelligence, Performance, and Price*. 2026. URL: <https://artificialanalysis.ai/models> (besucht am 08.05.2026).
- [28] Pranab Sahoo u. a. *A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications*. 5. Feb. 2024. arXiv: 2402.07927 [cs.AI]. URL: <https://arxiv.org/abs/2402.07927> (besucht am 17.08.2026).
- [29] Sander Schulhoff u. a. *The Prompt Report: A Systematic Survey of Prompt Engineering Techniques*. 26. Feb. 2025. DOI: 10.48550/arXiv.2406.06608. arXiv: 2406.06608 [cs]. URL: <http://arxiv.org/abs/2406.06608> (besucht am 08.05.2026). Vorveröffentlichung.
- [30] Tom B. Brown u. a. „Language Models Are Few-Shot Learners“. In: *Advances in Neural Information Processing Systems*. Bd. 33. Curran Associates, 2020, S. 1877–1901. arXiv: 2005.14165.
- [31] Jason Wei u. a. „Chain-of-Thought Prompting Elicits Reasoning in Large Language Models“. In: *Advances in Neural Information Processing Systems*. Bd. 35. Curran Associates, 2022, S. 24824–24837. arXiv: 2201.11903.
- [32] Kaiyan Chang u. a. *Efficient Prompting Methods for Large Language Models: A Survey*. 2. Dez. 2024. DOI: 10.48550/arXiv.2404.01077. arXiv: 2404.01077 [cs]. URL: <http://arxiv.org/abs/2404.01077> (besucht am 08.05.2026). Vorveröffentlichung.

-
- [33] Theodoros Rekatsinas u. a. „HoloClean: Holistic Data Repairs with Probabilistic Inference“. In: *Proceedings of the VLDB Endowment* 10.11 (2017), S. 1190–1201. DOI: 10.14778/3137628.3137631.
- [34] Yuliang Li u. a. „Deep Entity Matching with Pre-Trained Language Models“. In: *Proceedings of the VLDB Endowment* 14.1 (2020), S. 50–60. DOI: 10.14778/3421424.3421431.
- [35] Lin Long u. a. „On LLMs-Driven Synthetic Data Generation, Curation, and Evaluation: A Survey“. In: *Findings of the Association for Computational Linguistics: ACL 2024*. Findings 2024. Hrsg. von Lun-Wei Ku, Andre Martins und Vivek Sri-kumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, S. 11065–11082. DOI: 10.18653/v1/2024.findings-acl.658. URL: <https://aclanthology.org/2024.findings-acl.658/> (besucht am 08.05.2026).
- [36] Joao Fonseca und Fernando Bacao. „Tabular and Latent Space Synthetic Data Generation: A Literature Review“. In: *Journal of Big Data* 10.1 (10. Juli 2023), S. 115. ISSN: 2196-1115. DOI: 10.1186/s40537-023-00792-7. URL: <https://doi.org/10.1186/s40537-023-00792-7> (besucht am 08.05.2026).
- [37] Mandeep Goyal und Qusay H. Mahmoud. „A Systematic Review of Synthetic Data Generation Techniques Using Generative AI“. In: *Electronics* 13.17 (Jan. 2024), S. 3509. ISSN: 2079-9292. DOI: 10.3390/electronics13173509. URL: <https://www.mdpi.com/2079-9292/13/17/3509> (besucht am 08.05.2026).
- [38] Zhuoyan Li u. a. „Synthetic Data Generation with Large Language Models for Text Classification: Potential and Limitations“. In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. EMNLP 2023. Hrsg. von Houda Bouamor, Juan Pino und Kalika Bali. Singapore: Association for Computational Linguistics, Dez. 2023, S. 10443–10461. DOI: 10.18653/v1/2023.emnlp-main.647. URL: <https://aclanthology.org/2023.emnlp-main.647/> (besucht am 08.05.2026).
- [39] Naveen Reddy Singi Reddy und Mahitha Adapa. „AI-Driven Data Integration: Transforming Enterprise Data Pipelines through Machine Learning“. In: *Journal of Computer Science and Technology Studies* 7.12 (26. Nov. 2025), S. 110–119. ISSN: 2709-104X. DOI: 10.32996/jcsts.2025.7.12.16. URL: <https://al-kindipublishers.org/index.php/jcsts/article/view/11533> (besucht am 08.05.2026).
- [40] Janardhan Reddy Kasireddy. „The Role of AI in Modern Data Engineering: Automating ETL and Beyond“. In: *ICT for Global Innovations and Solutions*. Hrsg. von Saurav Bhattacharya. Cham: Springer Nature Switzerland, 2026, S. 667–693. ISBN: 978-3-032-02853-2. DOI: 10.1007/978-3-032-02853-2_47.
- [41] Soumen Chakraborty. „Beyond ETL: How AI Agents Are Building Self-Healing Data Pipelines“. In: *Journal of Computer Science and Technology Studies* 7.3 (8. Mai 2025), S. 741–756. ISSN: 2709-104X. DOI: 10.32996/jcsts.2025.7.3.81. URL: <https://al-kindipublishers.org/index.php/jcsts/article/view/9427> (besucht am 08.05.2026).
- [42] Raghav Maddali. „Autonomous AI Agents for Real-Time Data Transformation and ETL Automation“. In: *INTERNATIONAL JOURNAL OF RESEARCH AND ANALYTICAL REVIEWS* 10 (23. Juli 2023), S. 12. DOI: 10.5281/zenodo.15096256.

-
- [43] Jahangir Khan. *Intelligent Anomaly Detection in ETL Workflows Using AI*. 9. Nov. 2025. DOI: 10.2139/ssrn.5729002. Social Science Research Network: 5729002. URL: <https://papers.ssrn.com/abstract=5729002> (besucht am 08.05.2026). Vorveröffentlichung.
- [44] Raghavender Maddali. „Automating Data Quality Assurance Using Machine Learning in ETL Pipelines“. In: *IJLRP - International Journal of Leading Research Publication* 2.6 (8. Juni 2021), S. 1–11. ISSN: 2582-8010. DOI: 10.5281/zenodo.15107533. URL: <https://www.ijlrp.com/research-paper.php?id=1455> (besucht am 08.05.2026).
- [45] Shashank A. „AI-Enhanced ETL Processes: Leveraging Artificial Intelligence for Optimized Data Integration Systems“. In: *Journal Of Multidisciplinary* 5.8 (10. Aug. 2025), S. 219–225. ISSN: 2945-3445. URL: <https://sarcouncil.com/2025/8/ai-enhanced-etl-processes-leveraging-artificial-intelligence-for-optimized-data-integration-systems> (besucht am 08.05.2026).
- [46] Jahangir Khan. *Cognitive ETL: Using AI to Interpret Complex Data Relationships*. 10. Nov. 2025. DOI: 10.2139/ssrn.5728984. Social Science Research Network: 5728984. URL: <https://papers.ssrn.com/abstract=5728984> (besucht am 08.05.2026). Vorveröffentlichung.

Anmerkung zur Nutzung von Künstlicher Intelligenz

Generative KI-Werkzeuge wurden bei der Erstellung dieser Arbeit als Hilfsmittel eingesetzt. Die folgende Übersicht legt offen, in welchen Arbeitsschritten dies geschah und in welchem Umfang.

Werkzeug	Verwendungszweck	Umfang
Claude (Anthropic)	Überarbeitung eigener Textentwürfe, Vorschläge zur Gliederung und Kürzung	durchgehend, jeweils auf Grundlage eigener Entwürfe und nach eigener Prüfung übernommen
Claude (Anthropic)	Formale Prüfung des $\text{\LaTeX}$ -Quelltexts auf Verweise, Beschriftungen und Konsistenz	unterstützend
Codex (OpenAI)	Kritische Prüfung der Argumentation und Quellenverwendung, ergänzende Literaturrecherche sowie Überarbeitung sachlicher Aussagen, insbesondere zu Prompting, Reproduzierbarkeit und Kosten	ausgewählte Abschnitte der Einleitung, Methodendarstellung, Auswertung und Diskussion

KI-Werkzeuge wurden auch zur kritischen Prüfung und Überarbeitung der Argumentation eingesetzt. Die berichteten empirischen Messwerte wurden dabei nicht durch KI erzeugt oder ersetzt, sie stammen aus den dokumentierten Versuchsläufen. Quellenhinweise und Formulierungsvorschläge sind anhand der Originalquellen und der eigenen Untersuchung zu prüfen. Die Verantwortung für die Quellenverwendung, die Interpretation der Ergebnisse und den gesamten Text liegt beim Verfasser.

Davon zu unterscheiden ist der Gegenstand dieser Arbeit selbst. Große Sprachmodelle werden hier als Untersuchungsobjekt eingesetzt, um ihre Eignung für ETL-Aufgaben zu messen. Die dabei erzeugten Skripte und Ergebnistabellen sind Messdaten und nicht Bestandteil des Textes.

A. Anhang

Der Anhang dokumentiert die Bestandteile der Untersuchung, die für die Nachvollziehbarkeit der Ergebnisse erforderlich sind, den Lesefluss des Haupttextes jedoch unterbrochen hätten: die Grundstruktur und ausgewählte Wortlaute der Prompt-Vorlagen (Abschnitt A.1), die Aufgabentexte der neun ETL-Aufgaben (Abschnitt A.2), den Aufgabentext der explizit gegliederten Variante (Abschnitt A.3) sowie den Verweis auf das Repositorium mit Code, Daten und sämtlichen Ergebnisdatensätzen (Abschnitt A.5).

A.1. Prompt-Vorlagen

Die Vorlagen liegen als versionierte YAML-Dateien vor und bestehen jeweils aus einer Systemanweisung und einer Nutzervorlage mit Platzhaltern, die zur Laufzeit durch den Aufgabentext, die Eingabeverzeichnisse und den Ausgabepfad ersetzt werden. Die Zero-Shot-Vorlagen werden wörtlich wiedergegeben, die übrigen Vorlagen teilweise zusammengefasst. Die vollständigen versionierten Vorlagen einschließlich Beispielen sind im Repositorium nach Abschnitt A.5 dokumentiert. Die Vorlagen sind auf Englisch formuliert, als einheitliche Festlegung für diese Untersuchung. Ein Vergleich verschiedener Prompt-Sprachen wurde nicht durchgeführt. Die Aufgabentexte selbst sind deutsch.

Code-Generierung

v1, Zero-Shot: *System:* „You are an experienced data engineer. Respond only with executable Python code that uses pandas. No explanations, no markdown code block, no text outside the code.“

Nutzervorlage: „Write a self-contained Python script for the following ETL task: {task_description}. The input files are located in: {input_dir}. Write the result as a Parquet file to exactly: {output_path}. The script must run without any further input and must produce the output file at the specified path.“

v2, Few-Shot: Systemanweisung wie v1. Die Nutzervorlage stellt der Aufgabe ein vollständig gelöstes Beispiel voran, an dem das Modell Format und Lösungsstil ableitet:

Here is one fully solved ETL task as an example of the expected style. It is a different task from the one you must solve; use it only to understand the format and quality of the expected solution.

— EXAMPLE START —

Task: A file `employees_raw.csv` contains an employee table with the columns `employee_id`, `full_name`, `department`, `annual_salary_eur` and `active`. The data has the following issues:

- `full_name` and `department` contain leading/trailing whitespace,

- some department values are missing (empty),
- annual_salary_eur is stored as text with currency decoration (e.g. "55000", "EUR 60000", "52000 EUR"),
- active is stored as the German text "ja"/"nein".

Clean the table: trim whitespace in text columns, replace missing departments with 'UNKNOWN', convert annual_salary_eur to a float, and convert active to a boolean. Write the result as Parquet to employees_clean.parquet.

Input (employees_raw.csv):

```
employee_id,full_name,department,annual_salary_eur,active
1, Anna Schmidt ,Sales,55000,ja
2,Ben Krüger,,48500,nein
3, Carla Neumann ,Engineering,60000,ja
4,David Fischer,Marketing,52000 EUR,ja
5, Eva Wagner,Engineering,61000,nein
```

Solution:

```
1 import pandas as pd
2
3 df = pd.read_csv("employees_raw.csv")
4
5 for col in ["full_name", "department"]:
6     df[col] = df[col].astype(str).str.strip()
7 df["department"] = df["department"].replace("", "UNKNOWN").
    fillna("UNKNOWN")
8 df["annual_salary_eur"] = (
9     df["annual_salary_eur"].astype(str)
10    .str.replace(r"^\d.,", "", regex=True)
11    .str.replace(", ", ".")
12    .astype(float)
13 )
14 df["active"] = df["active"].str.strip().eq("ja")
15
16 df.to_parquet("employees_clean.parquet", index=False)
```

— EXAMPLE END —

Now solve the following task in the same style:

{task_description}

The input files are located in: {input_dir}. Write the result as a Parquet file to exactly: {output_path}.

The script must run without any further input and must produce the output file at the specified path.

Die Beispielaufgabe ist bewusst keine der neun evaluierten Aufgaben, damit sie keine Musterlösung vorwegnimmt.

v3, Chain-of-Thought: *System:* „You are an experienced data engineer. Respond only with executable Python code (pandas). Record your reasoning as comments (with #) directly in the code. No text outside the code, no markdown.“

Nutzervorlage: „Solve the following ETL task. Proceed step by step: first break the task down into sub-steps (as comments), then implement each step in code. {task_description} ...“

Direktverarbeitung

Die drei Direkt-Vorlagen entsprechen den vorstehenden in Strategie und Beispiel, verlangen jedoch die Ergebnistabelle statt des Codes.

d1, Zero-Shot: *System:* „You are a precise data-processing engine. You receive one or more input tables and a data transformation task. Apply the task directly to the data and return ONLY the resulting table as CSV (comma-separated, with a header row), wrapped in a single `csv` code block. Output every result row. No explanations, no prose, no row index column.“

Nutzervorlage: „Task: {task_description}. Input data (CSV): {data}. Return the COMPLETE resulting table as CSV inside one `csv` code block. Include ALL result rows, no index column, no commentary.“

d2, Few-Shot: Wie d1, ergänzt um dasselbe Beispiel wie v2, jedoch mit der Ergebnistabelle statt des Lösungs_codes als Vorbild.

d3, Chain-of-Thought: Wie d1, ergänzt um die Aufforderung: „Work through it step by step: first identify the individual sub-steps the task consists of, then apply them to the data one after another, so that each sub-step operates on the result of the previous one.“ Da die Antwort ausschließlich die Ergebnistabelle enthalten darf, wird kein sichtbarer Begründungstext erhoben. Ob und wie das Modell die angeforderte Zerlegung intern umsetzt, wird nicht beobachtet.

A.2. Aufgabentexte der neun ETL-Aufgaben

Wiedergegeben ist der Wortlaut, der als `task_description` in die Prompt-Vorlage eingesetzt wird. Für die schema-angereicherte Variante tritt jeweils eine Beschreibung des Zielschemas hinzu. Diese Ergänzungen gehören zur vollständigen Versuchsspezifikation und sind zusammen mit den Vorlagen im Repositorium dokumentiert.

Datenbereinigung

Einfach (Leerzeichen und fehlende Werte): Lies die Datei `customers_raw.csv` ein. Entferne führende und nachfolgende Leerzeichen aus allen Textspalten. Ersetze fehlende Werte in der Spalte `country` durch „UNKNOWN“. Schreibe das Ergebnis als Parquet-Datei `cleaned_customers.parquet`.

Mittel (Datumsformate): Die Spalte `registered_at` in `customers_raw.csv` enthält Datumsangaben in unterschiedlichen Formaten (ISO-Datum, deutsches Format DD.MM.YYYY, US-Format Month DD YYYY und Unix-Timestamps). Normalisiere alle Werte auf das ISO-Format YYYY-MM-DD. Schreibe das Ergebnis als Parquet-Datei.

Schwer (semantische Vereinheitlichung): Die Spalte `country` in `customers_raw.csv` enthält Länder in verschiedenen Schreibweisen und Sprachen (z. B. „DE“, „Deutschland“, „Germany“, „Deutshcland“). Vereinheitliche alle Werte auf den zweistelligen ISO-3166-1-alpha-2-Code (z. B. „DE“). Werte, die sich nicht eindeutig zuordnen lassen, sollen den Wert „UNKNOWN“ erhalten. Schreibe das Ergebnis als Parquet-Datei.

Datentransformation

Einfach (Typkonvertierung): Lies `products_raw.csv` ein. Konvertiere die Spalte `price_eur` von Text (z. B. „19,99“, „€19.99“, „19.99 EUR“) in einen numerischen Wert (Float, Dezimaltrennzeichen Punkt). Konvertiere die Spalte `in_stock` von Text („ja“/„nein“, „true“/„false“, „1“/„0“) in Boolean. Schreibe das Ergebnis als Parquet-Datei.

Mittel (abgeleitete Spalten): Lies `orders_raw.csv` ein. Berechne für jede Bestellzeile eine neue Spalte `total_eur` als `quantity * unit_price_eur`. Extrahiere aus `ordered_at` zusätzlich die Spalten `order_year` (Integer) und `order_month` (Integer, 1–12). Schreibe das Ergebnis als Parquet-Datei.

Schwer (Verknüpfung und Aggregation): Verknüpfe `orders_raw.csv`, `customers_raw.csv` und `products_raw.csv` über die jeweiligen ID-Spalten. Berechne dabei die Daten so weit wie nötig (Typkonvertierungen, fehlende Werte sinnvoll behandeln). Aggregiere anschließend pro Land (`country_code`, ISO-2) und Produktkategorie (`category`) den Gesamtumsatz (`total_revenue_eur = sum(quantity * unit_price_eur)`) und die Anzahl Bestellungen (`order_count`). Sortiere absteigend nach Gesamtumsatz. Schreibe das Ergebnis als Parquet-Datei.

Duplikaterkennung

Einfach (exakte Duplikate): Lies `customers_raw.csv` ein und entferne exakte Duplikate (Zeilen, in denen alle Spaltenwerte identisch sind). Behalte jeweils das erste Vorkommen. Schreibe das Ergebnis als Parquet-Datei.

Mittel (schlüsselbasiert): Entferne in `customers_raw.csv` Zeilen, in denen die `email` mehrfach vorkommt. Bei mehreren Zeilen mit gleicher E-Mail soll die Zeile mit dem jüngsten `registered_at`-Wert behalten werden. Schreibe das Ergebnis als Parquet-Datei.

Schwer (unscharfe Duplikate): In `customers_raw.csv` befinden sich Fuzzy-Duplikate: gleiche Person, aber unterschiedliche Schreibweisen des Namens (Typfehler, unterschiedliche Groß-/Kleinschreibung, mit/ohne Mittelinitial) und unterschiedliche E-Mail-Schreibweisen (Punkte vor dem @, unterschiedliche Groß-/Kleinschreibung). Erkenne diese Duplikate und entferne sie. Behalte jeweils die Zeile mit der vollständigsten Information. Schreibe das Ergebnis als Parquet-Datei.

A.3. Aufgabentext der explizit gegliederten Variante

Dieser Text stellt dieselbe Verarbeitung wie die schwere Transformationsaufgabe, benennt die erforderlichen Teilschritte jedoch ausdrücklich. Abschnitt 8.7.2 wertet die damit erzielten Ergebnisse aus.

Erzeuge aus den drei Roh Tabellen `customers_raw.csv`, `products_raw.csv` und `orders_raw.csv` eine Umsatzauswertung. Führe dazu die folgenden Schritte in dieser Reihenfolge aus:

1. Entferne in `customers_raw.csv` führende und nachfolgende Leerzeichen aus allen Textspalten.
2. Vereinheitliche die Spalte `country` in `customers_raw.csv` auf den zweistelligen ISO-3166-1-alpha-2-Code und lege sie als `country_code` ab. Die Rohwerte liegen in unterschiedlichen Sprachen, als Abkürzungen und mit Tippfehlern vor (z. B. „Deutschland“, „Germany“, „GER“, „Deutschcland“ → „DE“). Werte, die sich nicht eindeutig zuordnen lassen, sowie fehlende Werte erhalten „UNKNOWN“.
3. Entferne aus `customers_raw.csv` Datensätze mit mehrfach vorkommender `customer_id` und behalte jeweils das erste Vorkommen.
4. Konvertiere in `products_raw.csv` die Spalte `price_eur` (Text wie „19,99“, „19.99 EUR“) in einen Float und `in_stock` in einen Boolean.
5. Verknüpfe `orders_raw.csv` mit `customers_raw.csv` über `customer_id` und mit `products_raw.csv` über `product_id`. Alle Bestellzeilen aus `orders_raw.csv` bleiben dabei erhalten.
6. Aggregiere pro Land (`country_code`) und Produktkategorie (`category`) den Gesamtumsatz `total_revenue_eur = sum(quantity * unit_price_eur)`, auf zwei Nachkommastellen gerundet, sowie die Anzahl Bestellungen `order_count`. Die Spalten `quantity` und

unit_price_eur stammen beide aus orders_raw.csv. Der Preis aus products_raw.csv wird für die Umsatzberechnung nicht verwendet. Sortiere absteigend nach total_revenue_eur.

A.4. Laufzeitumgebung und Modellparameter

Tabelle A.1 dokumentiert die für die Ausführung der Versuche eingesetzten Software- und Bibliotheksversionen. Sämtliche Durchläufe erfolgten unter Windows 11 auf einem System mit einer dedizierten Grafikeinheit mit 8 GB VRAM.

Tabelle A.1.: Software- und Bibliotheksversionen der Versuchsumgebung (abgeleitet aus pyproject.toml).

Komponente / Bibliothek	Einsatzzweck	Version
Python	Laufzeitumgebung der Versuche und Skripte	3.12
pandas	Datenverarbeitung in Referenz und generiertem Code	2.2
pyarrow	Parquet-E/A und typgetreue Datenspeicherung	18.0
faker	Deterministische Basisdatengenerierung	30.0
anthropic	Offizielles SDK zur Anbindung von Claude	0.39
openai	Offizielles SDK zur Anbindung von GPT	1.54
google-genai	Offizielles SDK zur Anbindung von Gemini	0.8
ollama	Lokale Modellausführung (Llama)	0.4
pydantic / pydantic-settings	Typsichere Versuchskonfiguration	2.9 / 2.6

Tabelle A.2 fasst die für die vier Modelle wirksamen Sampling- und Reasoning-Einstellungen zusammen. Sie macht anbieterspezifische Einschränkungen transparent, die das Verhalten bei wiederholten Aufrufen beeinflussen.

Tabelle A.2.: Wirksame Sampling- und Reasoning-Konfiguration der vier Modelle über die Ausführungsmodi.

Modell	Temperatur	Reasoning im Direkt-Modus	Besonderheiten
claude-sonnet-5	0,0	deaktiviert (type: "disabled")	Fallback bei Ablehnung von T
gpt-5.6-terra	1,0 (Fallback)	aktiv (nicht deaktivierbar)	$T = 0$ vom Server abgelehnt
gemini-3.6-flash	0,0	gedrosselt (thinking_budget=128)	Budget 0 nicht zulässig
llama3.1:8b	0,0	nicht vorhanden	num_ctx auf 8.192 gedeckelt (VRAM)

A.5. Repositorium

Das Repositorium ist die ergänzende elektronische Dokumentation dieser Arbeit. Es enthält die vollständigen Versuchsvorlagen und Auswertungsartefakte, die folgenden Verweise benennen deren Ablageorte.

Sämtlicher Code, die Erzeugungsvorschrift des Datensatzes, die regelbasierte Vergleichspipeline, die Prompt-Vorlagen sowie alle Ergebnisdatensätze der 711 Modelaufrufe stehen unter <https://github.com/Luis198/Bachelorarbeit> zur Verfügung. Die Struktur folgt den Kapiteln dieser Arbeit:

`project/src/dataset/` Erzeugung des synthetischen Datensatzes einschließlich der Aufgabendefinitionen (Kapitel 5).

`project/src/baseline/` Regelbasierte Vergleichspipeline (Abschnitt 4.6).

`project/src/llm/` **und** `project/prompts/` Anbindung der Modelle und die Prompt-Vorlagen (Abschnitt 6.4).

`project/src/experiments/` Die Ausführungsschichten der einzelnen Untersuchungsebenen (Kapitel 7).

`project/src/evaluation/` Umsetzung der Evaluationskriterien aus Abschnitt 4.5.

`project/data/results_*/` Die Ergebnisdatensätze je Lauf sowie die erzeugten Skripte und Ergebnistabellen, getrennt nach Startwert des Datensatzes.

Die berichteten Ergebnisse stammen sämtlich aus dem Datensatz mit dem Startwert 2 beziehungsweise dessen halbierten Fassung mit dem Startwert 102.